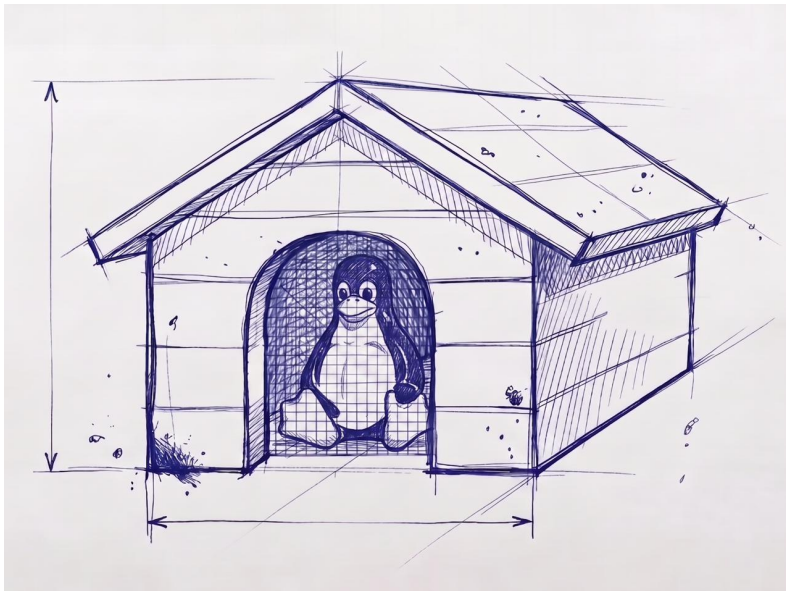


Project Kennel

Volume 2



Mechanism and Construction

Remco van Mook

Schalkhaar, June 2026

0. Reading guide

Project Kennel is a userspace reference monitor for the interactive user level, a way to run code under your own account without granting it your own authority. This is the second of its two volumes; the first set out the design without reference to any operating system, and this one realises that design on Linux. The altitude is inverted on purpose. Where the first volume held above the mechanism, refusing to name a syscall or a kernel feature so that its claims would read the same on any port, this one names them throughout: the kernel facilities the confinement is woven from, the processes that hold and drop privilege, the bus the workload talks across, the crates the trusted base is built in. Every claim here answers a question the first volume left open on purpose, the question of how, on this kernel, a contract stated there is kept.

The volume is in four parts. The first is the machinery: the kernel facilities and how they are woven into a confinement, the processes that build and supervise a kennel and the single one that holds privilege, the binder bus every kennel talks across, the crate topology of the trusted base, and the command surface that drives it. This part has no analogue in the first volume, because it is the substrate the first volume deliberately did not name. The second takes the resource classes the first volume's second part defined and gives each its Linux mechanism, from the filesystem through execution and the network to the local, cryptographic, and image surfaces and the smaller remaining ones. The third is the interplay: how a kennel spawns another, how kennels offer and consume one another's capabilities, and how a service too rich to broker is interposed as a confined instance of itself. The fourth is the language and the tools: the grammar a confinement is written in, the compiler that turns it into the artefact the runtime enforces, the keys that sign it, and the account the monitor keeps. The first volume did not separate the last two: it carried the interplay and the declared grant together, and this volume takes them apart because the mechanism does.

Three citation forms point out of the prose, into two registers and a catalogue. A name such as `construction-by-absence` points into the first volume's principle register (`PRINCIPLES.md`): it names the design posture a mechanism keeps, the con-

tract this volume is here to honour. A name such as `quarantine-the-unsafe` points into this volume's own register (`CONSTRUCTION.md`): it names a construction principle the code itself holds to, a property checkable by opening the repository rather than by reading the design. An identifier such as `T5.1` points into the threat catalogue (`THREATS.md`), shared with the first volume, and names a concrete threat the mechanism in hand closes. Literature is cited by name in brackets and resolved at the end, as in the first volume.

This volume assumes the first. A reader can follow the mechanism here without the design behind it, but the reason a mechanism is shaped the way it is lives in the first volume, and the principle citations are the way back to it. The machinery of the first part is assumed by every chapter after it, so it is the one part meant to be read before the others; a resource chapter then stands largely on its own, with its citations leading back to the design it realises and the threats it answers. The kernel floor is the project floor throughout, kernel 6.10 for Landlock ABI 5, and a mechanism that needs more says so where it is named.

Contents

Project Kennel: Construction Principles	1
Stance	1
Supply	2
Construction	3
Part I The machinery	5
1. Into the machine	6
1.1 What counts as a control	6
1.2 The construction facilities	7
1.3 The interposition facilities	7
1.4 Binder	8
1.5 Privilege and identity	9
1.6 The weave	9
1.7 The kernel floor	10
1.8 The same weave elsewhere	10
2. Process and privilege model	12
2.1 The processes	12
2.2 One privileged component	13
2.3 The factory: privilege bounded to one construction	14
2.4 The trusted init and the drop	15
2.5 Two identities, not one	16
2.6 The unsafe at this layer	16
3. Binder's Node 0	18
3.0 What binder is	18
3.1 From connect to call	19
3.2 The bus each kennel gets	19
3.3 kenneld on node 0	20
3.4 A transaction across the boundary	21
3.5 Control plane, not data plane	21
3.6 The crate and the unsafe	22
4. The trusted base	23

4.1 What counts as trusted	23
4.2 The vendored truth	24
4.3 The empty intersection	24
4.4 The unsafe, quarantined	25
4.5 The shape of the workspace	25
5. The command surface	26
5.1 The shim	26
5.2 Outside the trusted base	27
5.3 The control protocol	27
5.4 The verbs	28
Part II The resource classes	29
6. The filesystem	30
6.1 The view and the access	30
6.2 The constructed home	31
6.3 Writable, but ephemeral	32
6.4 Denials and the pseudo-filesystems	32
6.5 The trust manifest	32
7. Execution	35
7.1 What it gates	35
7.2 The allowlist and no_new_privs	35
7.3 What FS_EXECUTE checks	36
7.4 Writable, and interpreted	37
7.5 Composing the allowlist	38
8. The network	39
8.1 The four modes	39
8.2 The proxy as gateway	40
8.3 The two ACLs	41
8.4 The egress floor and its residuals	42
8.5 Loopback isolation	42
8.6 Binding and the inbound mirror	43
8.7 Shaping the bind	44
9. Local services	45
9.1 The ambient surface	45
9.2 The brokered-connect facade	46
9.3 The session bus is not a socket grant	47
9.4 Asymmetry, and the refused set	48
10. Cryptographic services	51
10.1 The signing oracle	51
10.2 Re-origination	51

10.3 Where re-origination stops	53
10.4 The line, generalised	54
11. OCI images	56
11.1 Booting an image without a daemon	56
11.2 The image as a declared substrate	57
11.3 The overlay	58
11.4 Closure-lock	58
11.5 What holds, and what is waived	60
12. The remaining surfaces	61
12.1 What the workload can observe	61
12.2 What the workload inherits	62
12.3 The terminal	63
12.4 Capabilities, mounts, and the floor	64
Part III The interplay	65
13. Dynamic spawning	66
13.1 The trifecta, split	66
13.2 Request, don't author	66
13.3 The handoff, and the daemon out of the path	68
13.4 Ephemerality, fate-sharing, and the depth bound	69
14. The service mesh	71
14.1 Compile-local, resolve at runtime	72
14.2 Deny-by-default and the broker	72
14.3 The catalogue is the filesystem	73
14.4 Supervision and the service-kennel trust class	74
14.5 The af-unix shape	76
14.6 The mesh bus	76
14.7 The brokered shapes	77
15. The interposed surface	79
15.1 The display server is constructed, not granted	79
15.2 The nested compositor and the brokered leg	80
15.3 The invariant, and where the enforcer runs	81
15.4 Isolation, the host leg, and the trust class	82
15.5 The surfaces ahead	82
Part IV The language and the tools	84
16. The policy compiler	85
16.1 The authored artefact and the enforced artefact	85
16.2 The fold	85
16.3 The signature is the commitment	86
16.4 Lint and the footgun warnings	87

16.5 The compiler is out of the trusted base	87
17. The settled artefact	89
17.1 The canonical body and its signature	89
17.2 What the runtime does with it	90
17.3 A signature is not a safety proof	91
17.4 Provenance and attestation	91
18. The keys	93
18.1 The key and its identity	93
18.2 The three tiers	94
18.3 Rotation and revocation	95
18.4 The honesty bound	96
19. The audit account	97
19.1 The risk report	97
19.2 The audit log	98
19.3 What the account is for, and where it stops	99
Part V Policy reference	101
20. The policy language	102
[audit]	102
[cap]	103
[[consumes]]	104
[dbus]	105
[env]	106
[exec]	107
[fs]	107
[identity]	111
[lifecycle]	111
[[mutable]]	112
[net]	113
[[provides]]	117
[rootfs]	118
[seccomp]	118
[service]	119
[signature]	119
[spawn]	120
[ssh]	121
[trust]	121
[tty]	122
[unix]	122
[unsafe]	123
[workload]	123

Common forms 124

21. Worked examples **125**

base-confined — the floor every kennel stands on 125

interactive (template) — the floor made into a usable human shell . . 132

interactive (policy) — the thin leaf that pins a workload 138

Project Kennel: Construction Principles

These are the postures the code and architecture hold to, sorted in three small categories: Stance, Supply and Construction. Where the design register states what the system is and why, these state how its code is built, under a reader who does not trust the author. They are construction, not design.

Each rests on a section of the coding standards, cited as §N. The standards state the rules; this register states the postures they serve. Where a construction principle is a design principle realised in code, the design slug is named: the two registers meet at the altitude line, posture above and mechanism below.

What is not here is the working discipline: how a claim carries its evidence, how a change earns its place, how the tests are written before the implementation they check, how a contradiction between the substrate and the corpus is surfaced rather than patched. Those govern the people and the process, not the landed artefact, and they live in the coding standards and the standing orders. This register holds only what shows up in the code and the architecture.

Stance

read-by-the-hostile

The reader does not trust the author. Code in the repository is read line by line by people who assume it is hostile until they have satisfied themselves it is not, and the bar it is held to is OpenSSH and libpam: auditable in full, by a stranger, with the author absent to explain it (§1). This is good-boy turned on the code itself. A design that refuses to ask the workload to be trustworthy cannot be implemented by code that asks the same of its reader. So the code is written to be checked, not believed: small enough to read, plain enough to follow, arranged so a reviewer can confirm a property rather than take it on faith. Cleverness that buys a line and costs an hour of reading is a net loss, because the hour is paid by everyone who audits the system and the line is saved once.

built-to-outlast

The code must compile, test, and run the same way on the next long-term-support distribution five years from now (§2). The toolchains are pinned, the build is offline against vendored sources, and release builds are reproducible hash for hash on two independent runners, so that what shipped can be rebuilt and checked long after the machine that built it is gone. A confinement tool an auditor cannot reproduce is one an auditor cannot trust, and trust that cannot be re-established on a fresh machine is trust with an expiry date. The cost of writing against a moving substrate is paid once by the author; the cost of a build that cannot be reproduced is paid by every future reader, so the trade runs one way.

Supply**own-your-supply-chain**

Every dependency is attack surface and audit burden, and the default answer to taking one is no (§5). What the standard library and the existing tree already provide is tried first; the question is not whether a crate exists but whether the thing could be written in a day and maintained without embarrassment. What is taken is pinned to an exact version, vendored as the audited bytes, and checked against an independently computed manifest rather than a hash the registry vouches for, because trust on first use against a registry no one controls is not a defence. This is `do-less` on the supply chain: the smaller the set of things outside the tree the system must trust, the smaller the set that can betray it. A hand-rolled loader that replaces a thousand vendored files is not reinvention; it is the refusal of a dependency whose cost outweighs its saving.

dont-roll-your-own

The opposite error is hubris, and the discipline refuses it as hard. Cryptography, transport security, compression, character-set decoding, the hardened parsers where a subtle bug is an exploit: these are minefields cleared over decades by specialists, and a fresh implementation forfeits the thousands of hours and eyes the existing one already carries (§5.1). Here the tree admits the dependency. The line between this and `own-your-supply-chain` is one question asked plainly: can we audit our own version in full, and would it earn as many eyes as the thing it would replace. Yes to both and the dependency is bloat; no to either and reimplementing is recklessness. The two halves are one rule reading its inputs, which is why a small reviewable loader and a refusal to write our own signature scheme are not a contradiction.

The project's one crypto primitive shows the pair deciding together. A solid security primitive was needed and was never going to be written here, so the only

question was which to depend on, and the answer was the smallest vetted one. Among the candidates, rustls was about 35,000 lines on its own and hundreds of thousands once its dependencies were counted; a sha256 crate was under 400 lines in itself but close to 8,000 with its tree; ed25519 came to about 800 lines all in, and won on total footprint, the least code to read that still does the job. The gap between each pair of figures is the lesson: the size of the thing you came for tells you nothing, because a dependency costs the whole tree it drags in, not the leaf. The direction is worth marking too: ed25519 is in the tree because supply chose the smallest vetted primitive, and the trust material the design register spends is spending what supply admitted, rather than the reverse.

Construction

parse-dont-validate

Every byte that enters from outside is untrusted until parsing has produced a typed value, and the parse is the validation, not a check bolted on after (§10). A typed value is evidence that the checks the type requires have run; raw input has no standing until it has become something the rest of the program is allowed to hold. Configuration is not exempt and is the sharp case, because a file looks like an internal artefact and draws less scrutiny than network bytes while arriving from templates fetched over the wire, deltas hand-written, caches that may have been tampered with, and the very agents the system exists to confine. Unknown fields are rejected, reads are bounded before they begin, and the crossing is visible in the code as a parser call rather than a silent conversion. This is `mediate-use-not-reach` and `construction-by-absence` at the type boundary: a constructed value is a mediated one, and what was never parsed was never admitted.

make-invalid-unrepresentable

Where a value carries security meaning it is a newtype whose constructor does the validation, and where an invalid state is forbidden by invariant it is removed from the type rather than checked at runtime (§11). A forbidden flag is not a boolean someone must remember to test; it is a shape that cannot be built. The constructed type is the proof the check happened, so the proof cannot be skipped without the value failing to exist. This is `construction-by-absence` carried into the type system: the same move that makes a withheld resource absent from a workload's world makes an invalid state absent from the program's, and in both the gain is that there is nothing left to guard, because there is nothing left to be wrong.

errors-not-panics

Shipped code has no unhandled path (§8). There is no `unwrap`, no `panic!`, no `todo!` in anything that ships; every fallible thing returns a typed error specific enough

to act on; and the one permitted `expect` carries, in its message, the invariant that makes it infallible, so the claim of safety is written where a reader can check it. At a trust boundary an error is an expected outcome, not an exceptional one, given a named variant and an audit record rather than a crash. This is `refuse-to-start` in the small: a component that meets a condition it cannot honour stops cleanly and says so, rather than proceeding on a value it had to assume. The `privhelper` goes further and aborts rather than unwind, because unwinding through cleanup it cannot trust is a worse failure than stopping.

quarantine-the-unsafe

`unsafe` and `C` are quarantined into a few small, single-purpose crates, each sized to be read in a single sitting, and every other crate forbids `unsafe` outright (§3, §4). The syscall and BPF glue that no safe abstraction covers is concentrated where two maintainers must read every block, not spread thin across the tree where each instance escapes notice. This is the design register's `rule-of-1` realised in the workspace: the hazard that cannot be removed is gathered into the few crates that carry it and nothing else, each holding one narrow purpose, so a bug in the `unsafe` code is bounded by the small crate that holds it. It composes with `dont-roll-your-own`: lean on a vetted crate to avoid the `unsafe` you do not have to write, and quarantine the irreducible remainder no crate covers. Avoid what you can, gather what you cannot.

PART I

The machinery

1. Into the machine

The design volume set out what a kernel is: a reference monitor at the user level, a mediator every access passes through that the workload cannot bypass or tamper with, built from two limbs. One limb is construction, where a resource is simply absent from the kernel's world. The other is interposition, where a resource that cannot be removed is reachable only through a transaction the monitor authorises. None of that named a kernel.

A modern Linux kernel offers a set of facilities that, taken one at a time, each do a small part of this: hide a class of resource, mediate access to another, hold a privilege for a moment. No single one of them is a sandbox. The confinement is the weave: the facilities combined, resource class by resource class, so that each class is either constructed away or placed behind a mediated transaction. What follows names the threads and shows how they cross; the later chapters pick up each in turn.

1.1 What counts as a control

Not every restriction the kernel can express is a control in the sense the reference monitor needs. The test is narrow and the design holds to it: a control counts only if it is enforced by the kernel and keyed to process identity, such that the confined workload cannot forge or bypass it from inside. A rule the workload sidesteps by changing its source address, rewriting its environment, or choosing a different path to open is a speed bump, not a control, and the design does not count it as one.

That test is the tamperproof property of the reference monitor (reference-monitor) restated in kernel terms, and it does the sorting. It is why the facilities below are the ones that carry the limbs and other plausible-looking mechanisms are not: a check the workload runs against itself, a convention it is asked to honour, a filter it can route around all fail the test by construction. It is also why some facilities appear only as fallbacks. Where the first-choice mechanism is unavailable on an older kernel, a coarser one may carry the class

1.2 The construction facilities

at reduced fidelity, and where even that is missing the policy does not run at all rather than pretend (`refuse-to-start`).

1.2 The construction facilities

The construction limb is built from namespaces, and its character is absence: what was never mapped or never bound into the kernel is not there to be reached, mediated or not (`construction-by-absence`).

The user namespace is the foundation the rest stands on. A kernel runs in its own, created as the operator so that the namespace is operator-owned, with precise identity maps rather than a subuid range: the host root line and the operator's own line, plus one per granted group. That gives the kernel a real uid 0 and `CAP_SYS_ADMIN` inside the namespace and nowhere else, which is what permits the `mount` and `pivot_root` that build the view. The mapping of host uid 0 is the one privileged act in the whole construction, and it is the reason a kernel is built by a privileged step rather than by the workload itself (T5.4). The mount namespace carries the constructed view: the kernel sees the filesystem its policy binds in and a root pivoted away from the host's, so a path that was not granted is not merely denied but absent. The PID namespace makes the kernel's first process PID 1 with a fresh `/proc`, so the host's processes are not visible to enumerate, and the IPC and network namespaces remove the System V and network surfaces the same way. Each namespace takes a class of resource out of the kernel's world rather than guarding it, which is the cheaper and stronger move wherever it can be made.

1.3 The interposition facilities

Some resources cannot be removed, because the kernel exists to use them: it must execute some binaries, reach some paths, open some network destinations. For these the design does not remove the class but places it behind a mediated transaction (`interpose-as-transaction`). Four kernel facilities carry that limb. Three are the standard sandbox vocabulary and are described here; the fourth, the one that does not look like it belongs, gets its own section after.

Landlock is the principal one. It applies a kernel-enforced, unforgeable, identity-keyed access policy to the filesystem (read, write, and with `FS_EXECUTE` a proper execute semantics), to network ports, and on ABI 6 to the scoping of abstract `AF_UNIX` sockets and signals. It is the mechanism that lets a kernel hold a path it may read but not write, or execute one binary and not another, with the decision made in the kernel and not in code the workload can reach. `Seccomp` carries the syscall surface that Landlock does not express: the filter that refuses `ptrace`, the `TIOCSTI` terminal-injection guard, and the `connect()` restriction that stands in for abstract-socket scoping on kernels below ABI 6. The `cgroup BPF` programs carry

network egress: the `inet4_connect` and `inet6_connect` hooks enforce the address allowlist in the kernel, on the connecting process, with no path around them, and this is the egress floor present in every network mode but the fully isolated one. Those three are the interposition vocabulary a sandbox reader already knows. The fourth is not.

1.4 Binder

The fourth interposition facility will stop a reader who knows the others cold, because it comes from Android: binder, exposed on Linux as `binderfs`. Its presence is not exotic taste, and the reason is worth setting out, because binder carries the whole local-services and inter-kernel story in the chapters ahead.

The problem it solves is one a socket cannot. Several of the things a kernel must talk to, the session bus or a sibling service kernel, speak a protocol that grants too much to forward raw while the application expects a conforming endpoint at a familiar path. Forwarding the raw socket makes the grant auditable once, at connect time, and then surrenders every call made across it. What the design needs is a chokepoint at the call rather than the connection: one point where every transaction a kernel makes with a service is mediated and audited, with kenneld the decision point for each, and where a kernel cannot reach a service it was not granted. The same primitive has to carry communication between kennels, a client kernel calling a standing service kernel, with neither holding ambient access to the other.

`binderfs` supplies all of that off the shelf. It is the kernel's per-mount-namespace Binder filesystem, stable well under the 6.10 floor the kernel already requires for Landlock. It gives synchronous call-and-return transactions with kernel-managed identity and no application framing; a service registry brokered by a well-known node that kenneld occupies as context manager; references that are opaque kernel objects a process either holds or does not, with no path to enumerate and no abstract name to probe, so a reference passes the test of §1.1 by construction; exact kernel-driven death notification; and per-instance isolation, each `binderfs` mount a fully independent instance the way `devpts` and `tmpfs` are, so two kennels' buses share no state. The property that turns it from a curiosity into a fit is that `binderfs` is `FS_USERSNS_MOUNT`: it mounts unprivileged inside the kennel's own user namespace, so the inter-namespace gateway is kernel-mediated and unforgeable and yet cost no host capability to stand up.

What the design declines is the part of Binder that is hard. Android passes node references between processes and maintains the capability graph that results, which is most of what makes that implementation subtle. The kennel uses none of it: references are issued by kenneld as context manager and do not transfer between kennels, which keeps the properties that matter, unforgeable references, synchronous

1.5 Privilege and identity

calls, death notification, per-instance isolation, and drops the bookkeeping that does not earn its place.

That leaves the real question, which is why adopt an IPC system from another world at all rather than write a small bus over socketpairs. The answer is the supply discipline (`dont-roll-your-own`). A homebrew bus would reimplement transaction routing, reference management, death notification, and instance isolation that binderfs already provides and that Android has run at a scale no in-tree bus will see, and the kernel already speaks `ioctl` to the kernel for Landlock and BPF, so binder's `ioctl` ABI is the same shape of code rather than a new dependency. The unfamiliar primitive is the conservative choice, not the adventurous one. How kernel holds the context-manager node, how the facades transact across the boundary, and the wire format itself come later; here it is enough that the strangest thread in the weave is there because it is the one that fits.

1.5 Privilege and identity

The third group of facilities is not a limb but the handling of privilege itself, and the design's posture is to hold it briefly and in one place. File capabilities let the one privileged binary carry exactly the capabilities its construction step needs and no general root, so the privilege that builds a kernel is a named, narrow set rather than ambient authority (`rule-of-1`). `PR_SET_NO_NEW_PRIVS` is set unconditionally before the workload runs: it is the precondition `seccomp` requires, and it is the kernel's bar on a process regaining privilege through a later `exec`, so a workload cannot climb back up through a `setuid` binary it finds. The uid and gid maps and the capset that drops the bounding set are what take the constructed kernel from its transient uid 0 down to the operator's identity with nothing attached to it the policy did not grant. This is `split-the-uid` in mechanism: the kernel keeps an identity and is stripped of the authority that identity carried on a traditional system. How these are sequenced, which binary holds which capability, and for how long, is the subject of the next chapter.

1.6 The weave

No single facility is sufficient, and the design says so plainly: the mechanisms are used in combination, resource class by resource class. The map is the architecture. Filesystem access is a mount namespace for the view and Landlock for the access inside it. Execution is Landlock `FS_EXECUTE` and `PR_SET_NO_NEW_PRIVS` together. A network address is the cgroup BPF connect hooks with no fallback, because nothing coarser meets the test. Abstract `AF_UNIX` is Landlock scoping on ABI 6 and a `seccomp connect()` filter below it. Process visibility is a mount namespace with `hidepid` and a PID namespace for the stronger cut. Most classes are carried by a

primary mechanism with a fallback named beside it, and a few are carried by one mechanism with no fallback at all, which means the kernel must provide it or the kernel does not start.

The weave is the reason the confinement degrades gracefully in fidelity and never silently in safety. Pull a thread on an older kernel and the class it carried often retains a coarser fallback, at a fidelity the policy can see and a residual the catalogue records. Pull a thread that has no fallback and the policy refuses to apply rather than run a kernel that looks confined and is not. Complete mediation is this map holding for every class at once; the verifiable property is that where it cannot hold, the system declines rather than degrades (`refuse-to-start`).

1.7 The kernel floor

Because the weave depends on specific kernel facilities, a kernel depends on a specific minimum kernel, and the design states the floor rather than discovering it at runtime. Landlock is the load-bearing thread, and its versions set the floor: the filesystem ACL arrives at 5.13, network ports at 6.7, `FS_EXECUTE` at 6.10, and the ABI 6 scoping that natively isolates abstract `AF_UNIX` and signals at 6.12. The project floor is 6.10, ABI 5; on 6.12 and above the scoping fallbacks fall away and the isolation is native. The cgroup v2 hierarchy, the cgroup BPF connect hooks, and the namespaces are all long-standing and effectively universal on a modern system. At policy load the kernel checks the features its policy requires against the kernel it is on, and where a required feature is missing it refuses to apply the policy and names what is absent rather than failing obscurely or proceeding without the control.

One requirement is a deployment property rather than a kernel version. The construction rests on an unprivileged user namespace, and some hardened distributions restrict that by default. Where the restriction is in force the namespace can be created but holds no capabilities inside it, so the first map write fails and no kernel can be built. The supported posture is an AppArmor profile that grants user-namespace creation to the daemon binary alone, restoring the one capability without relaxing the host-wide restriction. It is a one-time install step, the complement to the file capabilities the privileged binary carries, and where the restriction is absent no profile is needed.

1.8 The same weave elsewhere

Naming these mechanisms does not make the design Linux; it makes this volume Linux. The reference monitor and its two limbs are platform-neutral, and the porting assessments weave the same limbs from another kernel's facilities, graded against the same test from §1.1. On macOS the only viable enforcement layer is

1.8 The same weave elsewhere

Seatbelt, undocumented and version-volatile, and it grades inferior to Linux on recon-resistance and on same-uid loopback isolation, with the shortfall recorded as a residual rather than hidden. On FreeBSD, jails with VNET and Capsicum grade equivalent or superior on most controls, with the private network stack turning the egress path into a trusted plumbing problem the port must solve deliberately. The threads differ; the weave is the same shape, and a port that graded equal across the board would be hiding exactly the residuals an honest assessment exists to surface.

With the toolkit and the weave in hand, the rest of this part picks up the threads one at a time. The first is the set of processes that do the weaving and the single one among them that holds privilege.

2. Process and privilege model

Chapter 1 set out the kernel facilities and how the confinement is woven from them. A kernel does not assemble itself: a small set of processes builds each one and supervises it while it runs, and the reference monitor's tamperproof property turns, in mechanism, on which of those processes holds privilege and for how long. The shape of that set is where the detail begins, which processes exist while a kernel runs, what privilege each carries, and where the single privileged step sits. The design register's `split-the-uid` reduces, here, to one question asked of every process on the host: which file capability does it hold, and for how long.

The answer almost everywhere is none. Of the processes that exist while a workload runs, exactly one holds host-elevated privilege, it holds it for the length of one construction, and then it is gone.

2.1 The processes

A running kernel is a small set of processes, and naming them is most of the privilege story.

`kennel` is the operator's entry point, a 13-line shim (`kennel-shim`) that holds no authority of its own: it detects which environment it is in and execs the right `libexec`, the host-side CLI (`kennel-cli`) on the host or the in-cage spawn path inside a kernel. `kenneld` is the per-user supervisor, socket-activated by `systemd --user` on the first `kennel` run and resident for the session, one per logged-in user. It runs as the user with no capabilities. It validates and executes the settled policy for each kernel, drives construction, takes binder node 0, drains the audit ringbuf, and reaps the workload, and it does all of it unprivileged. That a daemon this central holds no capability is the first half of `split-the-uid` in mechanism: `kenneld` has the operator's identity and none of root's authority, and the authority it needs for the one privileged step it does not hold but borrows, briefly, from a separate binary.

`kennel-privhelper` is that binary, and the only one on the host with elevated privilege. Beneath it sit the processes that hold no host privilege. `kennel-bin-init` is

2.2 One privileged component

the `kennel`'s PID 1: a trusted root-owned binary, trapped inside the new namespaces with no host capability, that supervises the facades and the workload, reaps the `kennel`'s processes, and restarts a facade if one dies. It holds PID 1 so that no untrusted workload does, keeping that position, with its reaping duties and its host-facing reparent, on a trusted process rather than on the code the `kennel` exists to confine. The rest hold no host privilege: the per-`kennel` `host-netproxy` and the SSH bastion are plain user processes, the in-`kennel` facades are confined workload-side code, and the workload itself is dropped to the operator with its bounding set cleared. `host-netproxy` is a separate process for an architectural reason rather than a privilege one: it is the post-policy dialer, doing the blocking, adversarial network I/O once `kenneld` has made the decision, so that I/O never sits inside the supervisor (`control-not-data-plane`). The same eviction is made on the other adversarial input a session produces: the workload's terminal output, with its attacker-shaped ANSI escapes, is parsed by the unprivileged `kennel` CLI and never at the daemon, which sees the PTY as opaque bytes to route. Between them, network I/O and terminal parsing are the two data-plane jobs kept out of the supervisor, which is what leaves it a pure control plane. The privilege-bearing surface of the whole system is one small binary and three smaller sub-helpers, described next; everything else runs as the user.

2.2 One privileged component

`kennel-privhelper` is installed with the file capabilities `cap_setuid`, `cap_setgid`, `cap_setfcap`, `cap_sys_admin=ep`, `setuid` root at mode 4755 only as a fallback for filesystems that cannot carry capability xattrs. Each of the four earns its place and no more is taken. `cap_setuid`, `cap_setgid`, and `cap_setfcap` cover writing the identity map and dropping to the operator; `cap_setfcap` is needed because the map's 0 0 1 line and the operator line are written in a single `write(2)`. `cap_sys_admin` is the kernel's gate on writing a `uid_map` that maps host uid 0, and it covers the mount and `pivot_root` construction of the view. The factory holds nothing else: no `cap_net_admin`, no BPF capability.

The rarer privileged acts are not folded into the factory's capability set; they are delegated to three single-purpose sub-helpers the factory execs only when a policy needs them, each carrying one capability for one job. `kennel-privhelper-net` holds `cap_net_admin` and mirrors the `kennel`'s loopback prefixes onto the host's `lo` over `netlink`, bringing the same /28 and /64 up on both sides so the host can present a `kennel`'s inbound listeners at the `kennel`'s own address; the two are identical by construction, not connected. `kennel-privhelper-bpf` holds `cap_bpf`, `cap_net_admin`, `cap_perfmon` and, in host network mode only, attaches the egress BPF to the `kennel`'s cgroup. `kennel-privhelper-mounts` holds `cap_sys_admin` and over-mounts the exclusive-bind sentinel. This is rule-of-1 and quarantine-the-unsafe carried into privilege itself: the host-elevated surface

is partitioned so that the capability for a narrow job lives in a binary that can do only that job, and holding `cap_net_admin` there grants the one scoped act, not arbitrary network administration (T2.4).

2.3 The factory: privilege bounded to one construction

The `privhelper` is a kernel factory. Its one provisioning op, `construct`, does all of the privileged work for a kernel in a single short-lived invocation; the only standalone op beside it is `del-addr`, the address removal at teardown.

`kenneld` invokes `kennel-privhelper construct` over a `SOCK_SEQPACKET` socketpair and sends the construction half of the Plan: the uid and gid maps, the loopback configuration and the per-kernel addresses to add, the binderfs parameters, the view bind list, the pivot target, the egress BPF payload as a framed tail, and any `stdio` or `pty` descriptors over `SCM_RIGHTS`. The `privhelper` parses this host-side before it holds any namespace, and parses it as hostile: the operator line of the map is the caller's real uid taken from `/proc` ownership and never read from the wire, and the request is refused if it names an address outside the kernel's allocated IPv4 /28 or IPv6 /64, a cgroup the caller does not own, a gid the caller is not in, or any uid but the caller's own. The decoder for that half is bounded and fuzzed, because a bug in it is the one bug at this layer that matters (T5.4).

Only after validation does privilege touch the kernel. The factory holds neither `cap_net_admin` nor the BPF caps, so it drives the host-lo loopback mirror and the host-mode egress BPF attach by exec'ing the narrow sub-helpers; it provenance-checks and opens the root-owned `kennel-bin-init` by descriptor, and then `clone3s` a child with `CLONE_NEWUSER | NEWNS | NEWPID | NEWIPC | INTO_CGROUP`, adding `NEWNET` in every mode but host, so the child is PID 1 of a fresh namespace set, born directly inside the kernel's cgroup rather than migrated into it afterward. The migration was not free: profiling added to the spawn path in 0.3.0 traced 10 to 13 ms of otherwise-idle stall to the post-hoc move into the cgroup, and removing it by cloning into the cgroup is most of what brings a kernel up in under 10 ms from the command line. The dynamic spawn an agent triggers directly is faster again, an observed 3.5 to 6.5 ms depending on the hardware, because it skips the CLI and the process startup that comes with it, and that margin is what makes an agent's spawn-use-reap loop viable. It also closes the window in which a freshly-migrated process runs briefly unaccounted, so the cgroup's limits hold from the first instruction. The user namespace is operator-owned: the child clones as the operator and self-escalates to the kernel's uid 0 to do construction, which is precisely what lets the unprivileged `kenneld` reach the new binderfs instance later through `/proc/<init>/root`. In that child the factory writes the maps, brings up the in-namespace loopback, builds the view, mounts binderfs and chowns the device to the operator, `pivot_roots` and detaches the host root, and `fexecves`

2.4 The trusted init and the drop

`kennel-bin-init` by the descriptor it opened before the pivot, with empty `argv` and `envp`, because the host path to that binary no longer exists inside the view.

Then the factory exits. It is not a supervisor and not a reaper proxy; it reports the `init`'s host pid and is gone. `kennel-bin-init`, now PID 1 of its own namespace, outlives it, reparents to `kenneld` (which marked itself a child subreaper at startup), and `kenneld` waits on it directly for the workload's exit status. No privileged process is resident for the life of a kennel, and none is shared across kennels. The privilege is bounded to the single map-writing construction and then withdrawn, which is the conservative half of a deliberate trade: a long-running privileged daemon owning the same capabilities would cost fewer execs and buy continuous privileged exposure, and the current design refuses that exposure. The factory child is the only transient uid-0 actor in the system, and it never runs with host root visible to it, so the worst a construction-half decoder bug can be is a host-root bug, which is why that decoder is the most heavily checked code at this layer (T5.4).

2.4 The trusted init and the drop

`kennel-bin-init` is the root-owned binary the factory execs as PID 1, and it is deliberately small (338 SLOC) and `#![forbid(unsafe_code)]`. It makes no policy decision and runs no mount, netlink, device, or filesystem-lookup code; that work was done by the factory before the pivot. Its path comes from the root-owned deployment configuration; the `privhelper` verifies the binary is root-owned and not group- or other-writable, opens it before the `clone3`, and `fexecves` it by that descriptor. Exec by descriptor is the only way in, not an optimisation: after the `pivot_root` the host path the binary came from is gone from the view, so there is nothing left to exec by name, and the descriptor pins the exact file vetted before the pivot, with no window in which a symlink or a swapped path could redirect it. The root ownership is tamperproofing rather than power: root confers no host capability inside the kennel, but a root-owned binary is one the operator-uid workload cannot rewrite, and PID 1 running as the kennel's uid 0 is one the operator cannot signal or `ptrace`. It is launched into a cage where its root means only that the workload cannot touch it. Once running, it pulls the supervision half of the Plan from `kenneld` over the binder bus, then forks and supervises the facades and the workload.

The workload is dropped. It runs as the operator uid, never as the kennel's uid 0; only `kennel-bin-init` holds that, trapped post-pivot with no ambient host capability (T3.1). `PR_SET_NO_NEW_PRIVS` is set unconditionally before the workload runs, the bounding set is cleared per policy, Landlock is sealed, the cgroup BPF is attached, and `setrlimit` caps are applied. Against the classic escalation, a `setuid` binary executed inside the kennel, the framework's exec invariants `deny_setuid`, `deny_setgid`, and `deny_setcap` hold categorically, so a workload cannot climb back

up through a binary it runs (T3.1). There is identity here, the operator's uid, and there is no authority attached to it that the policy did not grant: the second half of `split-the-uid`, now at the bottom of the process tree.

2.5 Two identities, not one

That arrangement is where Kennel parts company with the common rootless container. A rootless container maps the invoking user to uid 0 inside its user namespace and runs the workload as root there; the confinement is the namespace boundary, and inside it the workload is root, with nothing between it and whatever the namespace still reaches. Kennel's user namespace carries two identities instead. The `0 0 1` line maps host root to the kennel's uid 0, and that root is the trusted side: the construction child that self-escalates to it to mount and pivot, and `kennel-bin-init` that holds it as PID 1. The operator line maps the operator to itself, and that is the workload, unprivileged, never uid 0.

Two identities in one namespace means the ordinary Unix DAC line between root and a user is kept inside the kennel rather than collapsed. The trusted components are root and the workload is not, so the process the kennel exists to confine cannot write a root-owned path, cannot chown a file it does not own, cannot do any of what the discretionary model reserves to uid 0, and it is stopped by ownership alone, before Landlock, seccomp, or the namespaces are consulted. The maps are precise identity lines, not a subuid range, and the granted gids are mapped the same way; but in a namespace whose only identities are root and a non-root user, gids are ornamental, since the privilege boundary is the uid line and nothing hangs off group membership. What stands inside the kennel is the separation Unix has drawn between a privileged owner and an unprivileged user for as long as it has existed, with the framework on the privileged side and the workload on the other.

The inversion is deliberate. Rootless buys the workload root inside a box and trusts the box; Kennel keeps a real root and a real non-root user in the same namespace and puts the workload on the unprivileged side of a line the kernel has enforced for decades. The namespace, Landlock, and seccomp are the confinement above; this is the floor beneath them, and it is the one layer that asks for no new mechanism, because it is only uid 0 declining to be the workload's.

2.6 The unsafe at this layer

The privilege model is also where the volume's quarantined `unsafe` mostly lives. Two of the five unsafe-bearing crates are the kernel-interface surface of the privilege model: `kennel-lib-syscall` (946 SLOC), which holds the raw syscalls, the seccomp filter, and the namespace primitives, and `kennel-lib-landlock` (249 SLOC), which holds the Landlock ruleset construction; `kennel-lib-bpf` (811 SLOC), the

2.6 The unsafe at this layer

third, is the loader the factory and `kenneld` use to attach the egress program. Each is small, single-purpose, in the TCB, and the only place its kind of `unsafe` is permitted to appear (`quarantine-the-unsafe`).

The point worth drawing out is that privilege and `unsafe` are quarantined separately, and neither coincides with the binaries one would expect. `kenneld` (7319 SLOC) and `kennel-privhelper` (1764) are the two privileged anchors of the system, and both carry no `unsafe` of their own: the `privhelper`'s power is file capabilities, and the few `unsafe` calls its construction needs are borrowed from `kennel-lib-syscall`. So the largest privileged binary holds no `unsafe`, and the crates that hold `unsafe` hold no privilege of their own. A reader auditing for memory hazard and a reader auditing for privilege are reading two different, small, named sets of crates (`read-by-the-hostile`), and that `kenneld`, the central daemon and the thing an attacker most wants, is both unprivileged and memory-safe-by-default is the shape the rest of the volume builds on (T5.3).

The processes and their privilege are the static picture. How they talk to each other while a `kennel` runs is the next mechanism the design left abstract, and it has a single answer on Linux: the binder.

3. Binder's Node 0

Chapter 1 placed the binder among the interposition facilities, and chapter 2 placed the processes that build a kernel and hold and drop its privilege. What joins them is the question chapter 2 left standing: how a confined workload reaches a service it is allowed to use without being handed a path to it. The answer is the binder bus, and the short form is that every kernel gets its own and `kernel` sits on node 0 of each as the broker that decides what crosses.

3.0 What binder is

A reader who has spent time in Landlock and cgroup BPF may never have touched binder, because it comes from a corner of the kernel that serves Android and little else. Binder is Android's inter-process communication mechanism, and on an Android device it is not one IPC among several but very nearly the only one: an app calling a system service, an app handing work to another app, the framework delivering an event, all ride binder transactions through a kernel driver, brokered by a userspace registry called the servicemanager.

It belongs to an inverted version of the Unix model. On an ordinary system a person owns and runs applications and the uid names that person; Android turns this around and gives each installed app its own uid, isolating apps from one another by uid the way a multi-user system isolates people. The users of an Android phone are the apps, mutually distrusting, and binder is the IPC built for that situation: a kernel-mediated channel across uid boundaries, with a context manager that brokers which named service a caller may find and reach, so no app holds ambient access to another. That shape is the kernel's shape too, which is the deeper reason the primitive fits rather than merely being at hand: a kernel is an app-shaped unit of mutual distrust, and the IPC it needs is the IPC Android already built.

And it is, by a wide margin, the most exercised IPC mechanism in existence. Binder carries the cross-process traffic of every Android device, billions of them, and has done so under continuous attack, fuzzing, and kernel-security scrutiny for more than a decade. Whatever its quirks, no userspace bus written from scratch will

3.1 From connect to call

ever see that volume of adversarial use; taking the driver and its model inherits that hardening instead of setting out to re-earn it.

3.1 From connect to call

The move the bus makes is from the connection to the call. The AF_UNIX shim grants a socket by binding its path into the constructed view and auditing the workload's connect; that gates once, at connect time, and says nothing about the authority that rides the protocol afterward. A D-Bus call can spawn a session process or read a stored secret; a Wayland socket can read the clipboard. Binder generalises the move the SSH bastion made first, putting the decision at the operation rather than the connection: `kenneld` is the policy decision point for every protocol call, and the workload holds only unforgeable binder references, opaque kernel handles with no path to enumerate and no abstract name to probe. That is the reference-monitor restated for local services, with the bus as the mediated transaction every call passes through (`interpose-as-transaction`).

3.2 The bus each kernel gets

Each kernel gets its own `binderfs` instance, a fully independent mount that shares no nodes with any other kernel's, the way `devpts` and `tmpfs` are independent per mount namespace. `binderfs` carries `FS_USERSNS_MOUNT`, so the instance mounts inside the kernel's child user namespace, in the same full-capability context the spawn already uses to mount `tmpfs`, `devpts`, and `proc`; there is no host-side mount and no separate privileged step. The spawn mounts a fresh `binderfs` at the view's `/dev/binderfs`, with `max=256` capping binder-device allocation per kernel as a denial-of-service bound rather than a tuning knob, allocates the standard device named `binder` through `BINDER_CTL_ADD`, and creates the conventional `/dev/binder` symlink so a stock binder client finds its driver at the default path with no per-workload configuration, the same principle the socket shim rests on.

Taking node `o` is where the daemon and the instance meet, and the mechanism is more particular than it looks. `kenneld` runs in the initial user namespace and must hold the context-manager fd for every instance, since one entity owning every node `o` is the precondition for routing a call from one kernel to another. It cannot be handed that fd: a binder fd is bound to the process that opened it, so a descriptor passed over a socketpair cannot be mapped or made context manager by a different process. `kenneld` therefore opens the device itself, through `/proc/<init>/root/dev/binderfs/binder`, which it is permitted to do because the kernel's user namespace is operator-owned and the operator `kenneld` is privileged over the instance. It then calls `BINDER_SET_CONTEXT_MGR`, which is one-per-instance, taking node `o`. The workload is later granted Landlock read and write on the de-

vice, reached through the `/dev/binder` symlink, but never on `binder-control`: only the spawn setup allocates devices, and it does so before the seal. When the kernel exits, the instance disappears with the child's mount namespace, with no host-side unmount, and pending transactions receive death notifications as every node is destroyed. Construction is the privileged step here, not the bus: the 0 0 1 map the instance's uid-0-owned nodes require is the privhelper's act (T5.4), not a binder-specific privileged surface.

3.3 kenneld on node 0

Node 0 is the service registry, the analogue of Android's servicemanager: a caller reaches it through the well-known handle and never by name, and its verbs carry the same names as Android's interface even though the transaction codes are `kenneld`'s own and nothing is wire-compatible. A service process registers a name with `addService`; a workload resolves one with `getService` and receives a node reference or a failed reply; `listServices`, `isDeclared`, and `getDeclaredInstances` are the introspection a caller may run on its own grants. Every name is bounded and validated, checked against the kernel's settled policy before `kenneld` records or resolves it, and audited on every verb. The check that a service is declared before it can be registered is the manifest check Android makes against VINTF, with the settled policy standing in for the manifest: a service the policy does not declare cannot register and reports as undeclared. Binder's general capability-passing, the transfer of arbitrary node references between processes, is not used; references are issued only by `kenneld`, and they cross between kernels only as `kenneld` brokers them on the mesh bus, the subject of the inter-kernel chapter. That drops the reference-graph bookkeeping that makes Android's implementation subtle while keeping the unforgeable references, the synchronous calls, the death notifications, and the per-instance isolation.

The bus carries the kernel's own control plane as well. `kennel-bin-init` is a binder consumer on the same instance, pulling its Plan from node 0 and sending the life-cycle notifications that tell `kenneld` a facade crashed or the workload exec'd. Those verbs are gated by the unforgeable binder caller identity, the `init`'s host pid at euid 0, a value the privhelper hands `kenneld` at construction rather than anything the caller puts on the wire, so a workload can address node 0 but cannot exercise them. The identity binder stamps on every transaction is the gate, not a token the payload carries, which is why the gate holds against a workload that controls its own payload bytes entirely.

A reserved prefix, `org.projectkennel.*`, is `kenneld`'s own. It is both context manager and the provider of a built-in set under that prefix, and two rules hold before any policy lookup: an `addService` for a reserved name from any caller other than `kenneld` is refused, with no policy override, and a `getService` for a

3.4 A transaction across the boundary

reserved name always resolves to `kenneld`'s local node and is never routed to a peer kennel. A reserved node exists only when its policy section is non-empty, so the absence of the node is itself the proof that the capability was not granted (*construction-by-absence*). The registry verbs are cheap in-memory operations, but a facade verb performs real host I/O, so node `o` is served by a pool of looper threads per instance rather than one: each looper receives, handles, and replies to its own transactions, and the pool is sized so that one looper blocked on a host dial does not stall the others. A dial carries a deadline, so a wedged or hostile target degrades to a refusal on that one transaction rather than tying up the instance.

3.4 A transaction across the boundary

The `AF_UNIX` facade is the example whose whole shape fits here, and it shows the bus replacing a path with a brokered connection. The workload sends a `CONNECT` transaction to the `org.projectkennel.IAfUnix/default` node carrying the requested socket path as a bounded, validated string. `kenneld` checks the path against the kennel's `[[unix.allow]]` list, the same settled grant the shim consumed. On `allow` it performs the `connect()` itself, host-side, and returns the connected `fd` in the reply as a `BINDER_TYPE_FD`; on `deny` it returns a failed reply and audits it. The workload receives an already-connected socket and never holds a path into the host's `AF_UNIX` namespace; the path does not appear in the constructed view at all, which closes the residual where a workload could enumerate the socket paths it was granted, or probe for ones it was not. The grant is exercised at the call, audited at the call, and the workload uses the connection without ever reaching the name behind it (*mediate-use-not-reach*). The D-Bus facade and the network facade are larger, each parsing a foreign wire protocol, and each takes its own chapter; the `AF_UNIX` facade is the one that shows the move whole.

3.5 Control plane, not data plane

What the workload gets back is an `fd`, and that is the point at which `kenneld` leaves. The connected socket flows its bytes directly between the workload and the far end; `kenneld` brokered the connection and then stepped out of the path it brokered, parsing control-plane structures and never the payload stream (*control-not-data-plane*). The `fd` and shared-memory object types are permitted within a single instance, between two parties already inside the same trust boundary, so passing the connected socket to the workload is sound.

Reaching a service in another kennel is a separate matter, gated by bilateral policy: the edge exists only where both kennels' signed policy declares it, a `provide` on one side and a `consume` on the other (T5.2). How that reach is mediated across the instance boundary is the subject of the inter-kennel chapter, not this one.

3.6 The crate and the unsafe

The binder ioctl ABI is hand-rolled in `kennel-lib-binder`, built in the same shape as the BPF loader crate, with `unsafe` confined to a single file holding the ioctl FFI and the crate listed among the few that carry it. There is no `libbinder`: that library and its NDK form carry Android-specific dependencies, so the crate binds the kernel's stable binder UAPI directly, the way the BPF crate compiles against the kernel headers (`own-your-supply-chain`, and `dont-roll-your-own` for what the kernel already provides). The split between mechanism and policy is the BPF crate's split again: `kennel-lib-binder` owns the command loop, the encode and decode of the command stream and transaction data, the context-manager looper primitive, device allocation, the protocol-version check that requires binder version 8 at open, and death-notification plumbing, and `kenneld` owns which registrations and resolutions are permitted, the registries, the reserved services, and cross-kernel mediation, under `#![forbid(unsafe_code)]`. The consequence worth stating is where the untrusted input lands. The command-stream decoder consumes bytes the workload controls, so it is an untrusted-input parser, reachable from inside every kernel because the bus is unavoidable, and it carries a fuzz target for that reason (T5.1). The `unsafe` is a small, separately reviewed, fuzzed leaf; the policy brain that decides every crossing holds none of it (`rule-of-1`, `quarantine-the-unsafe`).

4. The trusted base

Chapter 3 ended on a single crate's division of labour: the unsafe binder ABI beneath the policy brain that holds none of it. That division scales to the whole code-base, and what it scales to is the trusted base, the set of crates whose correctness the confinement rests on. How large that base is, how it is held small, and where the honest count of it parts from the flattering one are what follow.

4.1 What counts as trusted

The trusted computing base is the code linked into the binaries that enforce: `kenneld` the daemon, `kennel-privhelper` the capability helper, and `kennel-bin-init` the root-owned PID 1. A bug in any of them can break confinement, so their first-party dependency closure is the base whose correctness is load-bearing. At the time of writing, between releases 0.4.0 and 0.5.0, the whole workspace is 27 crates and about 35,000 lines; the TCB closure is 16 of them, about 21,000. The other 11, some 13,600 lines, are outside it: the operator CLI and the crates only it pulls, the in-kernel facades the workload reaches, and the D-Bus mediation engine.

What keeps those 11 out is a crate boundary, not a habit of not calling them. The operator CLI, the CLI-to-daemon control protocol, and the policy compiler are each their own crate, so the daemon links only what it needs to verify a settled policy and act on it. The compiler is the clearest case: it is the larger half of the policy code, about 5,000 lines of source schema, template resolution, deltas, signing, lint, and risk evaluation, and the daemon links none of it. Authoring depends on the runtime half and never the reverse, and only the CLI links the compiler, so a cargo tree over the daemon shows no compiler, no argument parser, no JSON. The minimisation is structural and checkable, not a property the daemon happens to have today (`rule-of-1`).

4.2 The vendored truth

The lines counted so far are first-party, and that is the flattering count. The base a compromise runs through is dominated by the vendored crates the trusted crates pull in: at the same snapshot, roughly 200,000 lines of vendored code against the 21,000 first-party in the closure, an order of magnitude more. An inventory that stopped at first-party would understate the audit surface by that order of magnitude, so the honest one counts the vendored code too, and sorts it on two axes: whether it is logic or bindings, and whether the input it sees is adversarial or trusted (*own-your-supply-chain*, *read-by-the-hostile*).

Most of the vendored bulk is bindings, not logic. `libc` alone is over 100,000 lines, and it is almost entirely `extern` declarations and constants; `nix` is another 30-odd thousand of typed wrappers over the same surface. These are platform declarations resolving to the system's own `libc.so` and the kernel, `cfg`-gated, with close to no per-line algorithmic risk, because nothing in them runs an algorithm over an input. The logic that does run in-process is far smaller, on the order of 60,000 vendored lines, and it is a short list: an ELF reader, the serialisation of the project's own typed structures, a signature verifier, the compiler for the project's own `seccomp` filter, and the parser for the signed policy artefact.

4.3 The empty intersection

Sorting on two axes is worth the trouble because of the one cell it leaves empty. Logic against bindings says what executes in the process; adversarial against trusted says what an attacker can steer. The cell that would hurt, vendored logic running on adversarial input inside the daemon, has nothing in it. The vendored logic that runs in the daemon all reads trusted input: the project's own ELF, its own typed wire, its own filter program, the signed and verified policy artefact. The one piece that touches attacker-supplyable bytes is the signature verifier, and verifying those bytes is the whole point of it, a guard rather than a victim. The ANSI terminal parser that does read workload-controlled output, the obvious place adversarial vendored logic would creep in, runs in the operator CLI and not at the daemon's read point, where the broker is a raw-byte router that parses nothing.

The one first-party parser of adversarial bytes left in the daemon is the binder command decoder, exactly the one chapter 3 quarantined and fuzzed (T5.1). Everything else adversarial is pushed to the untrusted side by construction. The D-Bus engine makes the discipline visible: it parses the hostile D-Bus wire, some 4,000 lines of it, but it is vendored onto the in-kernel facade and the operator-context delegate, never onto the daemon, so a cargo tree over the daemon stays free of it. Mediating D-Bus grew the facade's attack surface, on the untrusted side of the boundary, and left the trusted base unchanged. The host-side dele-

4.4 The unsafe, quarantined

gates are the same move for I/O rather than parsing: the outbound dialer and the inbound acceptor are dumb processes, each close to a netcat over an owner-only socket, while the daemon holds all the policy and none of the blocking work (`control-not-data-plane`).

4.4 The unsafe, quarantined

Five crates carry `unsafe`, and they are the same five throughout the system: the `rawsyscall`, `namespace`, and `seccomp` surface; the hand-rolled Landlock ABI; the BPF loader; the binder `ioctl` ABI; and the `SCM_RIGHTS` descriptor adoption. Together they are under 4,000 lines, and all of them are leaves or near-leaves in the dependency graph, depended on rather than depending. The safe operating-system primitives, path canonicalisation, uid and gid handling, the netlink and `usersns-map` handshakes, were deliberately split out into their own safe crate so the unsafe crates stay small enough to read in one sitting, which is the size a reviewer can hold in their head and a fuzzer can cover (`quarantine-the-unsafe`, `rule-of-1`). The trusted base is large in total, most of it vendored bindings that run no algorithm, and the part of it that is both first-party and unsafe is under 4,000 lines a reviewer can read end to end.

4.5 The shape of the workspace

The naming makes the inventory legible. A `kennel-lib-` crate is a library; a `host-binary` is a host-side delegate running in the operator's context, outside every `kennel`; a `facade-` binary is the in-kennel end of a connector, confined workload-side code; the rest are the daemon, the helper, and the inits. The dependency graph has a direction: `kenneld` sits at the top, pulling nearly every library to do its work, the unsafe mechanism crates sit near the bottom as leaves providing one primitive each, the libraries depend downward, and the authoring half depends on the runtime half and never the reverse, so there are no cycles and the trusted closure is exactly the downward cone under the three enforcing binaries. Of those three only one holds host privilege, `kennel-privhelper`; `kenneld` runs unprivileged as the operator and `kennel-bin-init` holds no host capability, as chapter 2 set out. The trusted base is defined by what would break confinement, not by what holds privilege, and most of it holds none. The whole workspace is blocking and thread-per-connection with no async runtime, which keeps the call graph something a reader can follow by hand.

5. The command surface

Chapter 4 drew the line around the trusted base and noted how much sits outside it: the operator CLI, the compiler, the argument parser, the terminal filter. The command surface is that outside, the operator's whole interface to the system, and the fact that nearly all of it falls beyond the trusted line is the shape worth seeing. The entry point holds no authority, the wire to the daemon is small enough to read in a sitting, and the work that looks heavy, compiling and signing a policy, happens here, on the untrusted side, where a bug is the operator's problem and not the confinement's.

5.1 The shim

The command a user or an agent types is `kennel`, and it is a 13-line static binary that holds no authority and does no work of its own. It detects its context and execs the right execution unit, replacing itself: the host execution unit on the host, the in-cage spawn-requester inside a kennel. What makes it small is that the detection is not a check. The shim execs the host unit first, unconditionally; on the host that succeeds and the host unit takes over, and inside a kennel it fails, because the host unit is absent from the kennel's view and not in its `exec.allow`, so the shim falls through to the spawn unit. The failure to reach the host unit is the context signal (`construction-by-absence`).

That the signal is a failure rather than a probe is what keeps the shim out of the security argument entirely. A wrong guess cannot reach host authority, because the thing a wrong guess would reach, the host unit, is unreachable from inside a kennel by construction, allowed nowhere and soon absent from every cage's view. The dispatch is therefore ergonomics and not a boundary: it decides which of two units runs, and the one that must not run from inside cannot be run from inside no matter what the shim does (`do-less`). One static artifact serves both sides, so there is nothing to install differently in a kennel than on the host.

5.2 Outside the trusted base

The host execution unit is the operator CLI, its own crate, outside the daemon's trusted base. It links the control-wire types and nothing of the daemon's enforcement code, so the dependencies that come with an operator tool, an argument parser, a JSON reader for the trust manifest, the terminal-escape filter, stay out of the daemon's closure entirely. The protocol cannot drift between the two, because both the CLI and the daemon depend on the one control crate for the wire types: one definition, linked twice (`rule-of-1`).

The CLI is stateless and splits cleanly in two. Some verbs contact the daemon over its control socket, the ones that start, attach to, stop, or list a running kennel. The rest operate only on local files and never open the socket at all: the whole policy group, the audit query, key generation, the trust-manifest review. This is the runtime-and-compiler split of chapter 4 seen from the operator's side. The compiler is the largest single thing the CLI carries, and it runs here, locally, turning a source policy into the signed settled artefact, while the daemon links none of it and only ever verifies the artefact's signature and acts on it. The heavy authoring work and the light enforcement work are on opposite sides of the trusted line, and the command surface is where the heavy side lives.

5.3 The control protocol

The wire to the daemon is deliberately small. The socket is the user's own, mode 0600 under the runtime directory, and the first thing across it is a version handshake: the client sends its schema version and build, and the daemon answers compatible or, on a mismatch, drops the connection without reading a request. A schema skew is refused before any request body or policy is parsed, so a client and a daemon that disagree about the protocol never get far enough to misread each other (`refuse-to-start`).

The request and reply bodies are fixed-layout struct messages, each a leading op or tag byte followed by primitively encoded fields. There is no JSON, no request identifier, and no method-name string: the op byte is the method selector. Integers are native-endian, strings and lists are length-prefixed and bounded, a string at 64 kilobytes and a list at 4,096 elements, and UTF-8 is validated on decode. The verbs are few, start a kennel, stop one, list them, and answer the SSH bastion's authorized-keys query, and their replies as few. An error is a first-class reply carrying a message, not a coded catalogue to keep in sync; a malformed frame, an unknown op or a truncated, oversized, or non-UTF-8 field, is refused at decode and closes the connection. A wire this narrow, with every field bounded and the selector a single byte, is one whose decoder makes most malformed input unrepresentable rather than handled (`make-invalid-unrepresentable`).

What the protocol does not carry is the workload's bytes. The standard streams travel as descriptors, passed alongside the start frame as ancillary data, not copied through the daemon: a non-interactive run hands over its three stdio descriptors and the workload is attached to them directly. An interactive run hands over a single socket, and the spawn seal allocates a controlling pty inside the kennel's own devpts, so the workload has real job control and the operator's own terminal is never exposed to it. The daemon's broker holds the pty master and proxies it, the CLI raw-modes the terminal and proxies bytes both ways, and the filtering of dangerous terminal escapes in the workload's output runs in the CLI, on the bytes' way to the real terminal, so the parser of workload-controlled output never executes in the daemon (`control-not-data-plane`). The daemon decides and routes; the bytes flow past it.

5.4 The verbs

The surface above the wire is a small set of verbs dispatched on the first argument, each parsing its own flags, with the command list rendered from an in-binary table so the help cannot drift from what exists. The lifecycle verbs are the ones that reach the daemon: `run` starts a kennel from a policy and runs a workload in it, foreground and, when interactive, detachable; `attach` reconnects a terminal to a running one; `stop` and `list` do as they say. A run from a pre-compiled settled policy needs no key, because the daemon verifies the artefact's signature against its trust store; only a run straight from source compiles and signs in memory and so needs a signing key.

Everything else is local. The policy group is the authoring surface, compiling, validating, signing, linting, diffing, evaluating a policy against the threat catalogue, and re-pinning a template to a newer version, all on files, none touching the daemon. `keygen` makes a signing key, `audit` queries a kennel's recorded account, and `review` re-pins a workspace's trust manifest, the deliberate operator act the run path will not perform as a side effect (T2.8). The weight of the command surface is here, in authoring and inspection, on the untrusted side of the line, which is the point: the operator's tool is rich and the trusted base it drives is not.

PART II

The resource classes

6. The filesystem

Part I built the machinery: the kernel facilities woven into a confinement, the processes that hold it, the bus, the trusted base, and the surface that drives it. Part II turns to the resource classes that machinery confines, a mechanism at a time, and it starts with the filesystem, because the filesystem is where the project's motivation lives. The thing a kernel must not see is the user's real home directory, with its keys, its tokens, and its shell history, and the account of how a kernel does not see it is the account of the filesystem.

6.1 The view and the access

Filesystem confinement is built from both limbs at once, and the design is explicit that neither alone is enough. The construction limb is the view: a mount namespace and bind mounts assemble what the kernel sees in its home and runtime directories out of only the granted paths, so the real directory contents are not there to be reached (*construction-by-absence*). The interposition limb is Landlock, a kernel-enforced ACL keyed to the process that grants read, write, and execute on the bound paths and denies the rest (*interpose-as-transaction*).

Each closes a gap the other leaves open. A view without an ACL can be read around: a kernel could follow a symlink out of a bound directory into the real path behind it, and with no ACL in the kernel nothing would stop the dereference. An ACL without a view leaks by enumeration: a `readdir` on the home directory would return every entry, including the ones the kernel cannot open, which both tells the workload what exists and confuses the user about what was granted. With both, a `readdir` returns only the granted entries because the others are absent from the view, and a symlink that reaches for a real path is resolved by Landlock to that path and denied there. The symlink class is the kernel's to handle: Landlock resolves a link to its target and applies the ruleset to the resolved path, so a link out of a granted directory gains nothing. The symmetric bind-mount escape, remounting a forbidden path into an allowed one, is closed by the kernel holding no `CAP_SYS_ADMIN` to mount with, by `MS_NODEV`, `MS_NOSUID`, and `MS_NOEXEC` on the

6.2 The constructed home

view's mount points, and by the namespace being `MS_SLAVE` so a mount inside it cannot propagate back to the host.

6.2 The constructed home

The home directory is the heart of the construction, and the move is to build a new one. The view begins as a fresh tmpfs that becomes the kennel's root, and into it the spawn binds the policy's granted paths from the real home, read-only unless the grant is writable. The grant names the paths and which of them may be written:

```
[fs]
read  = ["~/projects/acme/**", "~/.config/acme/config.toml"]
write = ["~/projects/acme/build/**"]
```

Each read path binds into the fresh root read-only and each write path binds writable, and the home those binds land in otherwise holds nothing:

~/	fresh tmpfs, writable
projects/acme/	bound read-only
projects/acme/build/	bound writable, persists to the host
.config/acme/config.toml	bound read-only

The system surfaces around them are built rather than shared: `/etc`'s identity files are written scrubbed of host specifics, naming an in-kennel user, so a lookup of the running identity returns the masked name and the constructed home rather than the operator's real login.

A policy names home as `~`, and only as `~`. The same `~/projects/acme` is the operator's real `/home/johndoe/projects/acme` on the host, where the bind source lives, and the kennel's own `/home/kennel/projects/acme` inside the view, where it lands, under the home of the masked kennel user. The two absolute paths differ, and `~` is the one token that resolves to the right home on each side of the bind. A literal host path in a grant would pin one operator's login into a document meant to be portable, and would name the host's home where the view's was meant, so a grant never writes one. Home is `~`, on both sides of the boundary, always.

The spawn then `pivot_roots` into the new root, and the Landlock ruleset is applied afterward so its rules key on the view's own inodes.

The result is the protection the project was built for. Inside the kennel, a listing of the home shows only the granted entries; a path to the user's SSH directory returns no such file, because the directory was never built into the view; and a

constructed path that tries to reach the real one is denied by Landlock as well, so the absence is backed by a denial. The kernel cannot enumerate, cannot discover, and cannot stumble into the credentials and state in the user's real home, because they are not in the world it was given (*construction-by-absence*). The protection is absence rather than redaction: the credential paths are not scrubbed in place, they are simply not bound, which is what closes the casual-reconnaissance case (T1.1) without any in-view filtering at all.

6.3 Writable, but ephemeral

The constructed home carries a Landlock write grant, because an application that cannot write to its own home breaks in surprising ways, and least astonishment is worth a grant here. What makes that safe is which surface is writable: it is the fresh tmpfs, not the host. Anything the workload creates directly in its home lands on that ephemeral tmpfs and is gone at teardown; only the paths bound writable from the real home survive, and a path bound read-only stays read-only at the VFS layer regardless of the home's own write grant. So the writable home is real enough for tools that expect it and empty of consequence by default: a workload that scatters files across its home persists nothing it was not explicitly granted a writable bind for, and the persistence it does get is exactly the set of paths the policy named.

6.4 Denials and the pseudo-filesystems

Beneath the grants sits a floor of categorical denials, evaluated before any allow, that templates carry by default and operators rarely touch. The pseudo-filesystems are constructed rather than inherited. The kernel gets a private tmpfs at /tmp, so it neither sees nor litters the host's; its /proc is the fresh one of its own PID namespace, showing only its own processes, with the host's hidden rather than filtered; and /dev is assembled from an allowlist, with the dangerous nodes denied by default, the memory devices, the GPU, audio, video, and input devices among them, present only where a policy passes one through deliberately. The kernel is given the device and pseudo-filesystem surface it needs to run and nothing of the host's that it does not, which is the same absence the home is built on, applied to the rest of the tree.

6.5 The trust manifest

One filesystem threat is not about what the kernel reads but about what it writes, and it reaches past the kernel's own life. A workload with a writable bind into a project can plant a deferred trigger, a build recipe, a repository hook, an editor task, that does nothing inside the kernel and fires later in the operator's own unconfined shell (T2.8). The defence is a host-readable marker the confined workload cannot

6.5 The trust manifest

forge. A `.trust-manifest.json` at the root of each writable workspace pins the hash of every known trigger:

```
{
  "version": "2.0",
  "generator": "kennel compile",
  "execution": {
    "triggers": {
      "Makefile": { "kind": "content", "sha256":
        ↪ "sha256:9f2b1c..." },
      ".git/hooks/pre-commit": { "kind": "content", "sha256":
        ↪ "sha256:4c81a0..." }
    }
  }
}
```

Each trigger path maps to the pinned hash of its bytes, which addresses a copy held in a store beside the manifest; the record carries more than the hash, the file mode so a revert cannot silently drop a setuid bit and the provenance of the pin, but the hash is what a divergence is caught against. The manifest is written by the CLI before the daemon is contacted, and refreshed only by the deliberate `kennel review`, never as a side effect of a run that would otherwise launder the workload's own edits. The parser for it runs in the CLI; the daemon never links it.

Keeping the workload from forging the pins is a construction problem, and it is solved with the same over-mount the view is built from. Inside every writable bind, after the binds and before the pivot, the spawn over-mounts an empty read-only file at the manifest's path. Inside the kennel the manifest is present but empty: the workload can neither read the pins to learn what is watched nor write them to cover a change. The host inode underneath is untouched, so the operator's tools read the real manifest. This is deliberately not a Landlock denial, because Landlock cannot make a file in a granted directory read as empty while its parent stays readable; the view over-mount is the only mechanism that delivers the no-read property, and it is applied here to one well-known path. The pinned bytes themselves are kept beside the manifest in a content-addressed, operator-owned store, masked the same way, which is what lets review show a diff and restore a baseline rather than only detect a change.

The manifest is enforced from two sides. While the kennel runs, the daemon watches the pinned triggers with an unprivileged `inotify` in the operator's context and acts on a mutation through the cgroup it already holds, warning, freezing the workload, or killing it, the one place the filesystem defence reacts on the workload live rather than only informing the host. After the run, cooperating host tooling refuses to execute a trigger whose hash has diverged from its pin, and `kennel review`

shows the operator a unified diff and re-pins on approval, the human sign-off the masked workload cannot perform; a revert restores each changed trigger from its stored bytes and removes a planted one. The masking and the pins are structurally complete, the workload provably cannot read or forge them, and the honest residual is the catalogue: the defence is only as wide as the set of trigger kinds it knows to watch. For the writable bind that needs no concurrent host access, marking it exclusive shadows the source host-side for the run, so the operator and the workload cannot share the path while the kernel lives, closing the live channel the enumerated catalogue cannot.

7. Execution

Chapter 6 settled what a kernel may read and write. Running a program is a separate grant. A binary the kernel can read is a file like any other until it is launched, and whether it may be launched is gated by a second Landlock access laid over the first, on an allowlist of its own, and what follows is that allowlist and the honest account of what it controls and what it does not.

7.1 What it gates

Execution control governs which programs a kernel may launch, by gating `execve` and its variants on a deny-by-default path allowlist. An empty allowlist runs nothing, so a bare base template can start no program at all and a derived template adds exactly the binaries it needs. The grant is on the act of launching a new program, not on the file's existence: the binary is present and readable through its filesystem grant, and the execution layer decides only whether the kernel will turn it into a running process.

The boundary is narrower than it first sounds, and the design states the narrow version rather than the flattering one. What is controlled is `execve`, the spawning of a new program. It is not a control over the code a running program then loads. A program the kernel is allowed to run can map and execute any file it can read, because the kernel checks the execute access at `execve` and not at the `mmap` a dynamic loader uses to pull in code. The honest claim is that a kernel cannot spawn a program that is not on its allowlist, not that it cannot run code that is not on its allowlist. For most workloads the two are close; for an interpreter they are the same statement read twice, since an interpreter evaluates code by design. The execution layer answers which binaries may launch, and the filesystem, network, and local-service layers answer what a launched binary may then do.

7.2 The allowlist and `no_new_privs`

An execution policy is a shell and an allowlist of paths:

```
[exec]
shell = "/bin/sh"
allow = ["/bin/sh", "/bin/dash", "/usr/bin/dbus-send"]
```

The allow list is the execute allowlist, and `shell` names the login shell the synthetic `passwd` gives the masked user. The allowlist is enforced by Landlock, with `LANDLOCK_ACCESS_FS_EXECUTE` on the granted paths and a denial on everything else, which is why the kernel floor for an execution policy is 6.10, where that access has its full semantics; on an older kernel the policy is refused rather than applied at reduced fidelity (`refuse-to-start`). The enforcement is independent of `$PATH`: a kernel that invokes a tool by bare name fails the `execve` if the resolved binary is not granted, no matter where it sits on the search path, and the kernel's `$PATH` is set only so the failure reads as a missing tool rather than a mysterious lookup. The binaries a confinement most wants to keep out, the privilege-changing ones, are kept out by simply never appearing on any allowlist: the escalation tools are not denied by name, they are absent from the grant, so they do not run (`mediate-use-not-reach`).

Beneath the allowlist sits a second mechanism that does not depend on it. `PR_SET_NO_NEW_PRIVS` is set unconditionally in every kernel, before the workload runs, and no policy can turn it off. With it set, a `setuid` or `setgid` binary does not gain its owner's identity on `execve`, and file capabilities do not apply, so even an escalation binary that somehow were on the allowlist could not lift the workload above the operator it runs as (T3.1). This is the execution-time face of keeping identity without authority: the workload cannot climb back to privilege through a binary it finds, because the kernel refuses the climb (`split-the-uid`). The flag is cheap, it is non-negotiable, and a framework that let a policy clear it would be misnamed.

7.3 What `FS_EXECUTE` checks

The execute access is checked at `execve`, on each file the kernel opens for execution, and for a dynamically-linked program that is two files, not one: the binary itself, and the ELF interpreter named in its `PT_INTERP`, the dynamic loader the kernel opens to run it. An allowlisted dynamic binary therefore needs the execute grant on its loader as well as on itself, or it does not start. It does not need, and does not get, an execute grant on the shared libraries the loader then pulls in. Those arrive through a memory mapping, and Landlock has no hook on `mmap` or `mprotect`, so a file mapped executable is never checked against the execute access at all. A library loads on read access alone, and no execute grant on it would change that.

The design follows that fact rather than fighting it. At compile time the toolchain reads each allowlisted binary's ELF with a parser rather than by running it, takes

7.4 Writable, and interpreted

the loader path from `PT_INTERP`, and settles the distinct loaders into the signed policy as their own fixed, auditable set:

```
[exec]
allow   = ["/usr/bin/python3"]
loaders = ["/lib64/ld-linux-x86-64.so.2"]
```

The operator wrote the `allow` line; the `loaders` set is the compiler's, `python3`'s `PT_INTERP` resolved and recorded so the `execute` check at `execve` finds the loader granted as well. A statically-linked binary names no interpreter and contributes none. The libraries are simply readable, covered by the ordinary filesystem grants that already reach the system library directories, and there is deliberately no `execute-allowlist` over them. A list of permitted libraries would be a policy the kernel cannot enforce, since the access it would name is never checked, and a control that cannot be enforced is worse than no control, because it reads as a promise the system does not keep.

7.4 Writable, and interpreted

Two limits on the allowlist are worth stating because a reader will otherwise assume them away. The first is that a writable path must not also be executable. Without that rule, a kernel granted write into a project and the right to run an interpreter could write a fresh binary and run it, or write a script and feed it to the interpreter, and the allowlist would have bought nothing. The `deny_writable` invariant closes it the way a `noexec` mount would, making the intersection of writable and executable paths empty by enforcement, so a binary placed in a writable directory cannot be launched from it.

The second is the interpreter caveat, and it is the honest boundary again at one remove. Granting an interpreter the right to run grants it the right to run anything it can read: a kernel whose only executable is a scripting language can execute any program written in that language, from a file, from standard input, from the network. The execution grant gates which interpreters start, not what they do once started, and a reader who expects an allowlisted interpreter to mean a single permitted script has misread the control. The containment for what an interpreter does lives in the other resource classes, not here. Where the workload's own first binary must be exactly the expected one, a content pin closes the remaining gap: the policy may carry a set of accepted digests for the initial binary, and the daemon verifies the resolved binary's hash against them before the handoff into the kernel, refusing the run on a mismatch. That check is on the bytes, made by the daemon and never by the init inside the kernel, and it is the one place the execution control reaches past the path to the content behind it.

7.5 Composing the allowlist

The allowlist in §7.2 was hand-listed, and for a one-off policy that is the honest form. In practice the same few binaries recur, an interpreter and its package tools, a version-control client, a build toolchain, and hand-listing them in every policy invites the drift a confinement exists to prevent. The recurring grants live in a fragment instead, a signed, version-pinned bundle a policy includes rather than copies:

```
# fragments/lang-python/policy.toml
name = "lang-python"

[[exec.allow.add]]
path = "/usr/bin/python3"
reason = "Python interpreter"

[[exec.allow.add]]
path = "/usr/bin/pip3"
reason = "Python package installer"
```

A fragment is additive: every entry is a `[[*.add]]` delta appended to the effective policy, so a fragment can only widen a grant and never remove or override one, which keeps composition order-independent. The `lang-python` fragment carries more than its `exec` grants, the writable `~/.cache/pip` the installer needs and the `pypi.org` proxy entry the index needs, so one include grants a Python workflow everything it touches across `exec`, `filesystem`, and `network` at once. A leaf pulls it in by versioned reference:

```
name = "my-agent"
template_base = "ai-coding-strict@v1"
include = ["lang-python@v1", "vcs-git@v1"]
```

Each include is resolved and signature-verified the way the parent template is, and byte-pinned in the lockfile, so the resolved set reproduces or the build fails. The loaders of §7.3 do not care where a binary's grant came from: the compiler resolves `python3's PT_INTERP` into the settled policy whether the operator hand-listed it or included `lang-python`. Two includes that add conflicting rules for the same destination fail to compile with an explicit conflict rather than a silent last-wins, and the resolution machinery that enforces all of this is the compiler's.

8. The network

Chapter 7 gated which programs a kennel may launch. Where a launched program may reach is the network's question, and it is the one most directly tied to the harm a confined agent can do, since exfiltration and command-and-control both travel outbound. The design's move is to give a kennel no network path of its own and route every connection it makes through a broker that decides, resolves, and dials on its behalf. The filesystem was confined by building the world the kennel sees; the network is confined by building the path the kennel takes, in both directions: the one it reaches out along, and the one a connection reaches back in along.

8.1 The four modes

A kennel's relationship to the network is one of four. In *none*, the kennel gets an empty network namespace with no interfaces and no inet sockets at all, a zero-cost case that needs no proxy and no loopback, right for an untrusted post-install script or a repository the operator only wants to read. In *constrained*, the default for a defensible template, the kennel gets its own namespace and reaches a specific allowlist of destinations through a per-kennel proxy. In *unconstrained*, the same namespace and proxy carry open egress, keeping the namespace boundary and the audit stream but dropping the allowlist to the invariant denies, for a build that truly needs the public internet with its traffic still recorded. The three share a shape: a private network namespace, and egress only through the broker.

The fourth, *host*, is the exception that proves it. A *host-mode* kennel shares the host's network stack rather than getting its own, which is the only way to run packet capture or raw-socket tooling, and it has no proxy because there is no namespace boundary to funnel through. The egress gate in that mode is the kernel ACL directly. Sharing the host stack reinstates host-network reconnaissance in full, the workload can read the host's interfaces, routes, and listening sockets (T1.6), so the mode is gated on the operator supplying a reason, the compiler records the reinstatement, and the diff tool surfaces it (`footgun-warn-dont-forbid`).

```
[net]
mode   = "host"
reason = "tcpdump needs the host's interfaces and routes"
```

A by-name allow cannot be expressed in host mode and is refused at compile time, because there is no proxy to resolve a name.

8.2 The proxy as gateway

In the three namespace modes the kernel has no route off its own loopback, so it has no network path to the proxy and no network path to anything: a connect to any address but its own loopback listener goes nowhere, because the namespace is empty of routes. Egress does not cross a network boundary at all. It crosses the binder gateway. The workload speaks SOCKS5 to an in-kernel shim on the loopback, the shim issues a CONNECT transaction to kenneld on node 0, and kenneld makes the decision: it checks the name against the policy, resolves it, re-checks the resolved address against the denies, and pins it. It then drives its host-side host-netproxy delegate to dial the pinned address, mints a socketpair, returns one end to the shim as a conduit, and the delegate splices the other end to the dialled socket. The shim splices the workload to its end, and traffic flows.

What crosses into the kernel is the conduit, a local socketpair end kenneld minted, never the dialled socket itself, which stays on the host. kenneld brokered the connection and touched no byte of its payload; the half-close and teardown ride the socketpair and the conduit's binder death notification (control-not-data-plane). This is the delegate pattern of chapter 2 and the binder gateway of chapter 3 in one path: a dumb host-side dialer doing the blocking I/O, the daemon as the decision point and nothing more, and the single controlled crossing being binder rather than a network route.

Routing every connection through the decision point buys three properties that a packet filter cannot. The policy lives in one place, in user space, where resolving and re-checking a name is ordinary code rather than a kernel program. The kernel only ever holds a name and never an address, so DNS rebinding is structurally impossible: kenneld resolves and pins under policy on each request, and a poisoned or shifting answer is caught before the dial rather than after. And the audit is free, because the component that logs the connection is the same one that decided it, with nothing to correlate after the fact. A name a kernel asks for and a name it is refused are the same event in the same log line. The whole of by-name egress policy lives in that decision, which is why DNS exfiltration is not a hole to be plugged but a path that does not exist: a kernel has no route to any resolver and cannot make a raw query, so there is no channel to smuggle through (T1.7).

8.3 The two ACLs

Egress policy is written at two layers, because two layers enforce it. The name layer is the proxy's: by host name, port, and protocol, evaluated by kernel with the resolver's answer vetted against it, the layer where the interesting policy lives because only user-space code can reason about a name:

```
[[net.proxy.allow]]
name      = "api.anthropic.com"
ports     = [443]
protocol  = "tcp"
reason    = "Claude API"
```

The kernel layer is a cgroup BPF and Landlock ACL by CIDR and port, with no names, because the kernel cannot resolve one:

```
[net.bpf]
families = ["AF_INET", "AF_INET6"]

[[net.bpf.connect.allow]]
cidr     = "*"
ports    = [443]
reason   = "https egress to any host"

[[net.bpf.connect.deny]]
cidr     = "10.0.0.0/8"
reason   = "no egress into the corporate RFC1918 range"
```

The two are not redundant. In the namespace modes the kernel is not routing by destination at all, the empty namespace already does that; what the kernel enforces there is the socket surface. A raw or packet socket would let a workload build and send frames directly, around the brokered path rather than through it, and a netlink socket would let it read or reconfigure the stack, so the family ACL denies all three at socket creation. Raw and packet sockets already need a capability an unprivileged kernel cannot obtain, which makes that denial the floor for a root-context kernel rather than the load-bearing control in the ordinary case, while the netlink denial bites even unprivileged. The same kernel layer gates every bind against the inbound ACL. In host mode, with no namespace boundary to lean on, the kernel CIDR ACL on connect becomes the egress gate itself.

The kernel ACL is author-writable in every mode, and the framework holds it to one direction: an author may narrow it but never widen it. A connect rule in a

namespace mode cannot reach past the proxy the namespace already pins the kernel to; it can only subtract further. The evaluation is deny-first, a deny match refusing regardless of any allow, so the narrow direction is the only one a rule can move the policy in. The kernel enforces the floor and the socket shape; the proxy enforces the reachable world.

8.4 The egress floor and its residuals

Beneath every mode sits a small set of denies the framework will not let a policy remove, and they are deliberately narrow. The mandatory floor is cloud metadata and link-local addresses, the server-side request forgery target whose reachability turns a fetch into a credential leak, denied even in `unconstrained` and `host`. It stops there on purpose. Private address space is not on the invariant list, because a kernel routinely and legitimately needs a local development server, a LAN database, an internal registry, and making private space permanently unreachable would be a floor that breaks more than it protects. A policy may add private denies where it knows it needs none; the framework imposes only the few that are never legitimate.

Two residuals are worth naming because the design refuses to paper over them. The first is that a destination allowlist cannot see inside TLS: a kernel permitted to reach a real API can carry data out inside its requests, and the answer is a tighter allowlist, not payload inspection the proxy was built specifically not to do (T1.8). The second is the honest price of keeping the daemon out of the data path. A conduit kennel mints for an approved connection lives as long as the connection does; there is no per-connection kill switch while the kennel runs, because a kill switch would have to sit in the byte stream to cut it, and putting kennel in the byte stream is the one thing the gateway design exists to avoid. Revocation is the whole kennel, frozen and killed through the cgroup, not a single conduit (T1.10). The control plane decides cleanly and completely; the price is that, once it has decided yes, the bytes are not its to reach back into.

8.5 Loopback isolation

The namespace modes give each kennel a small block of addresses of its own, a slice of the `127.0.0.0/8` loopback space and an IPv6 unique-local `/64`, allocated per user from a `subkennel` table that is the network analogue of `subuid`. The addresses are bit-packed and computed rather than written by hand, and they exist on both sides of the namespace boundary at once. Inside the kennel they are what its own loopback carries, so the workload binds, listens, and connects against them as it would against `127.0.0.1` anywhere. On the host the same addresses are added as aliases on `lo`, identical by construction and joined by no route, so a port the work-

load exposes shows up host-side at the kernel's own address and an operator's `ss` or `lsof` maps it straight back to the kernel that opened it.

This is the loopback mirror of chapter 2, and its point is attribution without plumbing: there is no routing, no address translation, and no interface beyond the aliases, so the same address reality holds on both sides and a kernel's sockets are observable from the host as themselves. The isolation is the namespace's: another kernel cannot reach these addresses, because each kernel's loopback lives in its own network namespace and is structurally invisible to the next; the operator's own shell can reach them through the host alias, which is correct, since the operator is the one in control. Raising the host alias is the single privileged step the namespace modes add, an address-add on `lo` scoped to the reserved space and carried by the same narrow sub-helper that holds `CAP_NET_ADMIN` for nothing else. The honest residual is at the mirror itself: a host-side listener standing at the kernel's address is reachable by anything on the host that can route to `lo`.

8.6 Binding and the inbound mirror

A kernel that runs a development server or a language server needs to bind, and the bind happens natively inside its network namespace, so the listener is reachable from within the kernel by ordinary loopback. The policy sits at the syscall: every bind is decided by the kernel bind ACL at the cgroup hook, and one that matches no rule fails there:

```
[[net.bpf.bind.allow]]
cidr   = "127.0.0.1/32"
ports  = [8080]
reason = "the local development server"
```

The bind ACL is the entire decision; exposing an allowed listener to the host is mechanism rather than a second policy, and that mechanism is the inbound mirror, the one place the binder data direction runs into the kernel rather than out of it. Egress has the kernel originating and so pulling outward; an inbound connection originates outside, so the leg that carries it inward runs from the daemon into the kernel. The mechanism is the mirror image of the proxy: a host-side delegate, the reverse of the outbound dialer, binds each exposed port on the host alias at the kernel's own address and accepts, and the in-kernel facade registers a callback once and then sleeps in the kernel, costing nothing while idle.

On an accepted connection the delegate mints a socketpair, splices the connection to its own end, and hands the other end to the daemon, which routes it to the sleeping facade as a one-way delivery that wakes it; the facade connects the workload's

native listener and splices. The daemon makes no fresh policy decision on the inbound path, because the bind ACL already decided what could be exposed at the moment the workload bound the port, and it touches no byte, because the delegate minted the conduit and does the splicing. The only descriptor that crosses into the kernel is a benign socketpair end; no host listener and no accepted connection ever does, so nothing crosses into the trusted base (`control-not-data-plane`). The direction is reversed knowingly: pushing into the kernel is the logical data direction for an inbound connection, and the callback the facade hands up is bounded by a death notification that stops the daemon pushing to a torn-down kernel, a one-way delivery that lets it never block on the facade, and a registration the daemon honours only for a port the policy already exposes.

8.7 Shaping the bind

The last of the inbound machinery is as much about not breaking tools as about safety. Much of the development-server ecosystem binds `0.0.0.0` by default, and denying that would break it, so the kernel rewrites instead: a cgroup hook on bind turns a bind to the wildcard address into a bind to the kernel's own private address, and `getsockname` reflects the rewrite, so a tool that asked for `0.0.0.0` reports that it is listening on the kernel's address and behaves correctly. A dual-stack socket is the trap underneath that: a v6 listener that also accepts v4 would slip the rewrite, so the kernel forces `IPV6_V6ONLY` on and intercepts the attempt to clear it, making a workload that wants v4 ask for it as v4. Two kernels can both bind the same port without colliding, because each binds inside its own namespace and the host mirrors land on distinct addresses, and the kernel's `/etc/hosts` shadows `localhost` to its own address so the tools that hardcode it still resolve to the right place.

What makes the shaping small is that the namespace has already removed the worst of what a bind could reach. A kernel has no LAN interface, no VPN or overlay interface, and no host `127.0.0.1` service inside its namespace to collide with or silently stand in for, because none of them were built into its network world (`construction-by-absence`). The bind ACL and these rewrites shape what is left, the kernel's own loopback, and decide which of its ports are mirrored out to the host. Through all of it the workload sees none of the binder mechanics underneath; the in-kernel facade is a translation layer and nothing more, and as a parser of the workload's own socket traffic it is treated as untrusted and fuzzed.

9. Local services

The network chapter brokered a kernel's reach to remote services through a facade in the kernel and a delegate on the host. The services already running on the workstation, the agent sockets and the session bus, are reached the same way, and they are a denser authority surface than the network is. A remote endpoint has to be found and connected to; a local agent socket sits at a known path, holds the operator's signing key or stored passwords, and answers anyone who can open it. The same broker pattern carries this resource class, and two of its members sit at opposite ends of a spectrum the previous chapter drew: a socket connection is a decision made once, at connect, while the session bus is a decision made on every message that crosses it.

9.1 The ambient surface

A normal user session is full of Unix-domain sockets that are pure capability. The ssh-agent socket signs challenges; the gpg-agent socket signs and decrypts; the keyring socket hands back stored passwords; the session bus socket reaches every service the desktop runs; the Wayland and PipeWire sockets carry the screen, the clipboard, the microphone, the camera; the Docker socket is root-equivalent. Many of them are unauthenticated by design: the access control is the socket file's permissions, so anything that can open the socket has the whole of what it offers. Inside a kernel that runs as the operator's own uid, those file permissions grant access rather than withholding it, and a confined workload that could open these sockets would have, silently, the operator's keys and the operator's session.

The session bus is the sharpest case, because a single grant of it is the whole session. A workload that can speak to the user's D-Bus can ask the file manager to read and write files, ask systemd to start an unconfined process in the session and so step entirely outside the kernel, read the operator's saved credentials from the secret service, send notifications that impersonate any application, and reconfigure the network. A bare socket grant for the bus is therefore not a small convenience, it is a grant of nearly everything the unconfined user can do, requested by proxy. The

resource class exists to give a workload the specific local service it needs without giving it the socket that would carry all the rest.

9.2 The brokered-connect facade

A Unix socket is reached, like a remote host, through a facade rather than through a path. The workload asks the `IAfUnix` node on its binder bus for a socket by the policy name it was granted, `kenneld` checks the name against the allowlist, performs the connect on the host side, and returns the connected descriptor into the kernel. The workload then speaks to the agent or service over an ordinary fd and never learns where the socket lives. No host socket path appears in the kernel's view at all, which is the property that matters: there is nothing to enumerate, nothing to probe, and nothing to connect to out of band, because the only way to reach a local socket is to name a grant and be handed the result (`mediate-use-not-reach`). A name the policy did not grant cannot be asked for, so default-deny is structural rather than a rule that has to remember every dangerous socket, and the decision and its audit land at the connect, on the call, not inferred from what the view happens to contain. This is the connect-time shape of the proxy from chapter 8: broker the connection once, hand back the descriptor, and step out of what flows over it (`control-not-data-plane`).

```
[[unix.allow]]
name = "gpg-agent"
real = "/run/user/1000/gnupg/S.gpg-agent"
shim = "~/gnupg/S.gpg-agent"
reason = "decrypt and sign with the operator's own gpg key; no key material
↳ enters the view"
```

The grant names a host socket at `real` and the path the workload finds it at under `shim`, and the workload opens the shim and learns nothing of the host side. The name is the whole interface: the policy grants `gpg-agent` and the broker resolves the rest, so a socket the policy did not name has no shim to open.

One high-value socket is deliberately not served this way. An `ssh-agent` reached through the facade would be a signing oracle: the workload could not read the key, but it could ask the agent to sign anything, for any destination, which is most of what holding the key would buy. Per-kernel SSH is routed through a re-origination bastion that binds each signature to an intended destination instead, so the agent socket is never the thing brokered. The facade carries the agent-shaped services where a blind connection is acceptable, the keyring and `gpg-agent` and the display and audio sockets, and the signing case is handled where the delegation can be made specific. What the facade does for these is a dumb connect: it resolves

9.3 The session bus is not a socket grant

a name to a host socket, performs the `connect`, and hands back the `fd` without parsing or filtering a byte of the protocol that then rides it. It vouches for nothing. A raw keyring or `gpg-agent` grant is therefore an operator's choice the operator owns, not a channel the framework stands behind, which is the line §9.4 draws: the framework refuses to *mediate* such a service and will not *build* a facade that parses and passes its messages, but a bare `connect` it did not have to parse is the operator's to make, loudly and against a warning.

Beneath the path-named sockets sits a second Unix-domain namespace the path rules do not touch. Abstract-namespace sockets are addressed by a name in kernel memory rather than a file, so no filesystem ACL applies to them, and a workload could otherwise reach a host service bound there with no path to deny. Land-lock scoping closes them: a kernel's domain is scoped so a `connect` to any abstract socket bound outside it is refused by the kernel, with no inspection of the address and no userspace dependency. The scope is defence-in-depth on top of the network namespace, which is the structural control, since a kernel that owns its own network namespace has an empty abstract namespace to begin with, nothing host-side having bound there. The two compose in one direction only: a kernel that asks to allow abstract sockets and to share the host's network namespace is a compile error, not a warning, because sharing the host stack puts the host's abstract sockets back within reach and the scope would be carrying a load the namespace was supposed to carry (T1.13).

9.3 The session bus is not a socket grant

D-Bus is brokered, never handed over as a socket, and the reason it cannot reuse the connect-and-stream shape of the proxy is the heart of the chapter. The proxy makes one decision, at `connect`, and afterwards the kernel holds an established socket it may stream over freely, because the security question was answered when the destination was approved. The session bus has a security question on every message: which service, which object, which interface, which method. The channel never becomes a post-decision stream, it stays security-relevant for its whole life, and a raw descriptor to the thing on the other end would hand the workload everything reachable through it. So a D-Bus message is mediated message by message, and the mediation is split across the trust boundary so that no trusted component ever parses a byte the workload controls.

```
[dbus.session]
enabled = true

[dbus.session.allow]
talk = ["org.freedesktop.Notifications"]
```

```
call = ["org.freedesktop.portal.Desktop=org.freedesktop.portal.FileChoose"
↪ r.OpenFile"]
broadcast = ["org.freedesktop.Notifications"]
```

The grant is at the granularity of the question. A `talk` entry names a whole destination; a `call` entry narrows to one `destination=interface.member`, so a workload may reach the file-chooser portal without reaching the rest of the portal service; and `broadcast` admits a signal only from a sender the kernel may already talk to. What the policy does not list, the table does not pass.

The in-kernel facade is the sole parser of the adversarial wire. It terminates the workload's bus connection, parses each method call, and emits a typed transaction carrying the vetted fields, destination and object path and interface and member and body, across the binder gateway. A delegate in the operator's own context holds the real bus connections, matches that typed transaction against the table the compiler built from the policy, and on a pass reconstructs a fresh, well-formed message from the typed fields and sends it. It never re-parses the workload's bytes, only reads the fields the facade already parsed. `kenneld` sits between them as the membrane and does the least of all: it neither parses the frame, so the D-Bus engine never enters its trusted base, nor applies the filter, which is the delegate's mechanical job, binding each connection to the in-kernel caller that opened it and relaying. This is the `parse-into-typed-form` discipline carried across a boundary (`parse-dont-validate`): the hostile-protocol parser is quarantined on the untrusted side, the authority that acts is on the trusted side, and only structured, already-parsed data crosses between them (`quarantine-the-unsafe`). Growth of the trusted base here is a decision point, not a parser.

9.4 Asymmetry, and the refused set

Outbound and inbound are not the same path, and keeping them separate is what makes the no-trusted-parser claim hold. Outbound method calls are adversarial and are parsed in the kernel as above. A reply to an approved call is trusted-origin, coming from the operator's own bus, so the delegate reads it and returns it and the facade hands the body to the workload as data, with no trusted component having parsed kernel-controlled bytes to produce it. Signals are broadcast by host services and are delivered only under a match-rule allowlist: a kernel receives a signal only from a service it is already allowed to talk to and only where it registered a matching subscription, so the delegate filters the bus's signal stream down to that set before forwarding. A signal from a service the kernel may not talk to never arrives, which is what stops a workload from watching the bus to learn what the operator is running. Conflating the two directions into one bidirectional filter would have smuggled an adversarial-wire parser into the trusted delegate, which is the thing the split exists to prevent.

Some services are not default-denied but refused outright. Everything a policy does not grant is already denied, and a user may override that denial for a service that is dangerous but conceivably legitimate, a connectivity query or a mount for a media workload, with a loud warning and a threat tag (`footgun-warn-dont-forbid`). Above that sits a small, named set the facade refuses regardless of policy, where naming the service in an allow list is a compile error rather than a warning. What puts a service on it is not the danger of the capability but the act the facade would have to perform to broker it. A `talk` or `call` grant is not a dumb connect: the facade parses the protocol, matches each message against the table, and passes the ones that clear it, which is the framework vouching that a given retrieval or call is a legitimate one. For a secrets protocol that vouching is attestation the framework cannot back (`authentication-never-attestation`): to mediate `org.freedesktop.secrets` is to stand behind which password fetch the workload may make, and the monitor has no ground to stand on. So the secret service is refused at compile, not because the passwords are dangerous but because brokering the protocol means vouching for its use. Session and process control are on the set for the same mechanism reason read the other way: to mediate them is to broker a spawn escape or session takeover, which would defeat the monitor's reason to exist. A framework that claimed confinement while mediating either would be theatre. The set is deliberately small and deliberately closed.

The rule under it is the line §9.2 named from the other side, and it is worth stating plainly because it does not fall where a reader expecting a capability-scoped refusal would put it. The framework refuses to *mediate or vouch*: it will not build a facade that parses a secrets or signing protocol and passes selected messages, because parsing-and-passing is vouching. It *warns* where it merely *connects and steps out*, because a bare connected fd carries no claim the framework made. The consequence is an asymmetry the text will not paper over: the D-Bus secret service is refused at compile, while a raw connect to the same secrets daemon's own socket is a warned `[[unix.allow]]` grant the operator may write. The two are not in tension once the refusal is read for what it is, a bound on what the framework will *build and stand behind*, never a bound on what the operator can *reach*. An operator who wires the raw socket has the passwords; the framework simply did not hand them over under a claim that the transfer was safe. Holding the operator's own expression of intent above a fully safe posture is the deliberate trade (`footgun-warn-dont-forbid`), and the raw route is loud, tagged, and surfaced in the risk report precisely so the choice is made in the open.

```
[dbus.session.allow]
talk = ["org.freedesktop.*"]
```

```
| # the compiler refuses this: `org.freedesktop.*` reaches  
| # `org.freedesktop.secrets`, which cannot be brokered to a kernel
```

A wildcard reaches the refused set as readily as an exact name, so the check is on what the pattern admits, not on what it spells. The operator cannot widen into the closed set by writing it broadly, and the refusal is a compile error rather than a runtime denial, caught before the policy is ever signed.

10. Cryptographic services

The local-services chapter brokered the agent sockets a workstation runs, and set one of them aside. An ssh-agent could not be handed through the same facade as a keyring or a display socket, and the reason it could not is the subject here. Signing is the capability where the broker pattern meets its edge: for one kind of signing the edge is a construction problem the framework solves, and for another it is a line the framework will not cross. The difference between the two is the whole of the chapter, and it is the difference between proving who you are and vouching that something is so.

10.1 The signing oracle

The obvious way to let a kennel use the operator's SSH key is to expose the agent socket into it, and it is wrong in a way that is worth being precise about. The ssh-agent protocol carries an opaque blob to be signed, not a destination. An agent, or a filtering broker placed in front of one, can bound which key signs but never which host the signature authenticates to, because the host is nowhere in what the agent sees. A workload that holds the socket ignores the curated client config it was given, opens the socket itself, and asks for a signature over a challenge it built for a host of its own choosing. The key that was meant for one server now authenticates the workload at every server that accepts it. The agent is a destination-blind signing oracle, and filtering the client the workload is free to bypass constrains nothing. The conclusion the design draws is flat: the workload holds no key and no agent socket, and it does not choose its own destination.

10.2 Re-origination

What replaces the agent is a bastion that re-originate the connection, and the mechanism turns on a synthetic key that is itself the destination. For each destination a kennel is granted, the toolchain mints a throwaway ed25519 keypair at compile time; the private half lands in the kennel's constructed `~/.ssh`, and the public half is recorded in the signed policy and bound to a forced command that

already names the destination and the operator's host-side options for reaching it. The binding is one `authorized_keys` line:

```
restrict,pty,command="ssh '-i' '~/ssh/id_gh' -- 'git@github.com'
↳ \"SSH_ORIGINAL_COMMAND\" ssh-ed25519 AAAA...
```

The destination is the literal `git@github.com` baked into the command, fixed by which synthetic key authenticated and never parsed from anything the workload sends. The workload controls only `SSH_ORIGINAL_COMMAND`, the remote command, forwarded as a single argument; it cannot rewrite the line, reach any other host, or authenticate at all with a key the policy did not mint. There is no oracle, because each key is a capability for exactly one destination.

That line lives in no file the bastion could read or a workload could rewrite. On every authentication the bastion's `sshd` hands the offered key to a root-owned helper and takes the answer as the key's authorisation:

```
AuthorizedKeysFile none
AuthorizedKeysCommand /opt/kennel/bin/kennel-akc %t %k
AuthorizedKeysCommandUser root
ExposeAuthInfo yes
```

The helper, `kennel-akc`, forwards the offered key to the running `kenneld` over the control socket and prints back the forced-command line bound to it, or nothing, in which case the login is refused. The bindings exist only in the daemon's memory, sourced from signed policy, so the answer is the daemon's live verified edge state rather than a file the unprivileged bastion user could edit, and any failure at all, no daemon or a garbled key or an unexpected reply, prints nothing and fails the login closed. The helper is installed root-owned because OpenSSH refuses an `AuthorizedKeysCommand` that is not. The helper read the offered key from the `%t %k` arguments; `ExposeAuthInfo yes` separately hands the forced command the identity of the key that authenticated, as `SSH_USER_AUTH`, so the lookup and the running session learn the key by different routes, `argv` for the one and the environment for the other.

The forced command is the only thing a session may do, and the rest of the bastion's configuration is hardened to keep it that way:

```
PubkeyAuthentication yes
PasswordAuthentication no
PermitRootLogin no
UsePAM no
```

10.3 Where re-origination stops

```
AllowTcpForwarding no
X11Forwarding no
AllowAgentForwarding no
PermitTunnel no
PermitOpen none
AllowStreamLocalForwarding no
Subsystem sftp /bin/false
```

Publickey login only, no agent to forward and so none to re-export as an oracle, every forwarding and tunnelling avenue shut, and SFTP wired to a binary that does nothing. A session that authenticates runs its one forced command and has no other move. The file is the surfaced template, admin-tunable through a root-owned cascade but never weakenable by the workload, which cannot write the paths it resolves from.

The kennel reaches the bastion over the same egress proxy chapter 8 builds, with no SSH-specific transport of its own: the kennel's network policy allows only the bastion's loopback endpoint as a host service, and `host-netproxy` dials it as it dials any other egress target. On a connection the bastion's forced command runs, as the operator and with no added privilege, an `ssh` to the bound destination, dialed host-side and so outside the kennel's egress entirely, and that outbound `ssh` signs with the operator's real key from the operator's own host-side store, against the destination the bastion fixed. The real key never enters the kennel. The kennel's `~/.ssh` is a `tmpfs` subtree holding only the disposable synthetic keys and a generated read-only config that points every host at the bastion; the operator's real keys, real config, and real `known_hosts` are absent by construction rather than by a deny rule, never built into the view (`construction-by-absence`). Because the real signature is bound host-side to a destination the workload could not choose, reusing one host's key against another is not a rule that can be broken but a path that does not exist (T1.6).

10.3 Where re-origination stops

Re-origination works for SSH because a signature there authenticates to a host, and a host is a property the bastion can verify and bind a key to. Commit signing has no such property, and the same fix does not reach it. A signature on a commit is not aimed at a destination that could be fixed; its worth is that the operator's identity vouches for the bytes, and it incorporates itself into the hash it covers, so there is nothing host-side a facade could check and bind its decision to. Behind the technical mismatch is the axiom that settles the matter. A kennel exists to contain code the operator does not trust. A signature made as the operator, by code running inside that boundary, stamps the operator's verified identity onto whatever

the workload presents, a release, a forged commit, malware, and it does so permanently, not for one live session. That is a trust root placed inside the very boundary the project exists to confine, which is incoherent: a claim with nothing behind it.

So the framework does not build a signing facade, and it refuses to pretend one is safe. An exposed gpg-agent is the same destination-blind oracle as the ssh-agent, worse for stamping a durable artefact rather than authenticating a transient session. A grant of it compiles and runs, unlike the bus services chapter 9 turns away at compile, and the split is the same rule §9.4 drew: the raw agent socket is a dumb connect the operator owns, not a facade the framework builds and vouches for, so it warns where a mediated signing service would be refused. The compiler derives a risk from it that `kennel policy risks` surfaces against the operator's reason: a loud footgun the operator may still pull, not a sanctioned path (`footgun-warn-dont-forbid`). What the axiom forbids is attesting as the operator, not signing as such, and an operator who does want a kennel to sign has a clean way to get it. The answer is a separate identity: an ordinary signing keypair the operator generates for the workload, distinct from their own, its private half granted through the ordinary read allowlist as a readable file in the view. No new machinery and no agent, so no oracle to abuse; the workload reads the key and signs in-process as that identity, and the signature is worth exactly the trust of an identity the operator stood up on purpose, read by a verifier as the workload's and never as the operator's. The line the axiom draws runs between those two identities, not against signing itself, which is the design volume's rule (§8.2) in mechanism: refuse the standing to vouch as the operator, grant without ceremony the standing to vouch as an identity the operator raised for the purpose. Where no separate identity is wanted, the plain path remains: the workload commits unsigned, and the human signs on review, before the push, from outside the kennel.

10.4 The line, generalised

The split under SSH and signing is the general shape of every capability the monitor brokers, and naming it keeps future facades honest. A capability is authentication-shaped when it is a constrained, host-verifiable act the monitor can bind to a checkable property: may I reach this host, may I open this connection, may I render this frame. Those are what interposition exists to carry, and the bastion is the worked proof that a dangerous one can be made safe by binding it to the property that makes it checkable. A capability is attestation-shaped when its worth derives entirely from the trust of its origin: vouching, signing, issuing a secret, trust that this is so. A kennel's origin is confined and untrusted by construction, so a kennel that attests to others is a trust root in the wrong place, and the category is refused rather than merely unbuilt (`authentication-never-attestation`). No broker the monitor grows, present or future, may be a secrets broker or a signing service, and routing the act through a keyring or a TPM or a vault does not rescue

it, because it only moves the attestation to a claim of being authorised to retrieve on the workload's behalf, which is authority the broker still cannot vouch for.

The positive form of the same line is where trust material legitimately enters a kennel. What a workload needs to hold, it holds because the operator placed it there: a secret a kennel needs arrives as a signed construction parameter, present in the view because construction put it there, never handed to the kennel by a peer at runtime. A hardware token follows the same rule from the other side. Its touch is the operator's presence, so it is used host-side by the bastion's outbound `ssh`, where the touch happens on the operator's own hardware, and the device is never bound into the kennel, because a workload that could drive the authenticator directly would have turned the presence check into one more thing it signs.

11. OCI images

An OCI image is a poor fit for the way Kennel builds a world, and Kennel runs images anyway. It runs them because images are how workloads are shipped today, the common denominator nearly every container tool and deployment assumes, and a confinement framework that could not accept the format the world has settled on would be confining nothing anyone runs. Confining them is a concession to adoption, not a claim that the model is good, and the work is to take the format the world hands Kennel and confine it as tightly as that format allows.

Every chapter so far assumed the kennel's world was built by Kennel out of host parts: a fresh tmpfs root, the host's own /usr borrowed read-only, an execute allowlist naming each binary. The OCI model changes the ground under all of it. The root is no longer constructed, it is an image the operator supplied, and the image is untrusted by the same logic that makes the workload untrusted. What changes when the floor of the world arrives as a single third-party artefact, and what has to be rebuilt because it does, is the subject here.

11.1 Booting an image without a daemon

The obvious way to run an image is to talk to a container daemon, and Kennel does not, because the daemon is the wrong thing to trust. A Docker daemon is a root-equivalent orchestration engine, and the two ways to reach it both fail the premises. Proxying its socket puts a large stateful protocol parser in the host trusted base in front of a `create-container` verb that is host-root. Running a rootless engine inside the kennel hands the workload `unshare` and `mount`, the two surfaces container escapes most often turn on. Kennel boots an unpacked rootfs directly under its own namespaces, as inert filesystem content, and takes neither bridge.

The `fetch` and the `run` are split into two verbs so that everything that parses the image runs where a parser bug is contained:

```
kennel oci build <name> -- <image-ref>    # fetch + unpack into the store
kennel oci run    <name>                  # boot the named entry under policy
```

11.2 The image as a declared substrate

The build pulls and unpacks into a per-operator store; the run resolves the named entry and boots it under its signed policy. Every parser the image touches, the registry protocol, the manifest, the tar extraction, the image's own runtime config, runs inside a confined kennel at workload authority, so a malicious image breaks a sandbox that was already disposable and never reaches the daemon (*quarantine-the-unsafe*). The trusted base is the size it was. The split also partitions the policy grammar: an OCI policy carries a mandatory `[rootfs]` block and never a `[workload]`, a normal policy the reverse, and the two are mutually exclusive at the block level rather than by a runtime check (*make-invalid-unrepresentable*). The substrate-replacement primitive exists only in the grammar the OCI verb consumes, so the universal model most workloads use never grows it.

11.2 The image as a declared substrate

Running a third-party image is a trust decision, and Kennel makes the operator own it rather than making it for them. The operator declares an image as the execution substrate the way they declare a host directory writable: an input the policy names and the signature covers. The grant is loud, in the class of host networking and device passthrough, carrying a required reason, and the compiler derives a substrate-trust exposure from its mere presence and surfaces it in the risk report against that reason (*footgun-warn-dont-forbid*, *tell-me-why*). Kennel does not vet the image, launder its contents, or assert anything about bytes it did not pin (T3.8).

What it does assert is provenance. The build pulls by digest, resolving a tag to a digest at build time and recording it, so even a tag-built image freezes to a pinned hash, and a build with no resolvable digest is refused, because a substrate with no provenance cannot be told apart from an error (*refuse-to-start*). The store entry the build writes is what the run consumes:

```
<store>/<name>/
  rootfs/          the unpacked image filesystem
  config.json      the image's runtime config
  digest           the resolved image@sha256 the build pulled from
  policy.toml      the scaffolded run policy (operator completes + signs)
```

The first three are the integrity unit, since the launcher trusts the config for the entrypoint and env; the policy sits outside it, covered by its own signature. The signature then covers the substrate, not the invocation: it pins the image by digest and the policy grants, but not an exact command line, so the same signed OCI policy can run different commands where the standard model's signed argv could not.

Provenance is the floor, and content integrity is opt-in hardening layered above it: a content-addressed store entry the runner verifies before pivot, and per-file verity that makes every library load and the config the launcher reads tamper-evident. The operator who needs those rungs pays for them, the one who does not is not blocked.

11.3 The overlay

An untrusted image cannot simply be the root, because Kennel needs a few files to win over whatever the image ships and a scratch image ships almost nothing. The view is built as an overlay. The image is a read-only lower, never the writable layer, because an image that could be written would take the workload's writes into the digest-pinned store entry and dissolve the integrity unit. Above it sits a small Kennel layer holding the handful of `/etc` files the resolver and the persona need, so they win by layer precedence rather than by overwriting anything, a real file outranking an image symlink at the same path with no unlink-and-replace. Below it sits a scaffold of empty mountpoint directories, so a distroless image that ships no `/proc`, `/dev`, or `/tmp` still boots.

```
lowerdir = kennel-etc : image : scaffold    # leftmost wins; the image is a
↔ read-only lower
upperdir = <ephemeral | persisted>         # always present; never the image
```

Those `/etc` files are a default and not a boundary: `/etc` is writable-through, a workload may copy a file up into its own layer, and it harms only itself, because what enforces confinement is the namespace, the BPF, the uid map, Landlock, and seccomp, never the content of a file.

There is always a writable upper, because an image expects a writable root, and the operator chooses only whether it survives teardown. The default discards it. Persisting it is the loud value, because a persisted upper accumulates divergence outside the integrity ladder, and the risk report says so. Whole-tree immutability is not the absence of an upper but a lock expressed over it, which is the next section.

11.4 Closure-lock

The unprivileged build cannot preserve the image's ownership. With no subuid range it cannot chown extracted files to the image's uids, so it flattens every inode to the single persona uid, and that does more than drop root: it erases every ownership boundary inside the image at once, root from app and app from everything, because there is now one owner. The workload runs as that owner, so nothing in

the tree is read-only to it by permission, its own binaries included. Left there, the image under Kennel would be strictly more permissive than the same image under Docker, where the app uid still cannot write root-owned `/usr`.

Closure-lock restores the one boundary the flatten destroyed, that the executable surface is not self-writable. It cannot be a permission scheme, because after the flatten there is no foreign owner left for permissions to protect, and it is not Landlock, because Landlock rights are additive and a broad write over `/` cannot be subtracted at `/usr`: default-writable-except-the-closure is not a shape a sub-path rule can express. The lock is a set of read-only mounts over the merged root, so a write to a locked path fails with `EROFS` regardless of owner, and the workload cannot undo it, holding no `CAP_SYS_ADMIN` in its namespace and barred from mount by seccomp. The locked set is derived at build and written into the policy in plain text, where the operator reviews and signs it:

```
[rootfs]
image    = "ghcr.io/org/app@sha256:..." # pinned by digest
reason   = "vendor image"                 # required; derives T3.8
readonly = ["/usr", "/bin", "/lib"]        # closure-lock: read-only mounts,
↳ write => EROFS
writable  = ["/usr/lib/python3.12"]         # carve a hole back out; a
↳ derived risk
```

The build proposes the resurrected boundary and the signature makes it real.

The derivation turns on one question, whether the image's `config.User` is non-root. An image that runs as root declares a writable substrate by intent, so nothing is locked, which is the same posture Docker gives a root container and the reason runtime `apt` and global `pip` keep working. That is a deliberate write-xor-execute waiver: the substrate is granted execute coarsely, with no per-binary allowlist inside an image, and with nothing locked the same paths are writable and executable at once, so the separation the standard run model holds between a read-only `/usr` and a default-deny execute does not hold here. A non-root image earns the separation back: the coarse filesystem closure, `/usr`, `/bin`, `/sbin`, and the library directories the merged-usr symlinks resolve into, is locked read-only, so its executable surface stays equal to the pinned image for the session. The estimate is high-level and has two known holes, an image that drops privilege in its entrypoint reads as root and gets no lock, and app code outside the system directories stays writable; the operator closes those by hand with a writable carve-out for over-reach and an added lock for under-reach.

11.5 What holds, and what is waived

The line worth ending on is which guarantees survive an untrusted root and which the operator knowingly gives up. Everything the confinement limb provides survives unchanged, because none of it ever depended on where the substrate came from: the network namespace and its egress boundary, the brokered crossings, the proxy and the network policy, the masked identity, the constructed device allowlist, seccomp, and the closure-lock boundary for a non-root image. The workload gains no capability, no `mount`, no nested user namespace, and the trusted base does not grow, because the launcher and every image parser run at workload authority and not in the daemon.

What the operator waives is content. The image supplies its own runtime closure, the loader and the libraries and the entrypoint, and Kennel pins the provenance of the whole rather than the identity of each part: the digest is the assertion, and the dynamic closure beneath it is unhashed unless the verity rung is enabled. The posture claim is therefore confinement, not content integrity. An OCI kennel is contained exactly as tightly as any other, and what the operator accepted, the moment they named an image as their floor, is that the bytes inside that floor are the image's word and not Kennel's.

12. The remaining surfaces

The resource chapters covered the large outward-facing classes, the things a workload reaches for. What is left is smaller and inward: the surfaces that are properties of the box the workload runs in rather than capabilities it asks for, each one minor next to the filesystem or the network and each one load-bearing in the threat model. They are what the workload can observe of its neighbours, what it inherits at startup, how its own terminal can be turned against the operator, and what it might use to climb or to map the machine.

A note on what is settled here. These surfaces are shipped and enforced as described, with one class of exception: a few convenience toggles over them are not built yet, and the parser rejects them today. A file to seed the environment from, a template to seed the home's dotfiles, and named shortcuts for GPU and hardware-token access are all roadmap. The controls they would name are reachable now through the general primitives, a forced environment variable, a device passthrough, a socket grant, so nothing in the threat model waits on them, and the toggles are sugar over mechanisms that already hold. Where the text reaches one of them, it says so.

12.1 What the workload can observe

A workload that can look into other processes need not reach the network to do harm. One that can `ptrace` the operator's shell reads its memory directly, every password and decrypted key and session token in it; one that can read another process's `/proc/<pid>/environ` lifts whatever secret was passed to it that way; one that can signal arbitrary processes can kill the daemons that were watching it (T1.1). The controls against all three are structural rather than declared. The PID namespace is the strong one: a process the kernel cannot see in `/proc` it cannot name to `ptrace` or to `kill`, and `/proc` is mounted so that even visible entries reveal nothing of their owners. Signal delivery is fenced a second way on a recent kernel, by a Landlock scope that refuses signalling any process outside the sandbox even when a PID is guessed, the same scoping family that closed the abstract-socket namespace two chapters back.

The policy carries `[unsafe.pttrace]` and `[unsafe.signal]` tables, and they are worth being clear about, because they are advisory. They state which targets a workload may reach, and that statement is real in that the diff shows it and the operator signs it, but the enforcement underneath is the PID namespace and seccomp, not the tables. Declaring one compiles with a warning (`footgun-warn-dont-forbid`): it records an intent, it does not impose a control the structure was not already imposing. The distinction matters wherever these surfaces appear, because several of them are held by construction and merely named by policy, and reading the policy as the enforcement would overstate what a table can do.

12.2 What the workload inherits

The environment a process starts with is a quiet credential surface, because the operator's shell is usually full of secrets, `AWS_SECRET_ACCESS_KEY`, `GITHUB_TOKEN`, an agent socket path, and a child inherits all of them by default. Kennel does not inherit and filter, it synthesises. The spawn builds the workload's environment from policy and execs with that, never consulting the parent's environment as a source of truth, so the default is empty and every variable present is one policy named, set in `[env].set`. This is the denylist lesson applied to the environment: curating the dangerous variables out of the parent's set leaks the one the list forgot, while building the set from nothing leaks only what was written down (`construction-by-absence`). A workload that truly needs a dynamic non-secret like `TERM` opts that single variable in, visibly, as a discouraged exception (T1.1).

The run context around the environment follows the same rule. `$PATH` is set from the execute allowlist rather than inherited, because it means nothing except as an index into the binaries that allowlist already permits. The login shell is policy-selectable but must be a binary the allowlist permits, so naming a shell the kennel could not then run is a compile error and not a runtime surprise (`make-invalid-unrepresentable`). The shell-init files are synthesised from policy and rebuilt every spawn, in a home that is the kennel's own and never the operator's. That rebuild is a security property rather than a convenience: a persistent, workload-writable `~/ .bashrc` runs on every future interactive shell in the kennel, which makes it a self-persistence vector, so the home is reconstructed by default and a policy that wants durable state names the exact paths that survive. Persisting a working directory is the ordinary low-risk case; persisting a shell-init file is a deliberate, named, diff-reviewed choice, and even then the blast radius is this kennel's own later runs, re-entered under the same confinement.

12.3 The terminal

A workload running in a terminal can turn that terminal against the operator two ways, and they need different answers. The first is input injection. The `TIOCSSTI` ioctl asks the kernel to push characters into the controlling terminal as though typed, which a confined process would use to queue commands that run in the operator's shell after the kernel exits. Kennel closes it by removing the target rather than policing the call: the workload's controlling terminal is a node in the kennel's own isolated devpts, and the operator's `/dev/tty` is not in the view at all, so an injection reaches only the workload's own session, which is the workload's to disrupt (construction-by-absence). No syscall filter is needed, because there is nothing on the other side to inject into.

The second is output. A workload writing to its terminal can emit escape sequences that act on the operator's emulator rather than printing, OSC 52 to read and write the system clipboard, OSC 9 and 777 to raise notifications, the device-control bands to carry emulator-specific commands, all of it an exfiltration and injection channel with nothing to do with `TIOCSSTI`. The output is filtered on its way to the operator's real terminal, the dangerous set dropped and the benign passed, and where that filter runs is the careful part. The daemon's PTY broker pumps the workload's output as raw bytes and never parses it, so the parser that interprets a hostile escape stream, a streaming vte state machine, lives in the attached client and not in the trusted base (quarantine-the-unsafe); the daemon conveys the decision and the client enforces it. The workload cannot choose its client, so it cannot opt out, and a consumer that bypasses the filter only exposes its own terminal. The filter is best-effort and not a proof: it shuts the low-effort clipboard and notification channel (T2.6) rather than every conceivable desync, and running a kennel in a terminal you trust stays the backstop.

The controlling terminal an interactive run needs is built to preserve that isolation. A real terminal is required for job control and full-screen tools, but forwarding the operator's own would hand the workload the very `/dev/tty` the input defence removes. So the pty is allocated from the kennel's own devpts after the pivot, the seal makes it the workload's controlling terminal and hands the master not to the CLI but to the daemon, which holds it in a per-kennel broker and pumps it raw to whatever client is attached. Because the master lives in the daemon, the operator can detach and reattach without ending the workload, the console model rather than an exec into the live sandbox: no second process is injected past the confinement, only a terminal taken over and handed back.

12.4 Capabilities, mounts, and the floor

The last surfaces are the ones a workload might use to climb or to map the machine, and the answers are uniform and dull by design. Linux capabilities are dropped to nothing: the bounding set is emptied so a `setuid` binary cannot raise the workload past its uid, `no_new_privs` is forced on as the execution chapter required, and the schema warns at the mere listing of a capability, because a confined uid-1000 workload that needs one is almost always a design error (`split-the-uid`). The mount table is narrowed by the same mount namespace the constructed view already uses: the workload sees only the mounts it was given, none of the operator's removable media or cloud sync or loop images, and a `mount` call fails for want of the capability just dropped.

Beneath those sits the syscall floor. Most of what Kenel confines reads better as a resource ACL than as a syscall filter, so `seccomp` is not the main event, but it closes the gaps the ACLs cannot reach: the abstract-socket connect Landlock has no word for, and a denylist of calls with no legitimate use in a kernel and a long history in exploit chains, `userfaultfd`, `bpf`, `ptrace`, the `process_vm` pair, `mount` and `pivot_root` and the rest. The cgroup the kernel runs in is one it cannot leave, which matters because leaving it would shed the BPF filters attached to it, so membership is an invariant the operator does not set and the workload cannot change. None of these is a knob the policy author reaches for; they are the floor the surfaces above stand on.

PART III

The interplay

13. Dynamic spawning

The resource chapters confined a kennel against everything it reaches. What follows is kennels in relation to one another, and the first relation is the simplest: one kennel asking for another. An agent that does real work needs helpers, a tool runner, a sub-agent, a server speaking some stdio protocol, and a confined workload has no authority to build a kennel of its own. How it gets a second kennel without the power to make one is the chapter.

13.1 The trifecta, split

An AI agent is useful only with both code execution and network reach, and a single workload holding both is the breach. A prompt-injected agent that can run code, see private data, and reach the network has everything an exfiltration needs in one process, the lethal trifecta in one place. The industry's answers are a virtual machine per task, heavy to orchestrate, or a container engine run inside the sandbox, which is root-equivalent and leaks. Kennel has a cheaper primitive to hand, because it can already build a tightly scoped kennel and wire a channel to it. The move is to split the trifecta across siblings. The agent holds neither capability: it cannot run code and it cannot reach the network. When it needs one, it asks the daemon to spawn a sibling that holds exactly that one, a tool kennel that executes with no network, or a fetch kennel that reaches the network with no execute grant, and talks to it down a channel. No single kennel ever holds both halves of the trifecta, and the agent that orchestrates them holds neither.

13.2 Request, don't author

The rule the rest of the chapter rests on is that a workload cannot introduce policy at runtime (`request-dont-author`). It does not write a child policy and ask the daemon to run it. It names a template the operator signed into the trust store, and it writes only the fields that template's manifest declares mutable. The signature was checked when the operator installed the template, so a spawn is a reference to a capability the operator already consented to, never a new grant authored by

13.2 Request, don't author

confined code. The agent's whole control over a spawn is the writes it makes to those declared fields, and that surface is the entire attack surface of the operation.

```
[spawn]
max_instances = 8
reason = "the orchestrator instantiates fetch, build, and tool helpers per
↪ task it is handed"

[[spawn.allow]]
template = "net-fetch@v1"
mutable = ["net.proxy.allow"]

[[spawn.allow]]
template = "argv-tool@v1"
mutable = ["workload.argv"]

[[spawn.allow]]
template = "untrusted-build@v1"
```

The grant carries `max_instances`, the concurrent-instance ceiling the compiler requires so the delegation cannot become a fork bomb, and a `reason`, because the waiver is loud and validated. Each `[[spawn.allow]]` names one versioned template the operator signed into the trust store, never a capability, and where present a `mutable` list narrows the requester to a subset of the fields that template already declares writable. `net-fetch@v1` exposes the proxy allow-list, itself bounded to a fixed set of package-registry destinations; `argv-tool@v1` exposes the command line, fenced by the template's `[exec]` floor; `untrusted-build@v1` is instantiated with no field open at all. The templates carry the authority, and the agent writes only into the blanks each one fences.

The fence is the template's, not the grant's, and it bounds values and not just which field is open. The `mutable` list opens `net.proxy.allow` for `net-fetch@v1`, but the template's own signed manifest declares what may go there, a fixed match set the spawn selects from:

```
# in net-fetch@v1's signed manifest
[[mutable]]
field = "net.proxy.allow"
match = [
    "pypi.org:443", "*.pypi.org:443", "files.pythonhosted.org:443",
    "registry.npmjs.org:443", "crates.io:443",
    "github.com:443", "ghcr.io:443",
    ...
]
```

So a spawn may add `pypi.org:443` and cannot add `exfil.example:443`: the field is a selection from a signed allow-set, never a blank the requester fills freely. The grant opens the field, the template bounds the values, and neither the agent nor the requester can write past the set the operator signed.

What the agent sends is a patch, a set of field-path and value pairs naming manifest fields, never a candidate document. The daemon rejects any path the manifest does not open, checks each value against that field's declared bound, and applies the survivors onto the resolved template, so the instantiation differs from the signed template only within the manifest and nowhere else (`make-invalid-unrepresentable`). The shape matters as much as the rule: because the agent submits writes against named fields rather than a whole policy, no adversarial policy document and no deep tree comparison ever enters the daemon, and the parser the project keeps out of its trusted base stays out here too (`quarantine-the-unsafe`). The capability floor of every spawn is the signed template's, and no field the manifest leaves frozen can be moved by any patch. So that the agent need not probe for its limits, the grant is readable: it can ask the daemon for its own spawn surface and be told which templates it may name and which fields it may write, with their bounds, and fill the fenced blanks rather than discover them by firing speculative spawns. The relation it creates is lateral, as every relation in Kennel is. The daemon is the spawner; requester and spawned kennel are siblings joined by a channel and a brokered lifetime, not a parent owning a child, and the requester cannot `ptrace` or signal the kennel it asked for, because reaching into it would defeat the isolation the spawn existed to create.

13.3 The handoff, and the daemon out of the path

The spawn itself is an outward broker of file descriptors, the same shape as every other thing the daemon brokers. The requester sends a `SPAWN` transaction naming the template and carrying its patch and no descriptors, with its reply flagged to receive them. The daemon validates the grant, resolves the named template from the trust store and verifies it against the signature the requester's own compiled policy recorded, re-checks that the template is eligible to be a spawn target, and applies the patch, all in its verify path and never in a policy compiler. Then it mints the channel, a socketpair for the bidirectional protocol and a separate pipe for the spawned kennel's standard error, injects the far ends into the new kennel's supervision plan, and returns the near ends to the requester in the reply. Nothing flows into the daemon in any of this: node `o` issues descriptors outward and never accepts them inward, and a spawn is one more case of that invariant, not an exception to it. The spawned kennel's `init` boots and seals it and, as its last act before `execve`, places the injected ends on the workload's `stdio` and runs the template's `entrypoint`.

Once the spawned kennel is running, the daemon and the bus are gone from be-

tween the two. The tool reads and writes its protocol natively over stdio, the requester reads and writes its end of the socketpair, and the bytes move kernel to kernel with nothing in the middle, the same control-plane-only discipline the network proxy keeps (`control-not-data-plane`). The protocol that rides the channel is opaque to Kennel on purpose. The spawn primitive is a generic transport for a confined stdio service, and a model-context server is one convention that rides it exactly as a language server or any other stdio tool would; teaching the daemon to understand the protocol's messages would put a per-message mediation surface and a dependency on someone else's evolving specification inside the trusted base, which is the thing the single chokepoint exists to refuse. Kennel writes no first-party interposer either, because a first-party one would re-import the exact parser it just kept out; an operator who wants tool-level mediation runs an existing proxy for it, confined in a disposable kennel between requester and tool like any other vendored code. The separate pipe for standard error is the small detail that makes the channel trustworthy: a traceback or a panic in the tool leaves on its own descriptor rather than corrupting the framed protocol, and the spawned kennel's own pre-execve failures, a seal that will not apply, a filter that will not compile, route to the host audit instead, so the requester sees a clean end-of-file for an infrastructure fault and tool output only once its own tool is the thing running.

13.4 Ephemerality, fate-sharing, and the depth bound

A spawned kennel leaves no trace on the host, and that is a separate property from the trust argument rather than a consequence of it. The instantiation is built in memory, because no child policy is ever authored to write to disk; the kennel takes a transient name that consumes no operator namespace; its root is ephemeral and it cannot write host disk, and what artifacts pass between siblings pass as memory objects over the channel, charged to and bounded by the spawned kennel's own cgroup. It also must not outlive its purpose, which three reapers enforce on the one freeze-and-kill mechanism. When the requester is done it closes the channel and the tool reads end-of-file and exits, the graceful path; if the requester instead crashes or is itself reaped, the daemon sees its binder session drop and kills the spawned kennel regardless of whether the tool noticed, the backstop for a hung tool; and independently of the requester, every spawn-target template declares its own lifetime bound, so a tool whose requester holds the session open forever still reaps itself at its declared limit. The concurrent ceiling is held by an atomic claim taken before construction and released on every terminal outcome, including a build that aborts before the kennel ever reaches the reaper, so a boot failure cannot leak a slot.

Spawning is one level deep by rule. A template that may be named as a spawn target may not itself carry the spawn grant, which is a fork-bomb prohibition and

not a feature deferred for later: recursion would turn the concurrent ceiling from a global bound into a per-node one and the lifetime coupling from a single hop into a chain, so the depth bound keeps both flat by construction. Eligibility to be a target, that a template carries no spawn grant of its own, declares a lifetime and resource ceilings, and declares the manifest that fences the write surface, is checked when the spawner is installed rather than the target, because a template cannot know at its own install which future policy will name it. That install-time check is fail-fast feedback, not the authoritative gate, because the trust store is mutable and a resigned entry could otherwise slip an ineligible target past a stale pass. Two things close the gap at spawn time: the requester's compiled policy pins the signature of every template it names, and the daemon verifies the resolved template against that pin and re-runs the eligibility check on it regardless, fail-closed on either. The pin carries the ordinary supply-chain cost, that a spawn target cannot be patched in place without every spawner that names it failing closed until recompiled, which is the deliberate trade of byte-exact integrity over convenience. The grant that opens all of this is loud, declared by the operator and surfaced in the risk report as the delegated-spawning exposure it is (T3.9).

14. The service mesh

A single kennel is complete on its own. It holds the capabilities its policy grants, reaches nothing else, and two kennels are mutually invisible by default. A mesh is the deliberate exception: a workload that uses a capability another confined kennel provides, with neither kennel holding the other's authority. A desktop application reaches a display server it does not run; an application reaches a cache without seeing the process behind it. The lateral model holds throughout, provider and consumer siblings under the daemon and neither nested in the other, and the only thing that crosses between them is the one connector the operator declared and the daemon brokered.

One declaration carries the whole surface. A provider lists each capability it offers under `[[provides]]`, by a public name, a typed shape, and the endpoint where it exposes the capability in its own view; a consumer lists each it reaches under `[[consumes]]`, by the same name and shape and the at where the broker presents it for the workload to act against.

```
# in the provider's policy
[[provides]]
name = "org.projectkennel.dbus-broker"
shape = "binder-connector"
endpoint = "/dev/binderfs-mesh/binder"
reason = "push per-session ACCEPT_SESSION to the broker's control node on
↳ the mesh bus"

# in a consumer's policy
[[consumes]]
name = "org.projectkennel.dbus"
shape = "dbus-name"
required = true
reason = "reach the standing D-Bus broker's mediation over the connector
↳ mesh"
```

Neither side names the other. A consumer names the capability the way a client names a service, never the process behind it, and nothing in either declaration

points at another kennel, so there is no cross-kennel reference anywhere in the policy surface to resolve at compile. The shape names the transport, and the surface carries three: `af-unix`, `dbus-name`, and `binder-connector`. The `af-unix` shape the broker delivers directly and the display service rides it; the other two ride the mesh bus, taken up where the mesh is (§14.6). Everything before that holds for all three, because declaration, catalogue, matching, and enablement are shape-agnostic and only the final handoff is not.

14.1 Compile-local, resolve at runtime

A cross-kennel match cannot be settled when a policy is compiled, and the design does not pretend otherwise. Policies are compiled and signed independently, so the compiler only ever holds the one policy in front of it, and the providers that can answer a consume are whatever the operator has installed by the time the workload reaches for it, not whatever existed when the consumer was signed. A signed artefact cannot be made to depend on the presence of another mutable, independently authored one, and resolving a consume against the known kennels at compile has no known kennels to resolve against. Matching is therefore a runtime property, and putting it anywhere else would be a guarantee the system could not keep.

So the two checks live in two places. At compile, the validation is local: each declaration is well-formed, the shape is one of the defined transports, a reserved name traces to a template signed by a key authorised for it (§14.4), and no two `[[provides]]` in the one policy claim the same name. None of these consults another kennel, so all of them hold on a signed artefact for its whole life. At runtime, the broker resolves a consumer's name against the catalogue, requires the optional key to match, enforces the expected shape, and applies the deny-by-default identity check. A consume marked `required = true`, the default, has its name checked for resolvability at construction, and a kennel whose hard dependency nothing provides fails to start rather than run missing something it needs (`refuse-to-start`); a `required = false` consume is skipped when unavailable and the workload tolerates the absence, the `systemd Requires=` and `Wants=` distinction evaluated against the installed set. Tooling may report which of a deployment's consumes currently resolve, but that is an operator diagnostic over a set that changes the moment a service is added or removed, explicitly advisory and never a signature gate.

14.2 Deny-by-default and the broker

Authorisation is the consumer's signed `[[consumes]]` and nothing else. A kennel reaches a capability if and only if its own operator-signed policy declares a con-

14.3 The catalogue is the filesystem

sume for it, and a kennel with no such declaration reaches nothing. The provider is passive: it offered the capability, and the operator decides who uses it by what it signs into each consumer's policy, never by an allowlist the provider maintains. This keeps the provider decoupled from its consumer set, the display service needing no knowledge of which applications render through it, and it is the same shape as the rest of the system, where a capability is granted on the kennel that uses it exactly as egress is granted on the consuming kennel and not as an allowlist on the destination.

When a workload first acts against its at, the broker runs a fixed sequence, each step a lookup or an equality rather than a fresh policy evaluation. It checks the request against the consumer's kernel-stamped identity and confirms the signed policy declares this consume, so the workload can request only what its policy already grants and cannot widen it (`request-dont-author`). It resolves the name against the catalogue to a single provider. It requires the optional key to match exactly, where the key is a discriminator both sides set to the same opaque value, so a keyed consumer is never brokered to a keyless provider and the public name a different kennel could also advertise does not by itself land a consumer at the wrong one. It enforces the expected shape, refusing a mismatched transport rather than misdelivering. Then it activates the provider if it is cold, bridges the connection, and steps out of the byte path. A failure at any step, no installed provider, a shape that disagrees, a key that does not match, is a denial and an audit record, never a silent fallback to some other provider.

Two properties of the name keep the broker honest. A reserved `org.projectkennel.*` name is claimable only by a template signed with the project maintainer key, so a workload cannot advertise the display service's name and have a consumer brokered to an impostor, the provider-name spoof the catalogue would otherwise carry. And a public name offered by more than one enabled provider is never collapsed to no winner: collapsing would let one provider revoke a name another serves merely by also claiming it, a denial of service by name-claim, so a second claimant adds a candidate and can never empty the name, the broker selecting among candidates by key and then by the per-user-over-per-host cascade. Through all of it the daemon brokers the connector and parses none of what rides it, which is what keeps it from becoming the relay its centrality would otherwise make it (`control-not-data-plane`, T5.3).

14.3 The catalogue is the filesystem

The catalogue the broker resolves against is not standing daemon state. It is a derived projection of the signed `[[provides]]` blocks of the providers the operator has enabled, and enabled means linked: the operator turns on what a vendor provides by placing a symlink to its signed policy into an enablement directory, eager

in `autorun/` or `lazy in ondemand/`. The split between what the signature covers and what the link covers is deliberate. The signed provider policy carries the capability and the supervision discipline, what the service is and how it tolerates its own crashes, which is the author's to vouch for; whether it autostarts, and whether eagerly or lazily, is the operator's deployment posture, expressed solely by which directory holds the link and kept out of the signed artefact. A vendor cannot bake autostart-on-every-host into a signed policy, and an operator cannot alter a service's restart discipline without re-signing it. Each side owns exactly its half.

Because the enabled set is the links on disk, `kennel daemon-reload` re-scans the enablement directories, re-derives the catalogue, brings newly linked eager providers online, and makes newly linked lazy providers resolvable without starting them; removing a link and reloading drops the capability and stops an eager provider. The filesystem is the registry, re-read on every reload and on daemon restart, so the daemon holds no enabled-set state a restart could lose or a bug could desynchronise.

The lazy posture is what makes a capability reachable before its provider runs. A provider linked into `ondemand/` is socket-activated on the first consume, the daemon doing for a provider `kennel` what `systemd`'s socket activation does for the daemon itself, while an eager provider is simply already up. This is why delivery is a socket the workload connects to and not a connected descriptor handed in at the workload's own construction: an inherited descriptor would have to exist before any consumer reached for it, which forces the provider up eagerly and defeats the lazy model, whereas a socket standing in the consumer's view from construction costs nothing while idle and makes the connect itself the trigger that resolves and activates the provider. A connect against a cold provider therefore blocks until the provider is ready or a deadline fires, a broker constant defaulting to five seconds, and a provider still pending does not satisfy a waiter. That single rule makes the flat, boot-order-free model safe under a dependency cycle: two services that consume each other, both enabled eager, each block on the other's readiness, both hit the deadline, and both resolve to a loud, observable double-timeout visible in the topology surface rather than a silent deadlock. A brokered connector is a live handle and not a standing guarantee, so when a provider restarts or dies its consumers observe an end-of-file and re-request the capability, which re-resolves and activates afresh; the mesh guarantees a connection to the named capability on request, never the same connection for the kennel's life.

14.4 Supervision and the service-kennel trust class

A provider the operator enables is a kennel the daemon keeps running on the operator's behalf, so it carries the one thing an ephemeral spawn does not, a supervision discipline declared in a `[service]` block and signed into the policy.

The block names a restart policy, a backoff that doubles on each successive attempt so an immediately crashing provider does not spin the supervisor, and a bound on attempts within the crash-loop window. An enabled provider is in one of three readiness states the catalogue projects and the topology surface reads: declared-but-pending, enabled and known but not yet constructed; declared-and-ready, reachable, a consumer's connect bridging straight through; and declared-but-failed, where construction or supervision gave up. A failed provider stays in the catalogue, so its failure is visible and a consume against it is denied and distinguishable from no-such-capability, and it is terminal until the operator intervenes, a failure made sticky and observable rather than self-clearing.

Who may claim a name is the load-bearing gate, because the name is the thing a consumer trusts: a kennel that resolves `org.projectkennel.wayland` is trusting whatever the mesh brokers it under that name to be the display service. The `org.projectkennel.*` namespace is the project's own, built in and claimable only by a maintainer-signed template, the same unit of trust a spawn target uses; an unreserved name is freely authored and signed by any valid key. A maintainer-signed reserved template is not frozen whole, exposing the bits a host legitimately varies as the same fenced mutable fields a spawn target declares, so a host runs a different inner compositor without re-authoring the reserved name or escaping its trust. A host may additionally reserve its own namespaces in the root-owned deployment config, purely additive and never able to redefine the project's.

The service-kennel trust class carries one deliberate widening of the single-leg discipline. The rule that an agent-composable spawn target holds at most one leg binds what an untrusted agent composes, because holding two at once would let it reconstitute across one kennel a capability the operator never granted whole. A service kennel is not that: it is a maintainer-signed template the operator deliberately enabled, vouched for by both the signature and the enablement, so it may hold several legs, the GUI service the worked member of the class. The exemption widens how many legs a service kennel holds, never what kind. It may vend capabilities a consumer uses, render, transport, reach a destination, but never capabilities whose purpose is to be trusted, a secrets broker or a signing service, which would place a trust root inside the boundary the project exists to confine (*authentication-never-attestation*). Trust material a kennel needs arrives as a signed construction parameter from the operator, never brokered from a peer at runtime. The consume grant is loud, each declaration carrying a reason surfaced in the risk report, and the standing shape leaves two narrowed residuals the catalogue records: a provider is a shared target whose compromise reaches every consumer it serves for its lifetime, and the brokering edge is a standing addition to the daemon's always-present surface, bounded by the parse-nothing discipline rather than removed (T3.10).

14.5 The af-unix shape

The delivered shape is `af-unix`, and its handoff is the same `fd-broker` every other capability rides. On a successful consume the broker attaches a connected descriptor to the provider, the byte path the daemon then steps out of. Where that descriptor connects is a host-owned rendezvous point: the daemon derives a per-capability directory under its runtime root from the signed catalogue triple of tier, name, and optional key, binds that directory into the provider's view at the parent of its declared endpoint, and connects to the host-side inode the provider binds its socket on, a bind mount being one inode set seen from two namespaces. This is byte-for-byte the host-socket facade that brokers an ordinary `[[unix.allow]]` socket, pointed at a provider's endpoint instead of a host path, and it binds a provider's own capability directory alone rather than a shared parent, so a provider sees only its own rendezvous socket and every cross-kernel reach stays brokered and deny-by-default. The confined display service is the first non-trivial consumer of all of this, its host-compositor leg riding this shape.

14.6 The mesh bus

A capability that crosses between two kennels needs a binder instance both can reach, and a kennel's own bus is private to it, so the mesh shapes ride a different one. The mesh bus is a single binder instance distinct from every per-kennel bus, with `kenneld` on its node `o`, on which a provider registers a node by `ADD_SERVICE` and a consumer receives a handle to that node by `SVC_CONNECT`. The per-kennel bus carries a kennel's calls to the daemon; the mesh bus carries one kennel's reach to another, and the daemon holds node `o` of both.

The instance exists where an unprivileged daemon cannot ordinarily put it. Mounting a `binderfs` needs the capabilities a user namespace confers, which `kenneld` holds only inside one it creates, so the mesh `binderfs` is mounted by a child the daemon forks for the purpose. The child stays under the daemon's own AppArmor profile, which carries the user-namespace grant across a fork and not across an `exec`, so the fork is load-bearing where an `exec` would lose the grant. It maps itself to the daemon's uid in a single-uid namespace, unprivileged and with no subordinate uids, mounts the `binderfs`, and the nodes it creates are owned by the daemon's own uid. `kenneld` serves node `o` by opening the device through the holder's `/proc/<holder>/root`, the same reach by which it holds node `o` of every per-kennel bus, pointed at the holder instead of an `init`. No privileged helper takes part: the mount the privilege model refuses the daemon directly is the one it makes for itself inside a namespace it is sovereign over.

Exposing the device into a consumer's view is a second problem, because the mount lives in the holder's mount namespace and a kennel's construction has a

private pid namespace in which `/proc/<holder>/root` does not resolve. A path cannot cross that gap, so a mount does. The holder, the one namespace that has the mount, clones it detached and movable with `open_tree`, and `kenneld` relays the descriptor to the kernel's init over a datagram the init binds at a fixed point in its view. The init moves the clone into place at `/dev/binderfs-mesh` and adds the device to the workload's filesystem ruleset before it forks the workload, so the broker and facade that use the bus open it by an ordinary path and know nothing of how it arrived. The clone shares the original's superblock, so a transaction on it crosses the user-namespace boundary intact, the property the whole arrangement rests on and the one most worth having measured rather than reasoned.

14.7 The brokered shapes

On the mesh bus the two declarable shapes deliver as their grammar promised. A `binder-connector` consume returns the provider's node handle, which the consumer reaches by name with whatever rides it opaque to the daemon. A `dbus-name` consume mints no socket: on the per-kernel bus the daemon answers the consumer's locator with the mesh device's path, and the consumer's existing D-Bus facade opens the mesh bus and reaches its broker there, so a brokered name widens an allow-set the facade already had rather than opening a new route.

The broker that name reaches is itself a kernel. A standing service kernel runs `dbus-broker`, holding one control node on the D-Bus mesh bus that only `kenneld` addresses, and the single verb on it is `ACCEPT_SESSION`: the daemon tells the broker to admit one session under a named filter, the broker mints a fresh per-session node, stores the filter against that node's cookie, and returns the node. `kenneld` forwards it to the consumer, which then speaks to the broker directly on its own session node, send and receive and close, with the daemon out of the byte path as everywhere else. When the consumer drops the session node its release tells the broker to reclaim the session. The daemon relays no frame; it authorises a session and steps aside.

What the broker enforces against is the consumer's identity, and the daemon resolves that without holding a table. On each connect it reads the kernel-attested pid of the caller, resolves that pid's cgroup to the kernel context the cgroup names, and looks up that context's one settled D-Bus filter. The cgroup is the restart-invariant name of the kernel, so a facade that restarts under a fresh pid resolves to the same identity, and the only standing state is the policy held against the context and dropped when the context is released. There is no session cookie to forge and no per-connection identity to leave stale.

The mesh bus is where the cross-kernel relay residual concentrates, and delivering the shapes does not dissolve it. A legitimately granted edge is a channel like any

other, so a provider a consumer is entitled to reach can answer across that reach, and a compromised provider can use the edge it was granted (T5.2). The grant is the consumer's to make and the daemon brokers rather than relays, which holds the residual to edges an operator signed, but a signed edge is real authority once given. The declaration surface, the catalogue, the matching, the supervision, and the enablement treat all three shapes alike, so the mesh shapes are the af-unix shape's siblings and not a second grammar (make-invalid-unrepresentable); what differs is only the transport the daemon brokers, never the way a consumer asks.

15. The interposed surface

The mesh of the previous chapter brokers a connector between two kennels and steps out. That works when the capability is a connection: a socket, a name, a node, something whose whole substance is the bytes that flow once it is wired. Some capabilities are not like that. A display protocol is a stateful conversation a client holds with a server that owns screens, input devices, and every other client on the machine, and mediating it the way the D-Bus facade mediates a bus, message by message against a filter, would put a parser of an adversarial display protocol in the trusted path, the one thing the project refuses everywhere else. The answer for a surface too rich to broker per message is to interpose a whole confined instance of the service between the workload and the real thing: a server constructed for the workload, holding the one downstream connection to the host, so the workload talks to something the kennel owns and never to the host at all. The pattern is *interpose-as-transaction* raised from a single descriptor to an entire service, and the confined GUI is its one worked instance today.

15.1 The display server is constructed, not granted

The host compositor is a shared resource, and handing a kennel a connection to it grants whatever that compositor exposes to its clients. What a compositor exposes varies: screen-capture globals, virtual input, layer-shell, the set of other connected clients. Some compositors gate these from a tagged sandboxed client through a staging protocol; most, the mainstream desktop default among them, do not. A design resting on the host compositor to deny privileged globals would be enforced on one machine and unenforced on the next, with no way to tell the difference from inside the kennel. That is the X11 problem in a milder form, confinement the protocol's server must opt into, which the framework does not own, and X11 itself admits no mediation that confines a client and stays out of scope.

So the display server is not borrowed from the host; it is constructed. The workload is given a display server that exists only for it, whose every global is one the framework placed there, and the host compositor is reduced to a single ordinary downstream client relationship held by a different kennel and never the workload's. This

is the absence path applied to the display (construction-by-absence): rather than enumerate what a graphical workload must be denied on the host compositor and hope the denial holds across deployments, present it a positively constructed compositor containing only what one confined application should see.

```
[[consumes]]
name = "desktop.display"
shape = "af-unix"
at = "wayland-1"
required = true
reason = "the workload's only display is the one the GUI-service kennel
↳ constructs"
```

In the workload's own policy the constructed display is an ordinary consume, an af-unix capability named like any other. The workload names a display and reaches whatever the catalogue resolves it to, here a GUI-service kennel's nested compositor, and it holds no leg to the host compositor anywhere in its policy.

15.2 The nested compositor and the brokered leg

The GUI-service kennel runs an upstream Wayland compositor inside the confinement, cage by default for its single-application kiosk shape, with Weston or sway where a kennel should present several windows, and it spawns one fresh instance per app connection. The confined app connects to that compositor, and its registry enumerates the inner compositor's globals and nothing else; the host's screen-copy, input-synthesis, and layer-shell globals and the host's other clients sit on a socket the app's view does not contain, absent rather than denied. There is no allow-list of host globals to maintain and no Wayland parser anywhere in the trusted path, because the inner compositor is unmodified upstream software and the isolation is a property of which compositor the app can reach, not of filtering the bytes it exchanges with one. None of the three compositors is patched; the GUI-service kennel is an ordinary confined kennel whose workload happens to be a compositor, its choice declared in the kennel's policy.

A nested compositor must itself reach some compositor to put pixels on the screen, and the GUI-service kennel holds that one connection. The inner compositor reaches the host through the same AF_UNIX brokered connect the host-socket facade performs, opening a Wayland socket presented at a path in its own view and letting the daemon broker the connect to the host compositor, relaying the connection's bytes and the SCM_RIGHTS descriptors it carries, the keymap and shm pools and dmabuf buffers a byte-only relay would silently drop. The path the compositor opens names the facade, not the host: the host compositor's own socket pathname is absent from the inner compositor's

view and from the app's beneath it, so the app cannot name the host socket, reach the host's other display clients, or open a second connection of its own (`make-invalid-unrepresentable`). The broker relays bytes and descriptors and parses none of the display wire (`control-not-data-plane`). Both legs are brokered `AF_UNIX`, the same `org.projectkennel.wayland` mesh shape of the previous chapter, and the split is eager against lazy: the inner-compositor-to-host leg is eager, brokered the moment the compositor starts because the host compositor is always up, while the app-to-inner leg is the mesh consume, socket-activated, the app's first connect to the Wayland socket at its own endpoint spinning up the dedicated inner compositor that did not exist until it reached for one.

15.3 The invariant, and where the enforcer runs

The load-bearing property is a standing invariant: confined GUI depends on no enforcement by the host compositor. The host sees one ordinary Wayland client, the GUI-service kennel's leg, and is asked to enforce nothing; every confinement property of the display is carried by a compositor the framework ships and controls, inside a kennel. Bring-your-own inner compositor is the right shape unconditionally, not a position to revisit when the mainstream desktop gains the staging protocol that lets a host compositor deny privileged globals to a tagged client. Where the inner compositor implements that protocol it is optional defense-in-depth, a second redundant denial, never the floor, because depending on the host compositor at all ties the guarantee to deployment-specific software outside the operator's policy. The enforcer must sit inside the boundary, and the nested compositor puts it there.

Which is why the inner compositor runs in the GUI-service kennel and never the app's. A workload must not be able to reach or subvert the component that confines its display, and placing the enforcer inside the thing it confines is the tamperproofing failure a reference monitor exists to avoid, so the model is two kennels rather than a compositor injected into the app's own view. The inner compositor's own interface is permissive by design, cage and its alternatives exposing screen-copy and virtual-input globals because a compositor's clients legitimately screenshot and drive the surfaces it owns, and that permissiveness is bounded by the nesting rather than asserted away. The capture and input globals operate on the inner compositor's own scene graph, which holds only the kennel's surfaces, and have no path to the host's scene because the inner compositor is itself a mere client of the host compositor; it can capture what it composites and inject into the windows it owns, both the kennel's, and a client cannot reach across its own connection to drive the server's other clients. A capability confined to one trust domain and exercised within it is that domain's own software operating on its own surfaces, not a leak (T2.7).

15.4 Isolation, the host leg, and the trust class

One inner compositor per app connection makes cross-kennel GUI invisibility structural. Two graphical kennels have two disjoint display servers with no shared global, surface, or input path, the same lateral isolation as their disjoint loopback subnets and socket views, and the isolation runs finer than the trust boundary requires: two apps even within one kennel do not share a display server unless they share a connection. That costs nothing, a nested compositor being cheap and living exactly as long as its connection, so the broker spawns one per connection and reaps it when the connection drops, the supervised set tracking the live connection population rather than a static declaration, closer to what the daemon does for a spawn than to a fixed sidecar. The clipboard and drag-and-drop follow the same construction: the kennel's clipboard is its inner compositor's, disjoint from the host's, so a workload neither reads the user's clipboard nor writes one the user will paste elsewhere, isolated because the compositor is the kennel's own rather than because a host compositor's clipboard policy was trusted to hold (T2.6). A consented, direction-aware bridge is a deliberate later grant, never the default.

The reach the model does not eliminate it concentrates and bounds. Exactly one party reaches the host compositor, the GUI-service kennel, and it reaches it only to vend connected descriptors to the inner compositors it spawns, and even there it is one ordinary Wayland client contained by the host compositor's own client isolation the way any application is. No workload kennel holds the host leg and no unconfined host process holds standing display capability. This is the display analogue of the host-reconnaissance residual: a single deliberate reasoned host-reach held by a confined kennel, declared with a required reason and surfaced in the risk report as an exposure that reads as reasoned rather than as a boundary quietly bending (T1.12, the T1.6 shape). An app kennel consumes `org.projectkennel.wayland` and the GUI-service kennel provides it, a reserved name only an operator-signed service kennel may claim, so a workload cannot advertise the display service's name and have a consumer's display brokered to an impostor; the consumer's grant is the coarse one, `may-reach-the-GUI-service`, never a per-global predicate that would drag a Wayland parser into the broker. The GUI-service kennel is the worked member of the service-kennel trust class whose multi-leg exemption the previous chapter defined: an operator-signed, deliberately enabled service may hold more legs than the single one an agent-composable spawn target is held to, because the operator vouched for the whole of it.

15.5 The surfaces ahead

The pattern generalises past the display, and two further surfaces are where it goes next, each declared here as the shape it will take rather than as something the runtime carries today.

The first is interactive file access. A graphical application that can touch only its pre-granted paths holds half a capability, because the defining desktop interaction is the user choosing a file at runtime that the application had no prior grant to. That choice is the fd-broker again under the user's live consent: the user picks a file and the application receives a descriptor to exactly that file, having held no filesystem and gained no path it can enumerate. The chooser is the host's own file dialog, a trusted process the user already relies on, so consent happens outside the confinement on the host the user trusts, and the picker is transient, summoned per request and gone when the choice is made, holding no standing capability. A confined GUI without it is a window that cannot open a document, so it is part of the display capability rather than an adjunct, and it is owed work rather than landed: the broker that drives a host picker and places the chosen descriptor into the workload's view is not yet in the tree.

The second is the relocation of the desktop's D-Bus broker. Notifications, open-a-URL, and the other desktop-service calls ride the D-Bus facade of the local-services chapter, which mediates the bus per message, and that facade's host-side half today runs in the operator's own context. Moving it into a confined nested instance is the interposed-surface pattern applied to the bus: the broker becomes a service constructed for the workloads that consume it rather than a delegate in the operator's context, interposed as an instance the framework owns. That relocation reshapes where the local-services boundary sits, so the two chapters reconcile when it lands rather than now, and until it does the facade stands as the local-services chapter describes it. Each of these is the same move the display made, a capability owned as a constructed instance and a broker rather than a foreign substrate shaped to fit, which is the thesis the confined GUI arrived at by eliminating every option that borrowed enforcement from outside the boundary: own the trusted path, depend on nothing foreign in it, and prefer a constructed instance to a filtered one.

PART IV

The language and the tools

16. The policy compiler

The chapters before this one described what the runtime enforces. What the operator writes is not that. A written policy names a template to extend, pulls in shared fragments, carries deltas that add and remove grants, and leaves variables for the runtime to fill, none of which the enforcement path should ever see. A naive system would resolve all of it on every spawn: parse the leaf, walk the inheritance chain, verify each template's and fragment's signature, check the lockfile, merge the includes, apply the deltas, validate the invariants, substitute the variables, and only then have something to enforce. That is a great deal of complex, fallible, security-critical code, and running it on the hot path of every `kernel run` is the wrong place for it. Kernel does not work that way. It compiles.

16.1 The authored artefact and the enforced artefact

The systems Kernel is measured against already draw this line. AppArmor authors a text profile and a parser compiles it into a binary policy loaded into the kernel, with the text never consulted at enforcement; SELinux compiles a monolithic binary policy from source modules. The authored artefact and the enforced artefact are different things, and the compile step is where the expensive and fallible work happens, once and deliberately, rather than on every enforcement. `kernel compile` takes a leaf policy and produces a settled policy: a flat, fully resolved document in which the inheritance chain has been folded, the includes merged, the deltas applied, and every source signature and lockfile pin verified. The settled policy carries no `template_base`, no `include`, and no delta operators, only the final effective rules, signed as a unit by the compiling authority. The division of labour is the point. The complex code runs at compile time, where the operator can review the output before anything runs; the spawn path links none of the template machinery and consumes an artefact whose shape is fixed and simple.

16.2 The fold

Resolution is a fold down a chain. The compiler walks a leaf's `template_base` to the root template, the minimal base-confined that every confined template extends,

and folds the chain root-first into one effective policy with no base left to resolve. Composition follows the model an SSH client uses for its configuration. A scalar takes the most-derived value that sets it and inherits where a level is silent. A list field is set by the most derived bare assignment, a child's list replacing the inherited one rather than appending to it, which is the behaviour every in-tree template relies on; the additive and subtractive operators a leaf author writes as explicit add and remove deltas are the later increment on top of that set-or-inherit base. Object sub-tables merge shallowly, field by field, the child overriding. One list field breaks the replace rule on purpose: the invariant denies are unioned down the chain and never replaced, so a child can only add to them, because their non-removability is exactly what makes them invariants. Inheritance is single-parent with no diamonds, and a user's own policy is a leaf that extends one template and is not itself extensible, which keeps developers from accumulating unmaintained team templates that drift from the reviewed set. Where unrelated templates must share a cross-cutting fragment, a corporate egress allowlist or a mandated audit block, that fragment is pulled in as a signed, version-pinned, additive-only include rather than forced into a contrived hierarchy. The folded result is then translated into the runtime's effective form, the flat representation the enforcement path reads.

16.3 The signature is the commitment

Integrity runs on ed25519 signatures end to end, and the compiler's handling of them is the security-critical half of its work. Every source template and fragment is signature-verified as the chain resolves, against the trust store the operator controls, so a chain that pulls in an artefact no trusted key signed fails to compile rather than folding an unsigned grant into the result. What a reference resolved to is then pinned. The lockfile records, for every reference, the signing-key identity and the artefact's signature. A reference with no prior pin is trust-on-first-use, accepted and recorded so the pin is established the first time it resolves, and a later compile that resolves the same `name@version` to a different signature is a hard error rather than a warning. A deployment that will not extend that first-use trust compiles under a require-signed posture, where an unsigned ancestor or include is refused outright rather than pinned. The signature is the pin. A kennel signature is an SSH-SIG, the armored OpenSSH detached signature `ssh-keygen` produces and verifies, and the pin commits to the canonical bytes through a SHA-512 of them carried in the signature's fixed preimage rather than over the bytes directly. Re-tagging a version to different bytes changes their hash and so the signature, and re-signing the same name under another trusted key changes the signature too, so both are caught. The hash lives inside the signature object rather than as a second commitment carried beside it, so the lockfile records one signature per reference as before. The settled policy the compiler emits is itself signed as an SSHSIG over its canonical body by the compiling authority, produced through `ssh-keygen -Y sign so`

that a key in a file, an agent, or a hardware token are alike to the compiler, one signature standing for the whole resolved result. The variables that can be fixed at compile, the installation constants a deployment knows when it builds the artefact, are substituted here and baked in; the ones intrinsically per-spawn are left for the runtime, recorded so it knows exactly which it owns.

16.4 Lint and the footgun warnings

Two kinds of judgement run over a policy, and they sit at different stages. Compile surfaces footgun warnings on the source, the loud grants and unenforced sections an author should see before signing. The linter runs later and judges the settled policy for internal incoherence, the combinations that contradict themselves once the fold has run: a grant the resolved network mode renders vacuous, a listener inherited under a mode that cannot use it. It exists because the fold can pull in a field the leaf author never wrote and never sees, the case where a leaf sets one network mode but inherits a proxy listener from its base and the two disagree, which no reading of the leaf alone would catch. Where `policy show` describes a settled artefact, `policy lint` judges it, and a non-empty result is a non-zero exit; the shipped template corpus is required to lint clean, asserted by a test, so the baseline an operator builds on is coherent before their own deltas land on it.

```
[[unix.allow]]
name = "ssh-agent"
real = "/run/user/1000/ssh-agent.sock"
shim = "~/.ssh/agent.sock"
reason = "the workload signs its own git pushes"

# compile warns, and does not refuse:
# [[unix.allow]] `ssh-agent` shims an SSH agent (name = "ssh-agent" /
# env = "SSH_AUTH_SOCK"): an exposed agent is a destination-blind signing
# oracle. Shim a raw agent only if you accept that any code in the kernel
# can sign for any destination.
```

The grant compiles, because the operator may have meant it, and it warns, because they may not have. A footgun is a grant the framework will carry but an author should see in full before they sign over it, so the warning names the grant, the threat it opens, and the narrower section that exists for the safe version of the same need.

16.5 The compiler is out of the trusted base

All of this, the TOML parsing of arbitrary templates, the chain walk, the glob handling, the include conflict resolution, the signature verification, is the most com-

plex and most fallible code in the policy path, and none of it runs in the daemon. The compiler is a separate crate the command-line tool links and the daemon does not, some three and a half thousand lines of authoring front end held behind a hard crate boundary, and a dependency query against the daemon shows none of it present. The runtime keeps only the verify-and-load half: check one signature over the settled policy, re-assert the framework invariants, fill the per-spawn variables, build the kernel objects, and spawn. The fallible resolver is quarantined where its failures are review-time feedback rather than a fault in the trusted base (*quarantine-the-unsafe*), and the daemon's reachable surface shrinks by exactly the code that does not need to be there.

That boundary is what makes the two operating modes possible without weakening either. A developer iterating on a policy does not type the compile step: `kennel run` of a source policy compiles it in memory when no fresh settled artefact exists, signs the result and records the lockfile pins, marks it a development build, and runs it, recompiling when it detects a changed input against the recorded provenance, so the edit-run loop stays tight. An organisation instead compiles centrally, on infrastructure that holds the templates, the fragments, the lockfile, and the signing key, and pushes only the signed settled policies to workstations that need none of the resolution machinery; there the runtime trust surface is one signature check against a pinned key. The authored artefact stays where the complexity is reviewed, and the enforced artefact arrives simple, fixed, and signed, which is the shape the next chapter takes up.

17. The settled artefact

What the compiler emits is the thing the runtime trusts, and it is shaped to make that trust cheap. A settled policy is flat. The inheritance chain has been folded, the includes merged, the deltas applied, the source signatures verified, and what remains carries no `template_base`, no `include`, and no delta operator, only the final effective rules. The constructs the compiler resolved are not simplified in the settled form, they are absent from it: there is no chain for the runtime to walk and no delta for it to apply, because the settled document has no place to express either (`make-invalid-unrepresentable`). The body the runtime reads is a small fixed set of fields, the kennel's name, the placeholders left for it to fill, the framework invariants the compiler asserted, the resolved effective policy, and a provenance block, signed as a unit.

17.1 The canonical body and its signature

A signature is only as good as the agreement between signer and verifier on what bytes it covers, and the settled policy fixes those bytes as its canonical form. The artefact is a TOML document, and its signature is an SSHSIG, the same object the SSH toolchain signs and verifies. The SSHSIG is computed not over the body directly but over a small fixed preimage that carries a SHA-512 of it: the namespace the signature is bound to and the hash of a deterministic serialisation of the body in field order, with the signature envelope that sits beside it in the document excluded from that serialisation. Signer and verifier derive the canonical bytes the same way and hash them the same way, so they agree on the covered content to the byte, and that single SHA-512 is the one place the body's bytes enter the signature, which makes the determinism of the canonical form the thing both ends must reproduce exactly. There is no sorted-key or normalised-number canonicalisation of the kind a JSON scheme would reach for, because both ends derive the same serialisation from the same struct and the schema carries no floats to normalise. The form keeps its one deliberate property, that a sub-table which is empty is omitted from the canonical bytes entirely, so a policy granting no SSH and no host sockets hashes to exactly the bytes it would have before those tables existed in the schema. New optional surface can be added without invalidating the signature of every pol-

icy that does not use it; the omission that keeps a signature stable across schema growth now keeps the hash under it stable, and the SSHSIG with it.

```
settled_schema_version = 2
compiler_version = "0.5.0"

# the canonical body: the resolved sections the runtime enforces
[fs]
# ...
[exec]
# ...

[signature]          # excluded from the bytes it covers
algorithm = "sshsig"
signature = "-----BEGIN SSH SIGNATURE-----\n...\n-----END SSH
↳ SIGNATURE-----\n"
```

The header fixes the schema version the runtime must understand and the compiler that produced the body, and the body is the resolved policy with every inheritance already folded in. The [signature] table sits in the same document but outside the bytes it signs, so a verifier strips it, derives the canonical form of everything else, and checks the SSHSIG against that.

17.2 What the runtime does with it

The runtime's whole interaction with a settled policy is short, and that brevity is the dividend the compiler paid for. It deserialises the document, verifies the one SSHSIG against the trust store in-process, de-armouring it, requiring its namespace and its embedded key to be the kernel-policy namespace and a trusted key, then recomputing the canonical hash and rebuilding the preimage for a single ed25519 check, re-asserts the framework invariants, substitutes the placeholders that could only be filled at spawn, and builds the kernel objects. None of the template machinery is linked on this path: there is no chain to walk, no fragment to fetch, no delta to apply, no source signature to chase, and no ssh-keygen to exec, because all of that produced the artefact already and left its result flat. The placeholders it fills are the ones intrinsically per-spawn, the kernel's runtime context, the invoking user, the generated name and home, recorded in the artefact as a deferred list so the runtime knows exactly which it owns. They are deferred because they cannot be otherwise: a settled artefact is distributed to a fleet unchanged, the same signed bytes on every machine, so a per-user or per-instance value has no single answer at compile and can only be filled where the kernel starts. It substitutes those and no others, and a settled policy that still carries an unsubstituted

17.3 A signature is not a safety proof

placeholder the deferred list does not name is refused rather than spawned, because such a placeholder is a compiler defect and enforcing a policy with a hole in it is worse than refusing one (`refuse-to-start`).

17.3 A signature is not a safety proof

A valid signature on a settled policy means a key the operator trusts vouched for the artefact. It does not mean the artefact upholds the structural guarantees that define what a kernel is, because those guarantees are the framework's and not the signer's, and a trusted key can sign a policy that quietly drops one. So the runtime re-asserts the framework invariants against the effective rules as a final gate, regardless of the signature: a climb-back bar that must be set, the `setuid` and `setgid` denials, the mandatory home shim, the cloud-metadata deny that no egress may remove, the process-isolation namespace, the proxy-only egress. These are a handful of cheap structural checks, and they hold whoever signed the artefact and however it arrived, by compile on the machine or by push from a fleet controller. A validly signed settled policy that violates one is refused at spawn. The list of invariants the compiler asserted travels in the artefact, but it travels as an audit record of what compile checked, never as a substitute for the runtime check; the runtime re-runs them regardless of what that field claims. This is the one place the runtime deliberately repeats the compiler's work, and it earns its keep, because it means no policy, whatever key signed it and whatever path it took, can disable the protections that make a kernel a kernel.

17.4 Provenance and attestation

The artefact describes itself. A provenance block records every input that produced it: the compiler version, the schema version, the threat-catalogue version the templates were authored against, and each resolved template and fragment by name, version, signing key, and signature. Because the signature commits to those bytes through their hash, established in the previous chapter, that last record is exact: a reader can see precisely which signed source artefacts, at which versions and which bytes, were composed into the result, and can do so without any of those sources present, reading only the settled file. Two settled policies can be diffed directly, and their provenance compared revision to revision, without resolving anything.

That self-description is also what makes the artefact attestable. Because the enforced policy is the signed policy, with no live resolution between the signature and the enforcement that could let the two diverge, the artefact's signature is its identity. An organisation that compiles centrally and pushes only signed settled policies can have a workstation demonstrate that it runs an approved revision, be-

cause the bytes it enforces are the bytes the organisation signed and the signature proves it. The runtime trust surface on that workstation is one signature check against a pinned key, the resolution machinery and the templates and the lockfile all left behind on the compile infrastructure, and what arrives is a flat, signed, self-describing artefact whose identity is exactly the policy it enforces. The keys that make that signature mean something are what the next chapter takes up.

18. The keys

A signature is only as good as the key behind it, and the previous two chapters leaned on signatures without saying where the keys live or what trusting one means. Every signed artefact in the system carries the same kind of signature: a source template, an included fragment, and a settled policy each bind their content to a key identity with a signature, and that identity resolves against a trust store the operator owns. What follows is the store, how trust moves through it, and the bound on what a signature can be asked to prove.

18.1 The key and its identity

A key is an ed25519 keypair, and a key identity is at once a filename and a signature-envelope record. The identity is the stem of a `.pub` file in the trust store, the private key sits beside it, and the signature on an artefact records which identity signed it; the verifier looks that identity up and checks the signature against the matching public key. The identity is a short filesystem-safe string, legible in `policy show` output and unambiguous as a filename on any target.

The on-disk form is the OpenSSH one. A public key is the familiar `ssh-ed25519` line, the base64 wire blob with the key identity carried in its trailing comment, so the keys the trust store holds are the keys `ssh-keygen` prints and lists and the identity shows up in an ordinary key listing; a private key is the unencrypted OPENSSH PRIVATE KEY envelope, with an encrypted key refused and the operator pointed at `ssh-keygen -p` to strip the passphrase rather than have the framework grow a passphrase-prompt path.

```
# ~/.config/kennel/keys/alice-laptop-2026.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI... alice-laptop-2026
```

```
[signature]
algorithm = "sshsig"
```

```
key_id = "alice-laptop-2026"
signature = "-----BEGIN SSH SIGNATURE-----\n...\n-----END SSH
↳ SIGNATURE-----\n"
```

The .pub file's stem is the identity, `alice-laptop-2026`, and the same string is the `key_id` the signature records, so verification is a lookup: read the identity off the envelope, find the .pub whose name matches, check the SSHSIG against the key it holds. The parser for this is small, the unencrypted ed25519 layout being fixed-width with no ASN.1 and no variable-length structure to speak of. The adoption of the OpenSSH format does not retire what came before it: a key is fundamentally its thirty-two public bytes, the signature scheme is ed25519 either way, and a key recorded in the prior base64 representation remains valid in the store, decoding to the same key the OpenSSH line would. The format is a more legible and more interoperable envelope over an unchanged key, not a new key system, so existing keys and the tooling around them keep working.

The keys are SSH keys, and so are the signatures over what they vouch for. A signature on an artefact is an SSHSIG, the same detached signature `ssh-keygen -Y sign` produces and `ssh-keygen -Y verify` checks, so an operator who trusts none of the framework's own code can verify a settled policy with the SSH toolchain they already have. Signing reaches the private key the way any SSH client does, so a key in a file, a key in an agent, or a key on a hardware token are alike to the framework, which writes no agent client of its own and asks the operator's existing one for a signature. Each SSHSIG is bound to a fixed namespace that marks it a kennel-policy signature, which is what keeps the convenience honest once the signing key is an ordinary SSH key: a signature made to authenticate a session or vouch for a commit carries a different namespace and cannot be lifted into a policy context, so a key may be reused across roles without the roles blurring. A signer held on a hardware token is recognised and declined for now, its signature being one the framework cannot reconstruct in process, with out-of-process verification for that case owed.

18.2 The three tiers

The store is three flat directories, searched as one. A vendor tier under `/usr/lib/kennel/keys` carries the project's own keys and is the authority for the reserved `org.projectkennel.*` namespace; a host tier under `/etc/kennel/keys` carries an administrator's fleet keys; a user tier under `~/.config/kennel/keys` carries a user's own. None of the tiers is a single key. Each is a directory of .pub files, any number may coexist in a layer, and every key in a layer is equally able to verify, so an organisation runs a host key per team or per pipeline and a user holds several of their own without any of them being privileged over the

18.3 Rotation and revocation

others. A signature verifies if its named identity matches any `.pub` in any searched directory.

<code>/usr/lib/kennel/keys/ projectkennel-2026.pub</code>	vendor tier, authority for <code>org.projectkennel.*</code>
<code>/etc/kennel/keys/ ops-fleet-2026.pub ci-pipeline-2026.pub</code>	host tier, the fleet's signing keys
<code>~/.config/kennel/keys/ alice-laptop-2026.pub</code>	user tier, the operator's own

Each tier is a directory, each holds any number of `.pub` files, and every key in a searched tier verifies equally, so a fleet runs one key per team or pipeline and a user keeps several of their own.

The tiers are not searched identically, and the difference is the security baseline. When the compiler resolves a template's inheritance chain, it verifies each ancestor against the system tiers alone, the vendor and host directories and never the user one, so a user template that extends `base-confined` narrows within the invariants that ancestor re-asserts but cannot weaken them, because the key that signed the ancestor it builds on had to be a host or vendor key. Settled-policy verification spans all three tiers, vendor first, then host, then user, and an earlier directory wins a clash of identity, so a vendor or host key is unshadowable by a user key that happens to share its name. The asymmetry is the point: a user may author and sign their own leaf freely, but the floor they build on is anchored in keys only the administrator and the package manager can place.

18.3 Rotation and revocation

Rotation is additive and lazy. A new key is placed in the store beside the old one and both verify at once; there is no expiry clock, no revocation list, and no on-line ceremony (`do-less`). New artefacts are signed with the new key, artefacts already signed with the old one stay valid until they are next recompiled, and the old key is removed once nothing anyone cares about still references it. This is the `authorized_keys` model exactly: add the new, retire the old on your own schedule. Because the store is re-read on every construction request rather than frozen when the daemon started, a key added or removed after the daemon came up is honoured on the next request without a restart.

Revocation is the same act of removal, and its reach is deliberately bounded to construction. Deleting a `.pub` from the store revokes that identity: the next run or construction request refuses any artefact it signed (`refuse-to-start`). There is no mechanism to reach into a kernel already running and kill it because its signing

key was just revoked, and that absence is by design rather than an omission. A running kennel is already confined by the policy it was constructed with, the daemon verified that policy once at construction and the running kennel holds the resulting trusted structure in memory rather than a live reference to the key, so revoking the key changes what may be built next and changes nothing about what is already running. Stopping a running kennel is a separate and direct act, `kennel kill`, not a side effect of pulling a key.

18.4 The honesty bound

The trust boundary under all of this is the filesystem, and naming what each tier rests on is what keeps the guarantee honest. The vendor directory is protected by the package manager and root, the host directory by root ownership, the user directory by the user's own ownership, and each tier's integrity is exactly the integrity of whoever owns its directory: a vendor key vouches that the reference templates are the ones the project maintainer signed and only a package update changes them, a host key vouches that the fleet artefacts were placed by the administrator and only root adds a new host signer, and a user key vouches that the user's own policies were compiled by the user. That last tier is wide on purpose, since any process the user runs can sign with a user key, and it is not a weakness, because the threat model confines the workloads the user runs and not the user's own tools. There is no promotion across the line: a user key cannot sign something that verifies as a system key, is never consulted when an ancestor is checked, and a user who needs fleet trust has the administrator install their public key into the host tier rather than elevating it themselves.

What a signature proves is narrow, and the narrowness is the honest part. The key proves who compiled an artefact, never what the policy inside it does. The grants are readable prose a reviewer reads for themselves; the signature only proves they were not altered between the compile that produced them and the construction that enforces them. The key confers no authority of its own and is checked, not exercised: the daemon verifies the signature and then trusts the in-memory structure, and the identity behind the key attests provenance and grants nothing. This is the same line the settled artefact drew when its signature was shown to be no proof of safety, seen now from the side of the key: the runtime re-asserts the framework invariants precisely because a trusted signer is not trusted to have upheld them, and the key's whole job ends at saying who vouched for the bytes.

19. The audit account

A reference monitor is asked for three things. It must mediate every access, it must resist tampering, and it must be small enough to verify and honest enough to account for what it did. The volume so far has been the first two made mechanism: every crossing driven through one gateway, the enforcer always outside the thing it confines, privilege held in the narrowest place and never in the daemon a workload can reach. What remains is the third, and it is two accounts rather than one. Before a kennel runs, a report says what its policy exposes. While it runs, a log records what it did. Together they are how the monitor answers for what it mediates.

19.1 The risk report

`kennel policy risks` evaluates a fully folded policy against the threat catalogue and reports what the grants expose and what they mitigate. Each finding names a threat by its catalogue identity, carries the catalogue's one-line residual for it, names the granting site that carries it, and quotes the reason the operator wrote there. So an exposure is never a bare label: it is a threat, the exact grant that opens it, the residual that remains after the framework's mitigation, and the operator's own stated cause for accepting it, read off the policy as a single legible line.

```
$ kennel policy risks
exposes
  T1.6 host-network reconnaissance, the residual after host mode
      at [net] mode = "host"
      why "the build needs the corporate proxy on a host-only VLAN"
mitigates
  T3.8 arbitrary egress, closed by the proxy default-deny
```

An exposure reads as the threat, the residual the catalogue records for it, the exact grant that opens it, and the reason the author wrote at that grant, so the report answers not only what a policy risks but why its author accepted the risk.

A tag reaches the report two ways, and the report keeps them apart. Most are authored, written on a grant by whoever wrote the template or the leaf. Some are derived: the compiler infers them from the shape of a grant, so a policy that opens host network mode has the host-reconnaissance residual reinstated against it whether or not the author tagged it (T1.6), a device passthrough carries the control-deactivation exposure (T2.1), a headless signing allowance reinstates the lateral-reach residual (T1.6). The derived set is what makes the report match what is enforced rather than only what someone remembered to write, and marking each finding authored or derived is what keeps an inference the compiler drew from being mistaken for a claim the author made.

The honesty of the report is structural rather than asserted. The threat tags live only in the source policy and never survive into the settled artefact, so the report reads the same folded source the compiler enforced from, which is the honest input and not a separately maintained story about it. A tag that names no catalogued threat is surfaced as a likely typo rather than silently dropped or counted as a real risk, and a policy authored against an older catalogue version earns a drift note. There is no invented risk and no theatre: the report says what the grants carry, what the compiler can infer, and what it could not resolve, and nothing beyond that. This is the warning surface the rest of the volume kept pointing at (`footgun-warn-dont-forbid`). Host network mode's reinstated residual, the delegated-spawning exposure, the substrate trust of a pulled image, the standing-service consume: each was called a loud grant surfaced in the risk report, and this is the report, the place a deliberate exposure is shown to the operator as the reasoned choice it was rather than forbidden outright or buried.

19.2 The audit log

While a kennel runs, the monitor records what it mediated in a structured per-kennel log. The record is tunable by resource class, the network, the filesystem, execution, the local-socket surface, and the bus each carrying their own setting, and by level within a class, from off through denies-only and a per-class summary to the full record of every mediated access. An operator logs every filesystem access and only denials on the network, or the reverse, or everything, by the deployment's need rather than a single global switch. The log is written to one of a few sinks, per-class files of newline-delimited JSON by default, or the system journal, or syslog, or the daemon's own output for a container that collects it from there.

```
{ "schema_version": 1, "ts": "2026-06-29T12:37:33.829451Z", "kennel": "build-ag_
↪ ent",
  "kennel_uuid": "7f3c...", "event": "net.connect", "resource": "net", "outcome":
↪ "deny",
```

19.3 What the account is for, and where it stops

```
"source": "kenneld", "host": "remco-vm", "pid": 58471, "dest": "exfil.example:4  
↪ 43"}}
```

Every record carries the same envelope, the schema version, a timestamp, the kernel name and its run identity, the event and resource class, the outcome, and the daemon as source, with the resource-specific fields beside it. A denial names what was refused and a mediation what was allowed, so the log is the account of what the monitor did, not what the policy said it might.

One definition governs all of this, shared between the compiler that reads a policy's audit section and the daemon that reads a standalone defaults file, and it lives in the runtime crate rather than the compiler. That placement is the same discipline that kept the resolver out of the daemon, applied once more: the daemon must resolve audit configuration at spawn time, so the schema it needs sits where it can reach it without linking the compiler the daemon is built to exclude. The format carries a schema version and is stable across releases, so a tool written to parse the log can rely on its structure holding.

19.3 What the account is for, and where it stops

The two accounts carry the uses that justify keeping them. A log of every resource access answers a forensic question after the fact, what a workload read between one time and another. It is evidence for a compliance claim, because a kernel that did not reach a protected resource did not reach it by the kernel's refusal and the log holds the refusals. It feeds policy refinement, a denied access the workload legitimately needed becoming the next grant the operator adds with a reason. And it is the input to anomaly detection, a monitoring layer watching for new destinations or unusual access or repeated denials. Every one of those is someone else's to build on top: the framework provides the record, never the watcher over it.

That boundary is the honest shape of the whole account. The log is write-once and append-only from the framework's side, and stronger tamper-evidence, signed segments or an external transparency log, is a downstream concern the format leaves room for without claiming to provide. The compliance story is bounded the same way: the framework claims no regulatory certification and provides the infrastructure that makes a certification-relevant claim demonstrable, the evidence rather than the verdict. The risk report says what a policy exposes and the log says what a kernel did; whether either is acceptable is a judgement with standing the framework does not assume for itself. It accounts; it does not absolve.

This is where the volume ends, because it is where the reference monitor's third demand is met. The account does not strengthen the confinement, which holds whether or not a log is ever read and whether or not a risk report is ever run. It

makes the confinement answerable. What a policy exposed can be said before it runs, what a kennel did can be read after, and both can be checked against the design the first volume set out, in a form a person or a tool can verify rather than take on faith. A monitor that mediates completely and resists tampering but cannot be inspected still asks to be trusted. One that also accounts for what it mediated, in the open and in its own grant's words, asks for less, and that is the whole of what this design has tried to do: hold the line, and show its working.

PART V

Policy reference

20. The policy language

Generated from `schema/policy.toml.schema`. Current as of 0.5.0.

[audit]

[audit]: sink selection, per-class levels, and per-sink tuning

```
[audit]
# optional
sinks    = ["..."]
```

Contains: [audit.dbus], [audit.exec], [audit.file], [audit.filesystem], [audit.journald], [audit.network], [audit.stdout], [audit.syslog], [audit.unix].

sinks optional. Active sinks (file, journald, syslog, stdout). Default ["file"].

[audit.dbus]

[audit.dbus] level.

```
[audit.dbus]
# optional
level       = "off" | "denies-only" | "summary" | "full"
```

level optional. One of off, denies-only, summary, full. One of off, denies-only, summary, full.

[cap]

[audit.file]

[audit.file] tuning.

```
[audit.file]
# optional
compress_after_seconds = 0
dir = "...
retain_count = 0
rotate_at_bytes = "...
```

`compress_after_seconds` optional. Gzip a rotated file this many seconds after rotation.

`dir` optional. Override the per-kernel directory (placeholders allowed).

`retain_count` optional. Keep at most this many rotated files per class.

`rotate_at_bytes` optional. Rotate at this size (e.g. "64M", "1G"; bare = bytes).

[audit.journald]

[audit.journald] — no fields; present to allow the empty table.

[audit.syslog]

[audit.syslog] tuning.

```
[audit.syslog]
# optional
facility = "...
```

`facility` optional. Syslog facility (user, daemon, auth, ...). Default user.

[cap]

[cap]: capabilities and `no_new_privs`.

```
[cap]
# optional
bounding_set = ["..."]
no_new_privs = true
```

`bounding_set` optional. The capability bounding set to retain (empty drops them all).

`no_new_privs` optional. `PR_SET_NO_NEW_PRIVS`. A framework invariant once resolved (must be true).

Example, from `toml/templates/base-bwrap`:

```
[cap]
no_new_privs = true
bounding_set = []
```

`[[consumes]]`

One `[[consumes]]` entry, a capability this kernel reaches over the mesh.

```
[[consumes]]
# required
name      = "..."/>

```

`at` optional. Where the brokered connector is delivered, in this kernel's own view.

`env` optional. Environment variable(s) synthesised into this kernel to name the connector.

`key` optional. An optional private match token; must match the provider's.

`name` required. The capability's public identifier, resolved against the catalogue at runtime.

`reason` decoration. Why this capability is consumed (required).

`required` optional. Whether the capability's absence fails kernel construction.
Hard dependency by

`shape` required. The transport it expects; the broker refuses a mismatched shape.
One of `af-unix`, `dbus-name`, `binder-connector`.

Example, from `toml/templates/gui-session`:

[dbus]

```
[[consumes]]
name = "org.projectkennel.wayland"
shape = "af-unix"
at = "/tmp/wayland-0"
required = true
reason = "the confined desktop session reaches its display server - a
↳ broker-spawned nested compositor - over the mesh"
```

[dbus]

[dbus]: D-Bus mediation.

Contains: [dbus.audit], [dbus.session], [dbus.system].

[dbus.audit]

[dbus.audit]: per-kernel D-Bus call audit verbosity.

```
[dbus.audit]
# optional
level      = "off" | "summary" | "full"
```

level optional. Verbosity ("off", "summary", "full"). One of off, summary, full.

[dbus.session]

[dbus.session]: the user session bus (the common case: notifications, portals).

```
[dbus.session]
# optional
enabled = true
```

Contains: [dbus.session.allow], [dbus.session.deny].

enabled optional. Whether this bus is reachable at all. Absent/false $\Rightarrow$ no connection to this bus.

```
[dbus.session.allow]
```

[dbus.<bus>.allow]: what the kernel may reach (an allowlist; default-deny).

```
[dbus.session.allow]
# optional
broadcast = ["..."]
call      = ["..."]
own       = ["..."]
talk      = ["..."]
```

broadcast optional. Signals the kernel may receive (a subset of senders it may talk to).

call optional. Finer than **talk**: specific `destination=interface.member` calls.

own optional. Names the kernel may own (be addressable as). Almost always empty.

talk optional. Destinations the kernel may call methods on and receive replies/signals from

```
[env]
```

[env]: environment curation.

```
[env]
# optional
deny      = ["..."]
pass      = ["..."]
set       = "..."
```

deny optional. Variables denied even if passed (globs allowed).

pass optional. Variables passed through from the caller's environment (globs allowed).

set optional. Variables forced to a specific value. Declared last: as a TOML table it must

Example, from `toml/templates/base-bwrap`:

```
[env]
set = { TMPDIR = "/tmp", NO_PROXY = "" }
```

[exec]

[exec]

[exec]: what may be `execve`'d.

```
[exec]
# optional
allow    = ["..."]
deny     = ["..."]
deny_setcap = true
deny_setgid = true
deny_setuid = true
deny_writable = true
path     = ["..."]
shell    = "..."
```

`allow` optional. Allowlisted binary paths (the `execve` allowlist). Execution is deny-by-default:

`deny` optional. Denylisted absolute paths or globs.

`deny_setcap` optional. Refuse file-capability binaries (framework invariant).

`deny_setgid` optional. Refuse setgid binaries (framework invariant).

`deny_setuid` optional. Refuse setuid binaries at `execve` (framework invariant once resolved).

`deny_writable` optional. Refuse execution of files in writable paths (framework invariant).

`path` optional. `PATH` search roots the resolver records for the workload's environment.

`shell` optional. The kernel's login shell: the synthetic-passwd `pw_shell` and

Example, from `toml/templates/argv-tool`:

```
[exec]
allow = ["/bin/echo", "/usr/bin/echo", "/bin/cat", "/usr/bin/cat",
↪  "/bin/true", "/usr/bin/true", "/bin/sh"]
```

[fs]

[fs] and its sub-tables.

```
[fs]
# optional
```

```
deny      = ["..."]
exclusive = ["..."]
read      = ["..."]
write     = ["..."]
```

Contains: [fs.dev], [fs.home], [fs.proc], [fs.tmp].

deny optional. Categorical denies (belt-and-braces over the constructed view). Replace or increment

exclusive optional. Writable paths bound **exclusively** (T2.8): while the kernel runs, kenneld

read optional. Paths granted read (and directory traversal / execute). Replace (read = ["..."]) or

write optional. Paths granted write. Replace or increment at the same key.

Example, from toml/templates/argv-tool:

```
[fs]
read = ["/usr/**", "/bin/**", "/lib/**", "/lib64/**"]
```

[fs.dev]

[fs.dev]: the minimal /dev.

```
[fs.dev]
# optional
allow    = ["..."]
```

Contains: [[fs.dev.passthrough]].

allow optional. The trivial pseudo-device baseline bound into the kernel's /dev (/dev/null,

[[fs.dev.passthrough]] entries (or { add, remove } increment), optional:

group optional. The owning group that gates access (e.g. dialout, modem, dip). Access is

path optional. The device node, an absolute path under /dev (e.g. /dev/ttyUSB0,

[fs]

reason decoration. Why this device is exposed (required).

threats decoration. Threat tags, required to carry an exposed tag (passthrough widens the

Example, from toml/templates/base-bwrap:

```
[fs.dev]
allow = [
    "/dev/null", "/dev/zero", "/dev/random", "/dev/urandom",
    "/dev/tty", "/dev/pts/**",
]
```

[fs.home]

[fs.home]: the constructed \$HOME view.

```
[fs.home]
# optional
persist = ["..."]
readonly = true
shadow = true
```

persist optional. Home-relative paths that **persist** across runs. By default the
readonly optional. Make the constructed \$HOME **read-only** (default: writable). The
home root

shadow optional. Whether \$HOME is shadowed by a constructed view (must be true
once resolved).

Example, from toml/templates/base-bwrap:

```
[fs.home]
shadow = true
```

```
[fs.proc]
```

```
[fs.proc]: procfs hidepid.
```

```
[fs.proc]
# optional
hidepid = true
```

hidepid optional. Mount /proc with hidepid=2.

Example, from toml/templates/base-bwrap:

```
[fs.proc]
hidepid = true
```

```
[fs.tmp]
```

```
[fs.tmp]: the private /tmp tmpfs.
```

```
[fs.tmp]
# optional
size      = "..."
writable = true
```

size optional. Size cap in human form ("512M", "1G").

writable optional. Whether the workload may **write** to its /tmp tmpfs (the Land-lock write grant). Absent ⇒

Example, from toml/templates/base-bwrap:

```
[fs.tmp]
writable = true
size = "512M"
```

[identity]

[identity]

[identity]: the workload's identity inside the kennel.

```
[identity]
# optional
group      = "..."/>

```

group optional. The workload's masked **primary** group name (synthetic /etc/passwd pw_gid
groups optional. Supplementary group names to retain (e.g. ["dialout", "plugdev"]). The user
user optional. The workload's masked user name, \$USER/\$LOGNAME and the synthetic

[lifecycle]

[lifecycle]: TTL and TTL action. ttl is the human form ("8h"); the

```
[lifecycle]
# optional
ttl          = "..."/>

```

ttl optional. Time-to-live in human form ("8h", "1h", "30m").
ttl_action optional. What to do at TTL expiry: "exit" (alias "stop", the default) ends the One of exit, warn, renew.

Example, from toml/templates/argv-tool:

```
[lifecycle]
ttl = "5m"/>

```

```
[[mutable]]
```

One `[[mutable]]` manifest entry: a leaf field a spawn of this template may write.

```
[[mutable]]
# optional
field      = "... "
freeform   = true
from       = ["... "]
match      = ["... "]
max        = 0
oneof      = ["... "]
type       = "... "
under      = "... "
# decoration
reason     = "... "
```

`field` optional. The dotted leaf-field path this entry opens (`net.proxy.allow`, `rootfs.writable`, `fs.write`).

`freeform` optional. Freeform bound: no shape at all, the loud last-resort footgun. Any value is accepted; a

`from` optional. Pool bound: the fixed set a spawn may append values from.

`match` optional. Pattern bound: the pre-baked net-destination shapes an open value must match

`max` optional. Pool bound: the maximum number of appended entries.

`oneof` optional. Oneof bound: the enumerated member list a spawn selects from.

`reason` decoration. The justification a freeform variant requires (the loud rule).

`type` optional. Predicate bound: the value type (currently `relpath`).

`under` optional. Predicate bound: the root the value resolves under (`RESOLVE_IN_ROOT`, `traversal-free`).

Example, from `toml/templates/argv-tool`:

```
[[mutable]]
field = "workload.argv"
freeform = true
reason = "the caller chooses the command to run within the [exec].allow
↳ floor; the cage contains it"
```

[net]

[net]

[net] and its sub-tables.

```
[net]
# optional
mode      = "none" | "constrained" | "unconstrained" | "host"
proxy_listen_v4_address = "..."
proxy_listen_v6_address = "..."
# decoration
reason    = "..."
```

Contains: [net.audit], [net.bind], [net.bpf], [net.ipv6], [net.proxy].

mode optional. Egress mode: "none" (own empty net-ns, no interfaces), "constrained" (own net-ns, One of none, constrained, unconstrained, host.

proxy_listen_v4_address optional. IPv4 proxy listen address as "offset:port" within the kernel's subnet. A

proxy_listen_v6_address optional. IPv6 proxy listen address as "offset:port".

reason decoration. Required (non-empty) only when mode = "host": the documented justification for

Example, from toml/policies/interactive:

```
[net]
mode = "host"
reason = "an interactive human shell needs direct, unmediated egress
↳ (git/ssh/curl) on the host network stack"
```

[net.audit]

[net.audit]: per-kernel egress audit log.

```
[net.audit]
# optional
level      = "summary" | "full"
log_path   = "..."
```

level optional. Audit verbosity ("summary", "full"). One of summary, full.

`log_path` optional. Where the per-kernel egress JSONL log is written.

Example, from `toml/templates/base-confined`:

```
[net.audit]
log_path = "~/.local/state/kennel/<kennel>/network.jsonl"
level = "summary"
```

`[net.bind]`

`[net.bind]`: bind-address rewriting policy (the wildcard-rewrite knobs; the bind

```
[net.bind]
# optional
allow_host_loopback_v4 = true
allow_host_loopback_v6 = true
allowed_ports = ["..."]
in6addr_any_policy = "rewrite" | "deny"
inaddr_any_policy = "rewrite" | "deny"
min_port = 0
```

`allow_host_loopback_v4` optional. Whether binding the host IPv4 loopback is permitted.

`allow_host_loopback_v6` optional. Whether binding the host IPv6 loopback is permitted.

`allowed_ports` optional. Explicit allowlist of bindable ports. When non-empty, the workload may

`in6addr_any_policy` optional. What to do with a wildcard IPv6 bind. One of `rewrite`, `deny`.

`inaddr_any_policy` optional. What to do with a wildcard IPv4 bind (`"rewrite"` / `"deny"`). One of `rewrite`, `deny`.

`min_port` optional. Lowest bindable port.

Example, from `toml/templates/base-confined`:

```
[net.bind]
inaddr_any_policy = "rewrite"
in6addr_any_policy = "rewrite"
allow_host_loopback_v4 = false
allow_host_loopback_v6 = false
min_port = 1024
```

[net]

[net.bpf]

[net.bpf]: the kernel/syscall ACL (the cgroup connect4/6 + bind4/6 BPF and the

```
[net.bpf]
# optional
deny_families = ["..."]
families = ["..."]
```

Contains: [net.bpf.bind], [net.bpf.connect].

deny_families optional. Denied socket families (inet_sock_create returns EPERM): AF_NETLINK, AF_PACKET, ...

families optional. Permitted socket families (defence in depth; e.g. ["AF_INET", "AF_INET6", "AF_UNIX"]).

[net.bpf.bind]

[net.bpf.bind]: the inbound BIND ACL (cidr + ports, deny-first).

Contains: [[net.bpf.bind.allow]], [[net.bpf.bind.deny]].

[[net.bpf.bind.allow]] entries (or { add, remove } increment), optional:

cidr optional. The CIDR ("10.0.0.0/8", a bare address, or "*" = 0.0.0.0/0 + ::/0).

ports optional. Permitted ports (empty = any port).

protocol optional. Transport protocol ("tcp", "udp", "any"). One of any, tcp, udp.

reason decoration. Why this rule exists (required).

threats decoration. Threat tags.

[[net.bpf.bind.deny]] entries (or { add, remove } increment), optional:

cidr optional. The CIDR ("10.0.0.0/8", a bare address, or "*" = 0.0.0.0/0 + ::/0).

ports optional. Permitted ports (empty = any port).

protocol optional. Transport protocol ("tcp", "udp", "any"). One of any, tcp, udp.

reason decoration. Why this rule exists (required).

threats decoration. Threat tags.

```
[net.ipv6]
```

[net.ipv6]: IPv6-specific options.

```
[net.ipv6]
# optional
force_v6only = true
```

`force_v6only` optional. Force IPV6_V6ONLY=1 so a dual-stack socket cannot escape the v4 rewrite.

Example, from `toml/templates/base-confined`:

```
[net.ipv6]
force_v6only = true
```

```
[net.proxy]
```

[net.proxy]: the user-space egress policy the per-kernel proxy enforces

Contains: [net.proxy.deny], [[net.proxy.allow]].

[[net.proxy.allow]] entries (or { add, remove } increment), optional:

`cidr` optional. A CIDR destination, when the rule is by-address rather than by-name.

`name` optional. The destination host (or dot-prefixed suffix). Mutually informative with `cidr`.

`ports` optional. Permitted ports.

`protocol` optional. Transport protocol ("tcp", "udp", "any"). One of any, tcp, udp.

`reason` decoration. Why this destination is permitted (required).

`threats` decoration. Threat tags.

`tls` optional. `tls.required` and friends.

```
[net.proxy.deny]
```

[net.proxy.deny]: the deny table: the non-removable invariant floor and the

Contains: [[net.proxy.deny.invariant]], [[net.proxy.deny.policy]].

[[net.proxy.deny.invariant]] entries, optional:

[[provides]]

cidr required. The denied CIDR (e.g. "169.254.169.254/32").
reason decoration. Why the deny exists (required).
threats decoration. Threat tags.

[[net.proxy.deny.policy]] entries (or { add, remove } increment), optional:

cidr required. The denied CIDR (e.g. "169.254.169.254/32").
reason decoration. Why the deny exists (required).
threats decoration. Threat tags.

[[provides]]

One [[provides]] entry, a capability this kennel offers over the mesh.

```
[[provides]]
# required
name      = "..."
shape     = "af-unix" | "dbus-name" | "binder-connector"
# optional
endpoint  = "..."
key       = "..."
# decoration
reason    = "..."
```

endpoint optional. Where the capability is exposed, in the provider's own view.

Optional: an omitted

key optional. An optional private match token, never advertised in the catalogue.

name required. The capability's public identifier, what the catalogue advertises. A reserved

reason decoration. Why this capability is offered (required).

shape required. The typed transport. One of af-unix, dbus-name, binder-connector.

Example, from toml/templates/dbus-broker:

```
[[provides]]
name = "org.projectkennel.dbus-broker"
shape = "binder-connector"
endpoint = "/dev/binderfs-mesh/binder"
reason = "kenneld pushes per-consumer ACCEPT_SESSION (bus + method filter)
↳ to the broker's control node on the mesh"
```

[rootfs]

[rootfs]: an OCI image unpacked as the kernel's root filesystem.

```
[rootfs]
# optional
image    = "...
path     = "...
persistence = "discard" | "persist"
readonly = ["..."]
writable = ["..."]
# decoration
reason   = "..."
```

image optional. The image@sha256:... the build pulled from; the runner refuses unless it equals the

path optional. The unpacked image rootfs (the store entry's rootfs/).

persistence optional. Rootfs persistence: "discard" (default) | "persist". "persist" is a One of discard, persist.

readonly optional. Closure-lock: rootfs paths Landlock denies writes to, the executable-closure

reason decoration. Why this substrate is trusted (required; the substrate-trust waiver is loud).

writable optional. Closure-lock holes: rootfs paths kept writable, carved back out of readonly

[seccomp]

[seccomp]: the seccomp filter (source carries a deny list; the resolver

```
[seccomp]
# optional
allow    = ["..."]
deny     = ["..."]
profile  = "..."
```

allow optional. Syscalls explicitly allowed.

deny optional. Syscalls denied on top of the profile.

profile optional. The baseline profile name ("default").

Example, from toml/templates/base-bwrap:

```
[service]
```

```
[seccomp]
profile = "default"
deny = [
    "userfaultfd", "perf_event_open", "bpf",
    "process_vm_readv", "process_vm_writev",
    "kexec_load", "kexec_file_load",
    "mount", "umount", "umount2", "pivot_root",
    "swapon", "swapoff", "reboot",
    "init_module", "finit_module", "delete_module",
    "personality",
]
```

```
[service]
```

[service]: the supervision discipline for a service kennel.

```
[service]
# optional
backoff = "..."
max_attempts = 0
restart = "always" | "on-failure" | "never"
```

backoff optional. Initial delay before a restart in human form ("500ms", "2s", default "500ms"); doubles

max_attempts optional. Restarts within the crash-loop window before declared-but-failed (default 5; must be ≥ 1).

restart optional. Restart discipline: always / on-failure (default) / never. One of always, on-failure, never.

Example, from toml/templates/dbus-broker:

```
[service]
restart = "on-failure"
```

```
[signature]
```

The [signature] envelope carried by a signed artefact.

```
[signature]
# required
algorithm = "..."
key_id    = "..."
signature = "..."
# optional
signed_fields = ["..."]
```

`algorithm` required. Signature algorithm. Must be "sshsig" ([SSH_SIG_ALGORITHM]).

`key_id` required. Identifies the signing key in the trust store. The SSHSIG also embeds the

`signature` required. The armored SSHSIG (-----BEGIN SSH SIGNATURE----- ...)

over the canonical

`signed_fields` optional. The top-level fields the signature covers (every field except

Example, from `toml/policies/interactive`:

```
[signature]
algorithm = "sshsig"
key_id = "kennel-maint-2026"
signature = "-----BEGIN SSH SIGNATURE-----\nU1NIU01HAAAAQAAADMAAAALc3NoL
↳ WVkMjU1MTkAAAAGKWVn2EUqe+ju7quWQv7aY/ihwM\npBFaipL7vU2UVH85IAAAAbcG9s
↳ aWN5LnYxQHByb2plY3RrZW5uZWwub3JnAAAAAAAAAAZz\naGE1MTIAAABTAAAC3NzaC1
↳ LZDI1NTE5AAAAQ04+Qm6EihZIVrHHJHc5lYaTh22kJbwk+8\nYVUo1VJ+bBxwFC1WisLK
↳ 1FSZ4fXR916/2Lo8NXGOLCGg+pvS0cygQ=\n-----END SSH
↳ SIGNATURE-----\n"
```

[spawn]

[spawn]: the delegated-instantiation grant.

```
[spawn]
# optional
max_instances = 0
# decoration
reason = "..."
```

Contains: `[[spawn.allow]]`.

[ssh]

`max_instances` optional. Concurrent-instance ceiling across this grant's spawns, the fork-bomb bound.

`reason` decoration. Why this delegation is extended (required; the spawn waiver is loud, validated at compile).

[[spawn.allow]] entries, optional:

`mutable` optional. Optional per-requester narrowing; the subset of the template's [[mutable]] manifest fields

`template` optional. The trust-store template name (`net-fetch`).

[ssh]

[ssh]: per-kernel SSH egress (source-only).

```
[ssh]
# optional
allow_headless = true
```

Contains: [[ssh.destinations]].

`allow_headless` optional. Whether a granted key may be driven by a non-interactive (CI) kernel with no

[[ssh.destinations]] entries (or { add, remove } increment), optional:

`dest` optional. The SSH destination, in the form the host-side `ssh` is invoked with
`options` optional. Host-side `ssh` invocation options for this destination, passed verbatim as argv

`reason` decoration. Why this destination is granted (required).

`threats` decoration. Threat tags.

[trust]

[trust]: the masked workspace manifest (T2.8).

```
[trust]
# optional
manifest = true
on_change = "warn" | "freeze" | "kill"
```

`manifest` optional. Maintain a `.trust-manifest.json` at the root of every writable/persistent
`on_change` optional. What `kenneld` does when a watched trigger is mutated during the run: One of `warn`, `freeze`, `kill`.

[tty]

[tty]: terminal hardening for an interactive (PTY) workload.

```
[tty]
# optional
filter_terminal_escapes = true
```

`filter_terminal_escapes` optional. Filter the dangerous escape sequences a workload could write toward the

[unix]

[unix]: AF_UNIX policy.

```
[unix]
# optional
abstract = "deny" | "allow"
```

Contains: `[[unix.allow]]`.

`abstract` optional. Abstract-namespace socket disposition (`"deny"` / `"allow"`).
 One of `deny`, `allow`.

`[[unix.allow]]` entries (or `{ add, remove }` increment), optional:

`env` optional. An environment variable to set to the shim path (e.g. `SSH_AUTH_SOCKET`).
`name` optional. A logical name (e.g. `"ssh-agent"`) for a per-kernel service instance.
`real` optional. The real host socket path.
`reason` decoration. Why this socket is granted (required).
`shim` optional. The shim path the socket is bound at inside the kernel.
`threats` decoration. Threat tags.

[unsafe]

Example, from toml/templates/dbus-broker:

```
| [unix]
```

[unsafe]

[unsafe]: the advisory footgun umbrella.

Contains: [unsafe.pttrace], [unsafe.signal].

[unsafe.pttrace]

[unsafe.pttrace]: pttrace across the kernel boundary (scoping from PID-ns + sec-comp).

```
| [unsafe.pttrace]
| # optional
| allow_from = ["..."]
| allow_targets = ["..."]
```

allow_from optional. Permitted sources.

allow_targets optional. Permitted targets ("self", ...).

[workload]

[workload]: the command the kernel runs, optionally pinned.

```
| [workload]
| # optional
| argv      = ["..."]
| cwd       = "... "
| pinned    = true
| sha256    = ["..."]
```

argv optional. The command + args (argv[0] is the program). Absent ⇒ supplied at kernel run.

cwd optional. Working directory inside the view (may carry a ~/<home> placeholder).

`pinned` optional. Refuse a CLI `--` override of `argv` unless `--force` (pin exactly what runs).

`sha256` optional. Accepted lowercase-hex SHA-256 digests of the workload binary; the spawn verifies

Example, from `toml/policies/interactive`:

```
| [workload]
| argv = ["/bin/bash"]
```

Common forms

Threat tags: carried inline on any grant that can widen surface, as `threats = { exposed = [...], mitigated = [...] }`.

The increment: any list field also accepts `[[<list>.add]]` / `[[<list>.remove]]`, folded over the inherited list at compile.

21. Worked examples

A single confined thing, followed from the floor to a running shell. Each tier is a real shipped file; the prose is the file's own. Read top to bottom and you watch confinement accrete: the floor denies, the template widens it four different ways, the leaf pins it.

base-confined --- the floor every kennel stands on

Template: base-confined — the floor every kennel stands on. Inherits: nothing — this is the root of every confined template. Threat catalogue: THREATS.md v0.4

The minimal confined posture: `no_new_privs`, `setuid/setgid/setcap` denial, the constructed-`$HOME` view, proxy-only egress, the cloud-metadata + link-local invariant denies (RFC1918 stays reachable — see [net]), a curated environment, and a defence-in-depth `seccomp` filter.

Deny-by-default across the board: `exec.allow` is `EMPTY`, which now denies ALL execution (execution is deny-by-default like `fs` and `net`) — so base-confined cannot run anything and is not usable on its own. A derived template/leaf adds the binaries it needs to `exec.allow`; the compiler then resolves and grants exactly the shared libraries those binaries link (see the [lib] section below). Only enforced sections appear here; sections the runtime does not implement (`dbus/x11/ptrace/signal`) are deliberately omitted — declaring them would warn.

See `docs/design/05-templates.md` and `templates/README.md`.

```
| template_name = "base-confined"
```

Capabilities (framework invariants)

```
| [cap]
| no_new_privs = true           # PR_SET_NO_NEW_PRIVS, always; cannot be set
| ↪ false
| bounding_set = []            # drop the entire capability bounding set
```

Execution (deny-by-default)

`exec.allow` is empty, which DENIES ALL EXECUTION: a merely-readable file is not executable. A derived template/leaf adds the binaries it needs; `permissive-exec` (a `**` entry) is the explicit opt-out that restores the open posture.

There is intentionally NO `exec.deny` list: under deny-by-default it would be moot (`sudo/su/pkexec/...` are not in any allowlist, so they never run; and the `deny_*` flags below + `no_new_privs` neuter `setuid` escalation regardless). A deny list that enforces nothing is theatre, so it is omitted rather than shipped as reassurance.

```
[exec]
allow = []
```

```
deny_setuid = true          # refuse setuid binaries at execve (framework
↪ invariant)
deny_setgid = true          # refuse setgid binaries (framework invariant)
deny_setcap = true          # refuse file-capability binaries
deny_writable = true        # refuse execution of files in writable paths
```

```
path = ["/usr/bin", "/usr/local/bin", "/bin"]
```

Filesystem

The constructed view (`fs.home`, below): `$HOME` is a fresh tmpfs into which only granted paths are bound. Non-granted paths do not exist in the view (T1.1).

```
[fs]
# Read baseline: the curated base every dynamically-linked workload needs
# (§4.2 construction-by-absence). Only these subtrees are bound into the
# view; everything else under /usr (headers, source, /usr/local) is simply
↪ not
# present. The filesystem floor prevents data leakage; the real enforcement
↪ is
# at exec.allow|deny (Landlock FS_EXECUTE). Absent subtrees on this host
↪ (e.g.
# /usr/lib64 on usrmerge distros, /usr/libexec on Debian) are silently
↪ skipped.
read = [
  # The curated /usr base:
  "/usr/bin/**", "/usr/sbin/**",          # binary search path (Landlock
  ↪ gates exec)
```

base-confined --- the floor every kennel stands on

```
"/usr/lib/**",                                # shared libraries, locale,
↳ multiarch
"/usr/lib64/**",                              # 64-bit lib path (Fedora/RHEL;
↳ skipped if absent)
"/usr/libexec/**",                            # helper binaries (git-core etc;
↳ skipped if absent)
"/usr/share/**",                              # arch-independent data (terminfo,
↳ zoneinfo, CAs, locale)
# usrmerge compat:
"/lib/**", "/lib64/**",
# TLS root CAs + linker config (not under /usr):
"/etc/ssl/**", "/etc/pki/**",
"/etc/ld.so.conf", "/etc/ld.so.conf.d/**", "/etc/ld.so.cache",
# update-alternatives symlink farm: many tools (awk+gawk, vi, editor,
↳ pager,
# java, ...) are reached via /usr/bin/<tool> -> /etc/alternatives/<tool>
↳ -> real
# binary. Without /etc/alternatives in the view the symlink dangles and
↳ the tool
# is "command not found". It is non-sensitive (symlinks only), bound
↳ read-only.
"/etc/alternatives/**",
# The libc/NSS files are SYNTHESISED per-kennel (scrubbed of host
↳ specifics),
# not bound from the host:
↳ /etc/{hosts,resolv.conf,nsswitch.conf,services,
# protocols,passwd,group,host.conf}. See kennel::etc and §7.2.5.
"/proc/self/**", "/proc/cpuinfo", "/proc/meminfo", "/proc/version",
"/sys/devices/system/cpu/**",                # processor-count detection
]
```

```
# Write baseline: none. The workload's writable space is the constructed
↳ $HOME and
# the private /tmp (below); a usable template or leaf grants its project
↳ write paths.
write = []
```

```
# There is intentionally NO fs.deny list. The constructed view is
↳ deny-by-default:
# only fs.read/fs.write paths exist in it at all, so the host's ~/.ssh,
↳ ~/.aws,
# /etc/shadow, /dev/mem, ... are simply absent - not present-but-denied. And
↳ a deny
# that fell *inside* a granted directory could not be enforced anyway
↳ (Landlock is
# allow-only and cannot subtract a single path). A long credential denylist
↳ here
```

```
# would be reassurance theatre; the real control is granting narrowly, not
↪ denying
# broadly. (T1.1/T2.1 are defended by the view's absence-by-default, not a
↪ denylist.)
```

```
# The constructed $HOME (T1.1). HOME is /home/<user> (the masked
↪ [identity].user,
# default `kennel`); granted ~/ paths are bound beneath it; the Landlock
↪ ruleset
# backstops on the resolved view ($7.2.3/.5). The home root is writable by
↪ default,
# but it is a fresh tmpfs - anything written directly there is ephemeral.
↪ Persistence
# is opt-in per path via `persist` (or a writable ~/ grant), which binds
↪ the real host
# inode read-write beneath the home. Set `readonly = true` to suppress the
↪ default
# home-write grant (only write-granted ~/ paths stay writable then).
[fs.home]
shadow = true
```

```
# Private /tmp (T1.1): a fresh tmpfs; the host /tmp is invisible.
[fs.tmp]
writable = true
size = "512M"
```

```
# Procfs: PID namespace + hidepid=2 - the workload sees only its own tree
↪ (T1.1, T1.6).
[fs.proc]
hidepid = true
```

```
# Minimal /dev: a constructed tmpfs with only these nodes bound in (T1.1,
↪ T1.6). Each
# granted node is also Landlock read/write/ioctl-able ($8.1).
[fs.dev]
allow = [
    "/dev/null", "/dev/zero", "/dev/random", "/dev/urandom",
    "/dev/tty", "/dev/pts/**",
]
```

base-confined --- the floor every kennel stands on

Libraries

There is no library allowlist, by design. Execution is gated by Landlock FS_EXECUTE, which the kernel enforces only at `execve(2)`: on the allowlisted binary AND its dynamic loader (`PT_INTERP/ld.so`, resolved by the compiler from each `exec.allow` binary). Shared libraries are `mmap'd` by the loader, which Landlock does NOT gate, so they load with `READ` alone (`07-3-exec`) — the `fs.read` grants above (`/usr`, `/lib`, `/lib64`) already make them readable. A library therefore cannot be “execute-gated” under Landlock; the kernel makes no such claim rather than ship an unenforceable filter. (To stop a workload running NEW programs, that is `exec.allow` — `default-deny`; to stop it loading code in-process, the workload must not be an interpreter you handed arbitrary input.)

Network

All egress funnels through a per-kennel `SOCKS5/HTTP` proxy; the cgroup BPF denies direct `connect()` to anything but the proxy (`fail-closed`). The proxy resolves names via the OS resolver and vets the answers against the allowlist + the invariant denies (`rebinding defence`) — there is no configurable `[net.dns]`.

```
[net]
mode = "constrained"
# Both families' proxy listeners are on by default in the proxied modes; a
↪ family
# is enabled iff its address resolves (no separate on/off flag).
# Listener address is computed from the kennel's <tag>/<ctx> ($7.3.2);
↪ override the
# host offset/port within the kennel's own subnet with
↪ proxy_listen_v4_address =
# "offset:port" (offset 1..=14, default "1:1080"). Project Kennel injects
# HTTPS_PROXY / HTTP_PROXY / ALL_PROXY pointing at the computed proxy
↪ address.

# Invariant denies - the destinations that are NEVER a legitimate egress
↪ target,
# enforced deny-first by the proxy (even in `open` mode) and non-removable
↪ by any
# derived template or leaf (T1.6). Deliberately NARROW: cloud metadata (the
↪ SSRF
# crown jewel) and link-local only.
```

```
# RFC1918 / CGNAT are intentionally NOT invariant denies. Making private
↪ space
# permanently unreachable is self-defeating - a kennel routinely needs a
↪ local dev
# server, a LAN database, an internal registry, or a corp service. In
↪ `constrained`
# mode (the default) nothing private is reachable anyway unless a
↪ `[net.proxy.allow]`
# names it; in `open` mode the operator has opted into arbitrary egress.
↪ Either way
# that is the policy author's call, not an immovable floor.
[[net.proxy.deny.invariant]]
cidr = "169.254.169.254/32"
reason = "cloud metadata IPv4 - never permitted from a kennel (mandatory
↪ invariant)"
[[net.proxy.deny.invariant]]
cidr = "fd00:ec2::254/128"
reason = "AWS IPv6 metadata - never permitted"
[[net.proxy.deny.invariant]]
cidr = "fe80::/10"
reason = "IPv6 link-local - no legitimate egress destination"
```

```
# Bind: a wildcard bind (0.0.0.0 / ::) is rewritten to the kennel's private
# loopback address (so dev servers work but are reachable only from inside
↪ the
# kennel's address space); host loopback is not bindable; no privileged
↪ ports.
[net.bind]
inaddr_any_policy = "rewrite"
in6addr_any_policy = "rewrite"
allow_host_loopback_v4 = false
allow_host_loopback_v6 = false
min_port = 1024
```

```
# Force IPV6_V6ONLY=1 so a dual-stack socket cannot escape the v4 rewrite
↪ (T1.6).
[net.ipv6]
force_v6only = true
```

```
# Per-kennel egress audit log (one JSONL record per request; denies at
↪ higher
# level). Written by the proxy; persists across runs.
[net.audit]
log_path = "~/local/state/kennel/<kennel>/network.jsonl"
level = "summary"
```

base-confined --- the floor every kennel stands on

AF_UNIX sockets

Default-deny; the constructed view contains only granted sockets. Abstract-namespace sockets are denied categorically — enforced natively by Landlock scoping (ABI 6, §8.1), not the legacy seccomp/AppArmor fallback. (T1.6.)

```
| [unix]  
| abstract = "deny"
```

Process introspection

The PID namespace hides every process outside the kennel; hidepid backstops it. (Cross-boundary ptrace and signalling are already prevented by the PID/user namespace — there is no separate control the runtime enforces, so none is declared. The advisory [unsafe.pttrace]/[unsafe.signal] sub-sections exist to *express* intent but warn that scoping comes from PID-ns/seccomp; this template needs neither. T1.6.)

Environment

The environment is SYNTHESISED, not filtered. The spawn clears the inherited env entirely (env_clear) and rebuilds it: the framework sets HOME, PATH, USER, LOGNAME, SHELL (and forwards the caller's TERM); set below adds fixed extras. There is no pass/deny here — a passthrough/denylist is fiction when nothing is inherited in the first place (the host's secrets, AWS_, _TOKEN, ... never reach the kennel because the whole env is wiped, not because they were named in a denylist).

```
| [env]  
| set = { TMPDIR = "/tmp", NO_PROXY = "" }
```

Seccomp (defence in depth)

Most enforcement is at higher layers (Landlock for files, cgroup BPF for net); this filter denies a small set of syscalls with no legitimate use here and a history of exploit-chain involvement. Default action: errno (EPERM).

```
| [seccomp]  
| profile = "default"  
| deny = [  
|     "userfaultfd", "perf_event_open", "bpf",  
|     "process_vm_readv", "process_vm_writev",
```

```

    "kexec_load", "kexec_file_load",
    "mount", "umount", "umount2", "pivot_root",
    "swapon", "swapoff", "reboot",
    "init_module", "finit_module", "delete_module",
    "personality",
]

```

Resource limits (off by default)

setrlimit(2) caps, applied in the seal. Nothing is set here; a derived template or leaf opts in. Names are the short rlimit resources (nofile, nproc, as, cpu, data, fsize, core, stack, memlock, ...); values are “soft” or “soft:hard”, each a number (optional K/M/G) or “unlimited”. Example:

```
| [ulimits]
```

```
| nofile = "8192"
```

```
| nproc  = "512"
```

```
| core   = "0"
```

```

| [signature]
| algorithm = "sshsig"
| key_id = "kennel-maint-2026"
| signature = "...envelope elided in print..."

```

interactive (template) --- the floor made into a usable human shell

Template: interactive — a credible human-driven shell.

A confined but genuinely usable workstation shell: bash + the standard coreutils, text tools, editors, archives, process inspection, and the common network tools, with HOST egress (a human is at the keyboard; direct egress on the host stack, the cloud-metadata invariant still applies). Execution is deny-by-default — the toolset below is exactly what runs; the compiler resolves each tool’s library closure ([lib], inherited).

\$HOME and /tmp are writable (ephemeral); grant a project dir + persistence in your own leaf. For an offline shell, derive base-confined instead (or set net.mode).

interactive (template) --- the floor made into a usable human shell

```
# run it directly:
kennel run interactive -- /bin/bash
# or derive it and add your project:
#   template_base = "interactive"
#   [[fs.write.add]] path = "~/work"
```

```
template_name = "interactive"
template_base = "base-confined"
```

Exec floor composed from the fragment catalogue (05-templates.md §5.10): the shells + POSIX userland + capability bundles, instead of hand-listing them.

```
include = ["core-shell", "core-coreutils", "core-file-mutation",
↪ "core-archive", "net-clients", "vcs-git"]
```

Host egress: a human-driven shell needs unmediated network (curl, git clone, ssh, ...) on the host stack — no SOCKS proxy. This shares the host network namespace and so reinstates the host-recon residual (T1.6), which is why host requires a reason; the cloud-metadata invariant deny + the net.bpf allowlist are still enforced via BPF/Landlock.

```
[net]
mode = "host"
reason = "an interactive human shell needs direct, unmediated egress
↪ (git/ssh/curl) on the host network stack"
```

```
# Re-list base-confined's read baseline (a template's fs.read replaces, not
↪ adds) and
# widen /proc to the whole namespaced procfs so `ps`/`top`/`free` see this
↪ kennel's
# own processes (the PID namespace already hides everything else).
[fs]
read = [
  "/usr/**", "/lib/**", "/lib64/**",
  "/etc/ssl/**", "/etc/pki/**",
  "/etc/ld.so.conf", "/etc/ld.so.conf.d/**", "/etc/ld.so.cache",
  "/etc/alternatives/**", # awk→gawk, vi, editor, pager ... symlink farm
  ↪ (else dangles)
  "/proc/**",
  "/sys/devices/system/cpu/**",
]
```

```
[exec]
shell = "/bin/bash"
# Only the binaries no fragment provides stay inline here; the shell, the
↪ POSIX
# userland, and the toolchains come from the `include` above.
allow = [
    "/usr/bin/clear",
    "/usr/bin/df",
    "/usr/bin/du",
    "/usr/bin/free",
    "/usr/bin/hexdump",
    "/usr/bin/ip",
    "/usr/bin/kill",
    "/usr/bin/nano",
    "/usr/bin/netstat",
    "/usr/bin/od",
    "/usr/bin/pgrep",
    "/usr/bin/pidof",
    "/usr/bin/pkill",
    "/usr/bin/ps",
    "/usr/bin/python3",
    "/usr/bin/scp",
    "/usr/bin/ss",
    "/usr/bin/ssh",
    "/usr/bin/strings",
    "/usr/bin/sync",
    "/usr/bin/timeout",
    "/usr/bin/top",
    "/usr/bin/uptime",
    "/usr/bin/vi",
    "/usr/bin/vim",
    "/usr/bin/vim.basic",
    "/usr/bin/watch",
    "/usr/bin/xxd",
    "/usr/sbin/arp",
    "/usr/sbin/ifconfig",
    "/usr/sbin/route",
]
# net-tools (ifconfig/route/arp) live in /usr/sbin; put it on PATH so they
↪ resolve
# by name. Landlock EXECUTE is independent of PATH (the absolute allow
↪ entries above
# are what grant execution); this is purely so `ifconfig` is found.
path = ["/usr/bin", "/usr/local/bin", "/bin", "/usr/sbin", "/sbin"]
```

```
[lifecycle]
ttl = "12h"
ttl_action = "warn"
```

interactive (template) --- the floor made into a usable human shell

```
[signature]
algorithm = "sshsig"
key_id = "kennel-maint-2026"
signature = "...envelope elided in print..."
```

include = "core-shell"

Fragment: core-shell (POSIX shells)

A composable capability bundle (05-templates.md §5.10): The POSIX shells. base-confined denies all execution and sets `[exec].shell` without granting it, so every interactive or scripted kernel needs the shell explicitly; this is that grant, once, instead of in every template.

```
| include = ["core-shell"]
```

Additive-only (§5.10): every entry is an `[[exec.allow.add]]` delta. Composing this widens the program menu only; `argv[0]` stays gated by the resolved `[exec].allow` under Landlock `execve` default-deny, and the cage (`net/fs/ttl/ceilings`) is untouched.

Adds to the exec floor:

- `/bin/sh` (the POSIX shell, and base-confined's default `[exec].shell`)
- `/usr/bin/sh` (the POSIX shell on a merged-usr system)
- `/bin/bash` (GNU bash)
- `/usr/bin/bash` (GNU bash on a merged-usr system)
- `/usr/bin/dash` (the Debian Almquist shell (`/bin/sh` on Debian/Ubuntu))
- `/usr/bin/env` (the `#!/usr/bin/env <interp>` shebang interpreter (the kernel's `execve` target for `env`-shebang scripts))

include = "core-coreutils"

Fragment: core-coreutils (Read / compute / text userland)

A composable capability bundle (05-templates.md §5.10): The non-mutating POSIX userland: read, inspect, filter, and transform. The block every interactive and agent template repeats. Carries NO filesystem-mutating tool — compose `core-file-mutation` for that — so a read-only kernel includes this alone and cannot write.

```
| include = ["core-coreutils"]
```

Additive-only (§5.10): every entry is an `[[exec.allow.add]]` delta. Composing this widens the program menu only; `argv[0]` stays gated by the resolved `[[exec]].allow` under Landlock `execve` default-deny, and the cage (`net/fs/ttl/ceilings`) is untouched.

Adds to the `exec` floor:

- `/usr/bin/cat` (concatenate and print files)
- `/usr/bin/ls` (list directory contents)
- `/usr/bin/head` (print the first lines)
- `/usr/bin/tail` (print the last lines)
- `/usr/bin/wc` (count lines/words/bytes)
- `/usr/bin/sort` (sort lines)
- `/usr/bin/uniq` (filter adjacent duplicate lines)
- `/usr/bin/cut` (extract columns)
- `/usr/bin/tr` (translate characters)
- `/usr/bin/nl` (number lines)
- `/usr/bin/tac` (concatenate in reverse)
- `/usr/bin/rev` (reverse lines)
- ...and 36 more

include = "core-file-mutation"

Fragment: core-file-mutation (Filesystem-mutating userland)

A composable capability bundle (05-templates.md §5.10): The write-side coreutils: create, move, remove, link, change mode. Kept separate from `core-coreutils` so a read-only kernel cannot mutate while a scratch/build kernel adds it.

```
| include = ["core-file-mutation"]
```

Additive-only (§5.10): every entry is an `[[exec.allow.add]]` delta. Composing this widens the program menu only; `argv[0]` stays gated by the resolved `[[exec]].allow` under Landlock `execve` default-deny, and the cage (`net/fs/ttl/ceilings`) is untouched.

Adds to the `exec` floor:

- `/usr/bin/cp` (copy files)
- `/usr/bin/mv` (move/rename files)
- `/usr/bin/rm` (remove files)

interactive (template) --- the floor made into a usable human shell

- /usr/bin/mkdir (create directories)
- /usr/bin/rmdir (remove empty directories)
- /usr/bin/ln (create links)
- /usr/bin/touch (create/update timestamps)
- /usr/bin/chmod (change file modes)
- /usr/bin/mktemp (create a temporary file/dir)
- /usr/bin/install (copy and set permissions)

include = "core-archive"

Fragment: core-archive (Archive + compression)

A composable capability bundle (05-templates.md §5.10): Tar and the common compressors/archivers — unpack a source tarball, compress an artifact.

```
| include = ["core-archive"]
```

Additive-only (§5.10): every entry is an `[[exec.allow.add]]` delta. Composing this widens the program menu only; `argv[0]` stays gated by the resolved `[[exec]].allow` under Landlock `execve` default-deny, and the cage (`net/fs/ttl/ceilings`) is untouched.

Adds to the `exec` floor:

- /usr/bin/tar (tape archiver)
- /usr/bin/gzip (gzip compress)
- /usr/bin/gunzip (gzip decompress)
- /usr/bin/zcat (stream a gzip file)
- /usr/bin/xz (xz (de)compress)
- /usr/bin/bzip2 (bzip2 (de)compress)
- /usr/bin/zip (zip archiver)
- /usr/bin/unzip (zip extractor)
- /usr/bin/zstd (zstandard (de)compress)

include = "net-clients"

Fragment: net-clients (HTTP(S) fetch clients)

A composable capability bundle (05-templates.md §5.10): The command-line fetch-client BINARIES. Distinct from `net-permissive`, which grants egress `DESTINATIONS`: this adds `curl/wget` to `exec.allow`; the leaf or `net-permissive` says where they may reach, and a `[[net]].mode = "constrained"` cage + the proxy bound that egress.

```
| include = ["net-clients"]
```

Additive-only (§5.10): every entry is an `[[exec.allow.add]]` delta. Composing this widens the program menu only; `argv[0]` stays gated by the resolved `[[exec].allow` under Landlock `execve` default-deny, and the cage (`net/fs/ttl/ceilings`) is untouched.

Adds to the exec floor:

- `/usr/bin/curl` (transfer a URL)
- `/usr/bin/wget` (retrieve files over HTTP(S)/FTP)

include = "vcs-git"

Fragment: `vcs-git`

A composable capability bundle (`05-templates.md §5.10`): `git` plus its per-subcommand helpers, and the system `git` config read-only. Include it for a kernel that does source control.

```
| include = ["vcs-git"]
```

Additive-only (§5.10). This grants the `git` *binaries* and the *system* config; it deliberately does NOT grant network egress — `git-over-HTTPS` to a specific forge is a destination the leaf states (or `net-permissive` adds), and `git-over-SSH` goes through the per-kernel SSH bastion (`[ssh]`, §7.10), never a real key in the kernel.

Adds to the exec floor:

- `/usr/bin/git` (git version-control client)
- `/usr/lib/git-core/**` (git's per-subcommand helper binaries, re-exec'd by `git`)
- `/etc/gitconfig` (system-wide git configuration (read-only))

interactive (policy) --- the thin leaf that pins a workload

Reference policy: `interactive` — the permissive, ready-to-run confined human shell.

Maintainer-signed and shipped enabled: `kennel run interactive` drops you into a confined `/bin/bash` with a broad workstation toolset — the `interactive` base (shells, `coreutils`, file tools, editors, archives, net clients, `git`, `python3`) widened

interactive (policy) --- the thin leaf that pins a workload

with the C toolchain and the development headers — and HOST egress. Deliberately permissive: a credible daily-driver shell, still inside the cage (no_new_privs, dropped caps, deny-setuid, shadowed \$HOME, masked /etc, private /tmp, self-only procs). Derive your own leaf to add a project dir or tighten the toolset.

```
| name = "interactive"  
| template_base = "interactive"
```

Widen the inherited interactive toolset toward a full dev shell: the C toolchain (cc/make/...) and the development headers, on top of the base's shells/coreutils/editors/net-clients/git/python3. The language-registry fragments (lang-python/lang-node) are NOT included — they carry by-name proxy-egress rules host mode rejects; under host egress python3 (inherited) reaches registries directly.

```
| include = [  
|     "toolchain-c",  
|     "dev-headers",  
| ]
```

Host egress (also the interactive default): a human shell needs unmediated network.

```
| [net]  
| mode = "host"  
| reason = "an interactive human shell needs direct, unmediated egress  
| ↪ (git/ssh/curl) on the host network stack"
```

```
| [workload]  
| argv = ["/bin/bash"]
```

```
| [signature]  
| algorithm = "sshsig"  
| key_id = "kennel-maint-2026"  
| signature = "...envelope elided in print..."
```

include = "toolchain-c"

Fragment: toolchain-c

A composable capability bundle (o5-templates.md §5.10): the C/C++ build toolchain — compiler, assembler, linker, archiver, and make. Include it for a kernel that compiles native code; the project tree itself is granted by the leaf, so this fragment adds only the tools.

```
| include = ["toolchain-c"]
```

Additive-only (§5.10). No network and no extra filesystem write: a build reads the toolchain (already on `fs.read` via `base-confined`'s `/usr/**`) and writes only into the project tree the leaf made writable.

Adds to the `exec` floor:

- `/usr/bin/cc` (C compiler driver (the default `cc`))
- `/usr/bin/gcc` (GNU C compiler)
- `/usr/bin/g++` (GNU C++ compiler)
- `/usr/bin/cpp` (C preprocessor (invoked by the compiler driver))
- `/usr/bin/as` (GNU assembler)
- `/usr/bin/ld` (GNU linker)
- `/usr/bin/ar` (static-archive tool)
- `/usr/bin/make` (GNU make)
- `/usr/lib/gcc/**` (gcc's per-version backend binaries (`cc1`, `cc1plus`, `collect2`), re-exec'd by the driver)

include = "dev-headers"

Fragment: dev-headers (Development headers + kernel source)

A composable capability bundle (o5-templates.md §5.10): Adds `/usr/src` and `/usr/include` back into the view (read-only). These are absent from the curated base (construction-by-absence) and only needed by build/compile workloads. Include this fragment in templates that compile C/C++/Rust code against system headers.

```
| include = ["dev-headers"]
```

Additive-only (§5.10): every entry is an `[[fs.read.add]]` delta. Composing this widens the read view only; the cage (`net/fs.write/exec/ceilings`) is untouched.

interactive (policy) --- the thin leaf that pins a workload

Adds to the exec floor:

- /usr/include/** (C/C++ system headers (absent from the curated base))
- /usr/src/** (kernel headers and source trees (absent from the curated base))